

Inference on Graphical Models: Problems and Algorithms

The problems, the algorithms, and the mathematical theorems and properties that validate them

M. J. Wilson

January 1, 2025

Abstract

The inference problems this project poses, the algorithms that answer them, and the mathematical properties each answer is validated against. The problems are stated first and ordered by complexity, from a continuous objective in a handful of coordinates to a lattice, a phylogeny and an iteratively decoded code; the algorithms follow, stated once and cited from every problem they serve; and the validation is a ladder, from enumeration where it is affordable through an inefficient algorithm where it is not to the capable one, scored against both. This is a document about algorithms and not about an implementation: no statement in it depends on how any of it is coded, and the companion paper carries the measurements.

Contents

I	The Problems	6
1	Notation	6
2	Mixed Discrete and Continuous Optimization	6
2.1	Message Passing on Factor Graphs	8
3	The Ladder of Validation	9
	<i>The General Problems</i>	9
4	Continuous Test Functions	9
4.1	The model	9
4.2	Validation	10
4.3	Extensions, and what is not attempted	11
4.4	The algorithms	12
5	The Gaussian Mixture	12
5.1	The model	12
5.2	As a factor graph	12
5.3	Validation	13
5.4	Extensions, and what is not attempted	13
5.5	The algorithms	15

6	The Count-Emission Mixture	15
6.1	The model	15
6.2	As a factor graph	15
6.3	Validation	16
6.4	Extensions, and what is not attempted	16
6.5	The algorithms	17
7	Hidden Markov Models	17
7.1	The model	17
7.2	As a factor graph	18
7.3	Validation	18
7.4	Extensions, and what is not attempted	19
7.5	The algorithms	19
	<i>Application: Phylogenetics</i>	19
8	Phylogenetic Inference	19
8.1	The model	19
8.2	As a factor graph	20
8.3	Validation	20
8.4	Extensions, and what is not attempted	22
8.5	The algorithms	23
9	Large Parsimony	23
9.1	The model	23
9.2	As a factor graph	23
9.3	Validation	23
9.4	Extensions, and what is not attempted	24
9.5	The algorithms	24
	<i>The General Problems, Continued: Spins on a Graph</i>	26
10	Potts Models in an External Field	26
10.1	The model	26
10.2	As a factor graph	26
10.3	Why an odd cycle is the whole story	26
10.4	Validation	27
10.5	Extensions, and what is not attempted	27
10.6	The algorithms	28
11	Frustrated Lattices	28
11.1	The model	28
11.2	As a factor graph	29
11.3	Validation	29
11.4	Extensions, and what is not attempted	31
11.5	The algorithms	31
	<i>Application: Spatial Transcriptomics</i>	31

12 The Coupled Spatio-Sequential Model	31
12.1 The model	31
12.2 As a factor graph	33
12.3 Validation	33
12.4 Extensions, and what is not attempted	36
12.5 The algorithms	36
<i>Application: Error Correction</i>	36
13 LDPC Decoding	36
13.1 The model	36
13.2 A second construction: bicycle codes	37
13.3 The algebraic codes, and a decoder that does not iterate	38
13.4 Capacity, which is nobody's decoder	39
13.5 The quantum code those matrices define	39
13.6 As a factor graph	40
13.7 Validation	40
13.8 Extensions, and what is not attempted	43
13.9 The algorithms	43
14 Turbo Decoding	43
14.1 The model	43
14.2 As a factor graph	44
14.3 Validation	44
14.4 Extensions, and what is not attempted	47
14.5 The algorithms	47
15 Polar Codes	47
15.1 The model	47
15.2 As a factor graph	48
15.3 Validation	48
15.4 Extensions, and what is not attempted	49
15.5 The algorithms	50
16 The Problems, and What Applies to Each	50
 II The Algorithms	 64
17 Factor Graph Optimization	64
18 Initializers	64
19 Relaxing a Discrete Decision	65
20 Surrogates and Bounds	66
21 Learned Proposals for a Discrete Search	67
22 Reading a Comparison	69

23 Algorithms: Continuous Test Functions	70
23.1 The algorithms that apply	70
23.2 The initializers	70
23.3 Choosing among these at the supported sizes	70
24 Algorithms: The Gaussian Mixture	71
24.1 Expectation–maximization and the gradient fit	71
24.2 The k -means++ start	71
24.3 Choosing among these at the supported sizes	72
25 Algorithms: The Count-Emission Mixture	72
25.1 Expectation–maximization and a seeded start	72
25.2 Choosing among these at the supported sizes	73
26 Algorithms: Hidden Markov Models	74
26.1 The forward–backward algorithm	74
26.2 Expectation–maximization	74
26.3 Two decodings, and they are different answers	75
26.4 Choosing among these at the supported sizes	75
27 Algorithms: Phylogenetic Inference	76
27.1 The simulation algorithm	76
27.2 The pruning algorithm	76
27.3 The discrete search	78
27.4 Spectral learning, and what it inverts	79
27.5 Distances, and the moment each inverts	79
27.6 Neighbor joining, and the four-point condition	80
27.7 The Hadamard spectrum	81
27.8 The tropical relaxation	81
27.9 Choosing among these at the supported sizes	82
28 Algorithms: Large Parsimony	84
28.1 Hill climbing on the score	84
28.2 Choosing among these at the supported sizes	84
29 Algorithms: Potts Models in an External Field	85
29.1 The belief propagation algorithm	85
29.2 The transfer matrix	86
29.3 Ground states as cuts	86
29.4 Alpha expansion	87
29.5 The heat bath, Gibbs sampling and cluster moves	87
29.6 Choosing among these at the supported sizes	88
30 Algorithms: Frustrated Lattices	88
30.1 The algorithms that apply	88
30.2 The initializer	89
30.3 Choosing among these at the supported sizes	89
31 Algorithms: The Coupled Spatio-Sequential Model	90
31.1 Block-coordinate ascent	90
31.2 The data-sampled burn-in	91
31.3 Choosing among these at the supported sizes	92

32 Algorithms: LDPC Decoding	92
32.1 Sum-product in the log domain	92
32.2 The all-zero codeword	93
32.3 Choosing among these at the supported sizes	94
33 Algorithms: Turbo Decoding	94
33.1 BCJR and the extrinsic exchange	94
33.2 What pins it	95
33.3 Successive cancellation, and the list	95
33.4 Choosing among these at the supported sizes	95
III Validation	96
34 The Properties That Referee Each Method	96
34.1 Exact answers, where an oracle exists	96
34.2 Structural invariants, where no oracle reaches	97
34.3 Recovery, which is the acceptance test for a fit	97
34.4 Where the likelihood has no maximum, or no curvature	97
34.5 Agreement across implementations is a tolerance	98
34.6 Published constants, and asymptotic results	98
35 Relevance to the Roadmap	98
35.1 Beyond the Roadmap	98
A Continuous Sampling: Hamiltonian Monte Carlo	104
A.1 The sampler	104
A.2 The integrator: leapfrog against Yoshida	104
A.3 Adaptation, as a warm-up	105
A.4 The regime the adaptation sits in	105
A.5 What a warm-up buys	105
A.6 What pins it	106
B Tempering	106
B.1 Schedules, annealing and replica exchange	106
B.2 Setting the ladder from what it measures	108
B.3 Tempering against a sampler, and against restarts	108
B.4 What pins it	108
C Reinforcement Learning	109
C.1 The decision process, and what is exact in it	109
C.2 Dynamic programming: policy iteration and value iteration	109
C.3 Monte Carlo methods	110
C.4 Temporal-difference learning	110
C.5 Value function approximation	111
C.6 Policy gradient	111
C.7 Actor-Critic and Proximal Policy Optimization	112
C.8 Planning with a Learned Prior and Value	113
C.9 What applies to the problems of this document, and what does not	113
C.10 Derivations	114

D	Interval Estimates	114
D.1	A continuous parameter: the observed information	114
D.2	A rate: an interval for a binomial proportion	115
E	Proofs	115
E.1	Pruning is the marginalization over internal states	115
E.2	Forward–backward is the same marginalization on a chain	116
E.3	Sum-product on a tree, and its loopy stationary point	116
E.4	The region-based free energy	117
E.5	The evidence gradient is a posterior expectation	117
E.6	The Bethe free energy is stationary at a sum-product fixed point	118
E.7	The region-based free energy is stationary at a parent-to-child fixed point	118
E.8	Detailed balance for a cluster move in a field	118
E.9	The exchange ratio in parallel tempering	118
E.10	Density evolution on the erasure channel	119
F	Derivations of the Bounds	119
G	Algorithms Cited From the Text	120
G.1	Initializers	122
G.2	Samplers	122
G.3	Optimizers	122
G.4	Surrogates	122

Part I

The Problems

1 Notation

A consistent notation across the problems and the algorithms this text describes, and across the companion paper: one symbol, one meaning. Table 1 states it in two halves. Above the rule are the symbols every section uses; below it are those belonging to one application, which appear nowhere else and carry no meaning outside it.

Two conventions govern what follows. Optimization is minimization of a negative log-likelihood: every objective $\mathcal{L}$ is a quantity to be made small, and a likelihood enters as $-\ln \mathbb{P}(\mathbf{D} \mid \cdot)$. And $\mathcal{L}$ is a function of $\boldsymbol{\theta}$, the unconstrained vector, rather than of the parameters a person names; ϕ connects the two, and Section 17 says why the distinction is not cosmetic. A probability is written $\mathbb{P}(\cdot)$ and its logarithm $\ln \mathbb{P}(\cdot)$, the logarithm being natural throughout.

2 Mixed Discrete and Continuous Optimization

Each problem below couples a *combinatorial* search over a structure to a *continuous* optimization of parameters given that structure. The two are not interleaved steps of one optimizer: a structural move changes what the parameter vector *means* and how long it is, so it constructs a new objective rather than stepping inside an existing one. Everything else here follows from that division.

Symbol	Meaning
<i>The factor graph, its parameters, and its objective</i>	
x_i	a discrete variable of a factor graph, $x_i \in \mathcal{A}$
$\mathcal{A}$	the state alphabet, $ \mathcal{A} = k$
ψ_a	a factor: a non-negative function of the variables $x_{\partial a}$ it touches
$m_{a \rightarrow i}, m_{i \rightarrow a}$	a message from factor to variable, and from variable to factor
b_i, b_a	a belief: the marginal of a variable or a factor, exact on a tree
Z	a partition function; $\ln Z$ the evidence
$\mathcal{G}$	a general undirected graph
$\mathcal{T}$	a tree topology
ρ	the root of a tree
$\mathcal{N}(\cdot)$	the neighbourhood reachable in one step
$\boldsymbol{\theta}$	unconstrained parameters, $\boldsymbol{\theta} \in \mathbb{R}^d$
ϕ	the constraint map, $\phi(\boldsymbol{\theta})$ the feasible parameters
$\mathbf{D}$	the observed data
$\mathcal{L}$	the scalar being minimized
$\mathcal{H}$	a Hamiltonian: the energy of a configuration, script
$\mathbb{P}(\cdot)$	a probability; $\ln \mathbb{P}(\cdot)$ its logarithm, natural throughout
$T, \beta = 1/T$	a temperature and its inverse; a schedule is $T(t)$ over a budget of t steps
$\mathbf{1}[\cdot]$	one where its condition holds and zero where it does not
<i>Statistical physics: spins on a graph</i> (Sections 10, 11)	
σ_i	a spin, the variable of a Potts graph: $x_i \equiv \sigma_i$
J_{ij}	the coupling on the edge $\langle i, j \rangle$
h_i	the external field at node i
$\mathbf{T}$	the transfer matrix between adjacent columns of a lattice
<i>Sequential models: a chain and its emissions</i> (Sections 7, 12)	
z_t	a hidden state, the variable of a chain: $x_t \equiv z_t$
$\mathbf{Z}$	a hidden configuration, $(z_1, \dots, z_T)$
$\mathbf{A}$	a transition matrix
$\mathcal{E}$	an emission family
<i>Phylogenetics: a tree and its substitution model</i> (Sections 8, 9)	
$\mathbf{Q}$	a rate matrix; $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ its transition matrix
$\mathbf{S}$	a symmetric exchangeability matrix
$\boldsymbol{\pi}$	a stationary or initial distribution
$\boldsymbol{\tau}$	branch lengths, in expected substitutions per site
H	the Sylvester–Hadamard matrix, italic
<i>Error correction: a code and its channel</i> (Sections 13, 14)	
$\mathbf{H}$	a parity-check matrix, bold; the codewords are the c with $\mathbf{H}c = 0$
L_i	the channel log-likelihood ratio at bit i

Table 1: The notation of this document and the companion paper. The symbols above the rule are general and mean the same thing in every section; each group below it belongs to the application named beside it, where the general variable x_i is identified with the name that application’s readers arrive with. Three symbols share the letter H and are kept apart by their face: a Hamiltonian is script, a parity-check matrix bold, and the Sylvester–Hadamard matrix italic.

2.1 Message Passing on Factor Graphs

A tree under a substitution model, a Potts model on a graph and a hidden Markov chain are one object: a set of discrete variables $x_1, \dots, x_N$ and a set of factors ψ_a , each a non-negative function of a subset $x_{\partial a}$, with

$$p(x) = \frac{1}{Z} \prod_a \psi_a(x_{\partial a}), \quad Z = \sum_x \prod_a \psi_a(x_{\partial a}). \quad (1)$$

The bipartite graph between variables and factors is the factor graph [45, 41]. In its *normal* (Forney) form every variable is an edge between exactly two factors, a variable shared by more passing through an equality factor $\delta(x, x', \dots)$ [27]; the form changes the drawing and not the distribution, which is checked by computing the same marginals on both. Observations enter as factors: a leaf’s state is an indicator, an emission a unary factor over the hidden state, so the graph is over latent variables alone and a density and a probability are handled alike.

Sum-product sends, from factor a to variable i ,

$$m_{a \rightarrow i}(x_i) = \sum_{x_{\partial a \setminus i}} \psi_a(x_{\partial a}) \prod_{j \in \partial a \setminus i} m_{j \rightarrow a}(x_j), \quad m_{i \rightarrow a}(x_i) = \prod_{b \in \partial i \setminus a} m_{b \rightarrow i}(x_i), \quad (2)$$

and on a tree one leaf-to-root and one root-to-leaf pass are exact (Appendix E.3): on the tree’s factor graph this is pruning, Equation 54; on a chain it is the forward recursion, Equation 51; on a pairwise graph it is Equation 67. On a loopy graph the fixed point is the Bethe approximation of Equation 68. Replacing the sum by a maximum gives max-product, whose max-marginals carry the MAP assignment—on a chain, Viterbi. One implementation of Equation 2 serves every problem class, each pinned to the evaluator that predates it.

The evidence is the objective. Where the factors carry parameters θ , the partition function of Equation 1 is the marginal likelihood of the observations folded into them, $Z(\theta) = \mathbb{P}(\mathbf{D} \mid \theta)$, so maximum likelihood over a model with latent variables *is* maximization of $\ln Z$ — the mixed case in which the discrete variables are summed out and the continuous ones fitted. Its gradient is a posterior expectation,

$$\frac{\partial \ln Z(\theta)}{\partial \theta} = \sum_a \mathbb{E}_{p(x_{\partial a})} \left[\frac{\partial \ln \psi_a(x_{\partial a}; \theta)}{\partial \theta} \right], \quad (3)$$

the sum over factors of the derivative of a factor’s own log-potential averaged under the belief at that factor (Appendix E.5). Two things follow, and both are used throughout. The marginalization that Equation 2 performs is what an optimizer of the continuous parameters needs, so one pass serves both — the expectation step of an expectation–maximization is exactly the beliefs, and the maximization step reads Equation 3. And where the graph is loopy the beliefs are the Bethe approximation of Equation 68, so the fitted optimum is the stationary point of an approximate evidence and is reported as one.

An application enters as a factor. A leaf’s observed character, an emission, a parity check and a gated emission are each one factor of Equation 1 and nothing more; the branch lengths of a phylogeny are the parameters of the pairwise factors along its edges. Each problem statement below says which factors it has, and that statement is the whole of its structure. Equation 2 is validated where each of its instances already has an oracle, and by nothing new: on a tree-shaped Potts graph against enumeration over $q^{|V|}$ configurations, on a chain against the sum over k^T paths, on a tree at one site against pruning, and on a loopy graph against the pairwise belief propagation of Equation 67—the same approximation reached by two codes, which is agreement and not truth. The Forney form is pinned by computing the same marginals on both drawings. An algorithm agreeing only with itself is a second opinion, not an evaluator (Section 34).

3 The Ladder of Validation

Every claim below is refereed, and what referees it is decided by the size of the instance rather than by the method. The arrangement is a ladder, and each problem statement climbs as much of it as its sizes allow.

At the base is *enumeration*: the sum over every configuration, every path, every internal labelling or every topology. It is exponential, which is what makes it an oracle rather than a second opinion, and it fixes the smallest fixtures — a size is chosen because enumeration reaches it, and every enumeration here refuses more than 200,000 states rather than attempting them.

Above it are the *simple, inefficient* algorithms: a transfer matrix over a strip, a direct marginalization that shares no recursion with the recursion it scores, a reference implementation with no acceleration in it. They reach sizes enumeration does not and are exact where they apply, so they carry the referee one rung further at a cost that is polynomial rather than affordable.

Above them are the *capable* algorithms, the ones a result is actually produced with. They are scored against the two rungs below wherever those reach, and past that against each other and against the simulated truth — the generating parameters or structure of a seeded fixture, to a stated tolerance or coverage. Agreement between two capable algorithms is agreement and not truth (Section 34).

Which rung runs when is a second axis, and the two are independent: a check is refereed somehow and separately gated somewhere. Four gates are used, and each problem’s supported sizes name theirs. The *per pull request* tier carries the sizes a merge waits for and is bounded by a stated duration. The *stress* tier carries the same claim one size up, past that bound and inside a developer’s. The *release* tier carries everything, and a check too slow for the first tier is moved here rather than waited for. And the *key* fixture is one instance per problem — the largest declared one whose whole simulate, fit and assert run fits a stated cap — run in full so that at least one instance of every problem is exercised end to end rather than in pieces. A claim moved up a tier keeps a small sibling below it where one exists, and where none does the gap is named rather than left implicit.

A problem states its own ladder in its validation, and the Potts lattice of Section 10 is where all three rungs are visible at once: enumeration of $q^{|V|}$ configurations up to eleven nodes at three states, the transfer matrix over a strip of any length at a width that fits, and belief propagation, sampling and the cut solvers scored against both and then run where neither reaches. What the ladder does not reach is named rather than covered: Section 16 marks each such cell and says which oracle would close it.

The General Problems. The four problems below are stated in the language of Section 2.1 and in nothing else: an objective over a handful of continuous coordinates, a mixture of two kinds, and a chain. None carries an application, and every algorithm any of them uses is general.

4 Continuous Test Functions

4.1 The model

Three standard functions with minimizers known in closed form, so the optimizer of Section 17 is checked against something that is not a likelihood surface: on a likelihood, a fit that stops

early and a parameter the data identify weakly look the same, and these separate the two [57].

$$f_R(x) = \sum_{i=1}^{d-1} \left[b(x_{i+1} - x_i^2)^2 + (a - x_i)^2 \right], \quad x^* = (a, \dots, a), \quad a = 1, \quad b = 100, \quad (4)$$

$$f_{Ra}(x) = 10d + \sum_{i=1}^d \left[x_i^2 - 10 \cos(2\pi x_i) \right], \quad x^* = 0, \quad (5)$$

$$f_H(x, y) = (x^2 + y - 11)^2 + (x + y^2 - 7)^2, \quad (6)$$

the last with four equal minima of value 0, at $(3, 2)$ and three published points. Rosenbrock is a curved valley whose difficulty is conditioning alone; Rastrigin is a quadratic bowl under a cosine ripple with about 10^d local minima, every one satisfying the first-order condition; Himmelblau has no single answer, so a method that always returns the same minimum is reading its start. They are not likelihoods: the Hessian at a minimum is not an observed information, and no interval is built from it. Figure 1 sketches the shape of each.

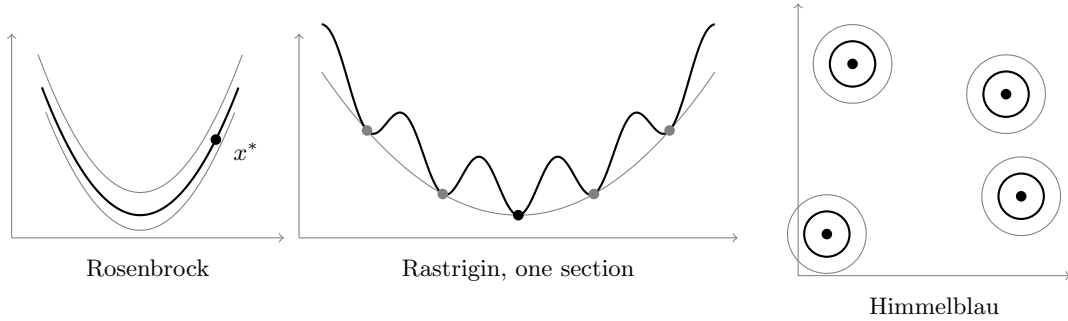


Figure 1: The three surfaces, schematically. Rosenbrock is a single curved valley whose difficulty is conditioning alone, and its minimizer is the marked point of Equation 4. Rastrigin, drawn as one one-dimensional section, is a quadratic bowl under a cosine ripple: the black point is the global minimum of Equation 5 and every grey point a local minimum satisfying the same first-order condition, of which there are about 10^d in d dimensions. Himmelblau has four minima of equal value, Equation 6, so “the” optimum is not a well-formed question and a method that always returns one basin is reading its start. A sketch of the shape of each surface, not a rendering of any of them; the rendered surfaces are Figure 2.

Supported sizes. The size here is the dimension d , and it is the only problem in this document whose oracle costs nothing at any size, the minimizers being closed forms. Rosenbrock is fitted at $d = 2, 3$ and 5 and differentiated against the closed form at 4 ; Rastrigin is fitted at $d = 2$ and carried to 3 and 4 by the checks that need no fit, the roughly 10^d local minima already making a single fit’s success a rate rather than an outcome; Himmelblau is two-dimensional by definition, its four minima published points. There is no per-tier ceiling and no enumeration: what grows with d is not the cost of the referee but the number of basins a multi-start has to find, which is why the fraction of starts reaching the global minimum is the quantity asserted.

4.2 Validation

- *The analytic minimizer* referees a fit directly, to a stated tolerance and in several dimensions for Equation 4. It establishes that the optimizer reaches the right point on a surface where “the right point” is known — which a likelihood cannot offer, and the reason these functions are carried.

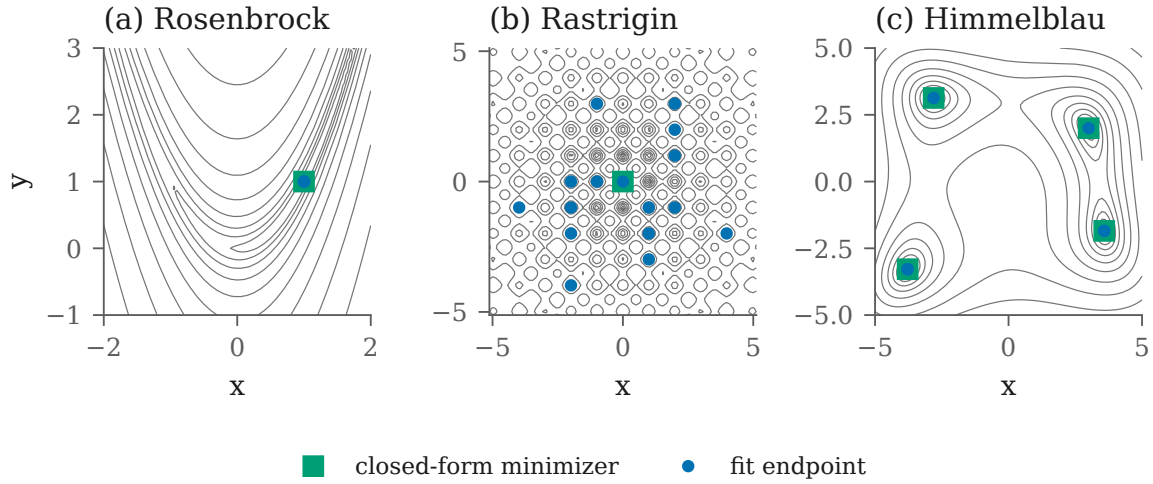


Figure 2: The three continuous test functions in two dimensions, with the minimizers known in closed form and the endpoints of L-BFGS fits from Gaussian random restarts around each function’s own start, the origin for Rastrigin (seed 20260908). (a) Rosenbrock, 4 restarts at scale 1: 4 of 4 reach the minimizer (1, 1). (b) Rastrigin, 16 restarts at scale 2: 1 of 16 reach the global minimum at the origin, every fit stopping at the local minimum of the cell it started in, so a single fit of this surface reports convergence and not optimality. (c) Himmelblau, 8 restarts at scale 3: 8 of 8 reach one of the four equal minima, which one depending on the basin the start fell in. Contours are drawn in $\log(1 + \text{value})$.

- *The hand-written closed-form gradient* referees the automatic one exactly. It establishes the differentiation, separately from the optimization, so a failure of one is not read as a failure of the other.
- *Each of Himmelblau’s four minima* is required to be reachable from its own basin. It establishes that the multi-start reports a spread rather than one answer; a method that always returns the same minimum is reading its start, and that is the failure this function exists to expose.
- *A rate on Equation 5* — the fraction of starts from which a converged fit is the global minimum — is asserted *low* rather than asserted to succeed. It establishes the bound on what a single fit of any multimodal surface here may claim; it establishes nothing about a particular run.
- *An equal budget of objective evaluations* is what compares restarts against an annealed chain with a polishing fit. It establishes which wins on this surface at that budget, and, as with the frustrated lattices, nothing general about either method.

4.3 Extensions, and what is not attempted

What it would take to go further. The comparison here is restarts of a local fit against an annealed chain; the method it has not been run against is a population one, an evolution strategy adapting a covariance from its own samples being the standard baseline for a rugged continuous surface, scored at the same budget of objective evaluations [34]. Two reporting extensions matter more than they sound. A rate over starts is a statistic with an interval, and stating the interval is what makes two methods separable [1]; and the seeds, the budget and

the number of runs behind a rate are part of the claim rather than its provenance [35]. Both apply to every equal-budget comparison here, and this is the surface where they can be checked against a known answer.

What is deliberately not attempted. Constraints, which would make the constraint map of Section 17 the thing under test rather than the optimizer. A noisy objective, since these functions are carried because their value is exact and a fit that stopped early is distinguishable from a parameter that is not identified. And high dimension: what grows with d is the number of basins, so a larger d makes the rate smaller and the measurement no sharper.

4.4 The algorithms

Section 23 states them: the quasi-Newton fit of Section 17 under the identity constraint map, the multi-start of Algorithm 8, the Hamiltonian sampler of Appendix A read as an energy, and the annealing of Appendix B on a schedule. Nothing here is specific to these surfaces, which is why they referee the machinery the likelihood problems share.

5 The Gaussian Mixture

5.1 The model

Independent observations $y_1, \dots, y_N$, each drawn from one of K components chosen by a weight vector,

$$p(y_i) = \sum_{k=1}^K w_k \mathcal{N}(y_i; \mu_k, \sigma_k^2), \quad \sum_k w_k = 1, \quad w_k > 0, \quad (7)$$

with the component label z_i latent and retained by the simulator so that a clustering has something to be checked against. It is the hidden Markov model of Section 7 with the chain removed: the Gaussian family is the same emission family, and the likelihood inherits its lack of a maximum (Section 34) unchanged.

Supported sizes. K components in single figures, N observations from hundreds to hundreds of thousands, and one or two channels. The simulator is checked at two components over 200,000 draws, where a component frequency's standard error is under 0.0011; the budgeted comparison of methods runs five components at 1.5 standard deviations of separation over 500 observations at the release tier. **The refereed size is one-dimensional and the largest is not.** The exact clustering cost that referees a seeding is a dynamic programme over cut positions in the sorted observations, so it exists only on the line: in higher dimension the same problem is NP-hard and Equation 50 would have nothing to be checked against. A component may instead emit several channels, independent given the label, so that a mixture matched in shape to the projection of Section 12 can be stated; that instance carries no oracle and is refereed by the planted labels and the generating parameters. Enumerating the K^N labellings is out of reach at every size here, so the responsibilities are refereed by the hidden Markov model's own recursion instead.

5.2 As a factor graph

N variables $x_i \equiv z_i$ with one unary factor each, $\psi_i(z_i) = w_{z_i} \mathcal{N}(y_i; \mu_{z_i}, \sigma_{z_i}^2)$, and no pairwise factor at all. Equation 2 is then a normalization per variable, and the belief at i is the responsibility

$$r_{ik} = \frac{w_k \mathcal{N}(y_i; \mu_k, \sigma_k^2)}{\sum_{k'} w_{k'} \mathcal{N}(y_i; \mu_{k'}, \sigma_{k'}^2)}, \quad (8)$$

the object an HMM’s forward–backward pass produces with a recursion and a mixture with a division. The M step receiving the same object either way lets one emission family serve both, and Figure 3 is Figure 6 with the transition factors removed.

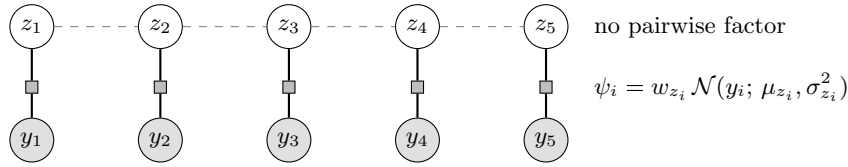


Figure 3: The Gaussian mixture as a factor graph: N component labels z_i over K values, one shaded unary factor each folding in one observation y_i , and no pairwise factor at all. The dashed lines mark where the transition factors of Figure 6 would sit; removing them is exactly what turns the hidden Markov model into the mixture, and it turns Equation 2 into the per-variable normalization that is Equation 8. The same emission family serves both, and receives the same responsibilities from either. A sketch of the structure and not a rendering of any instance.

5.3 Validation

- *The hidden Markov model’s M step on the same responsibilities* referees the mixture’s, at equality. It establishes that one emission family serves both models; being two of ours it is agreement, not truth.
- *The gradient fit of the same likelihood* referees expectation–maximization at its fixed point, and monotonicity referees every iterate. The two share the model and nothing else, which makes their agreement evidence; neither establishes that the shared optimum is global, and where the components overlap the fixed point is approached linearly, so an iterate stopped at a count sits above its basin’s optimum by an amount a polish removes.
- *The exact clustering cost*, a dynamic programme over the cut positions of the sorted observations, referees the seeding of Equation 49 and is the one oracle this section’s initializer has. It establishes that the mean cost over seedings satisfies Equation 50; it says nothing about the likelihood, and whether a seeded start reaches a better optimum than a uniform one is measured over seeds and reported whichever way it falls.
- *Recovery up to the permutation of components* is the referee for the fit itself, with the alignment part of the test. It establishes calibration on data from the model.
- *The variance floor* is checked as a refusal in both algorithms: a component collapsing onto one observation sends the likelihood to infinity, so the fit refuses rather than reports it (Section 34).

5.4 Extensions, and what is not attempted

What it would take to go further. Raising the dimension costs the initializer’s only oracle: the exact clustering cost is a dynamic programme over cut positions in sorted observations and has no counterpart above one dimension. Replacing it needs either a certified bound on the seeding cost that does not need the optimum, or a regime where the planted labels alone referee and the section says so — Section 6 already does the second at ten components. The unbounded likelihood is the other direction: a prior on the scales, or a penalized objective, gives a maximum that exists rather than a floor that refuses [15, 53]. Choosing K rather than declaring it is model selection, and the enumerable instance is where a criterion for it could be refereed.

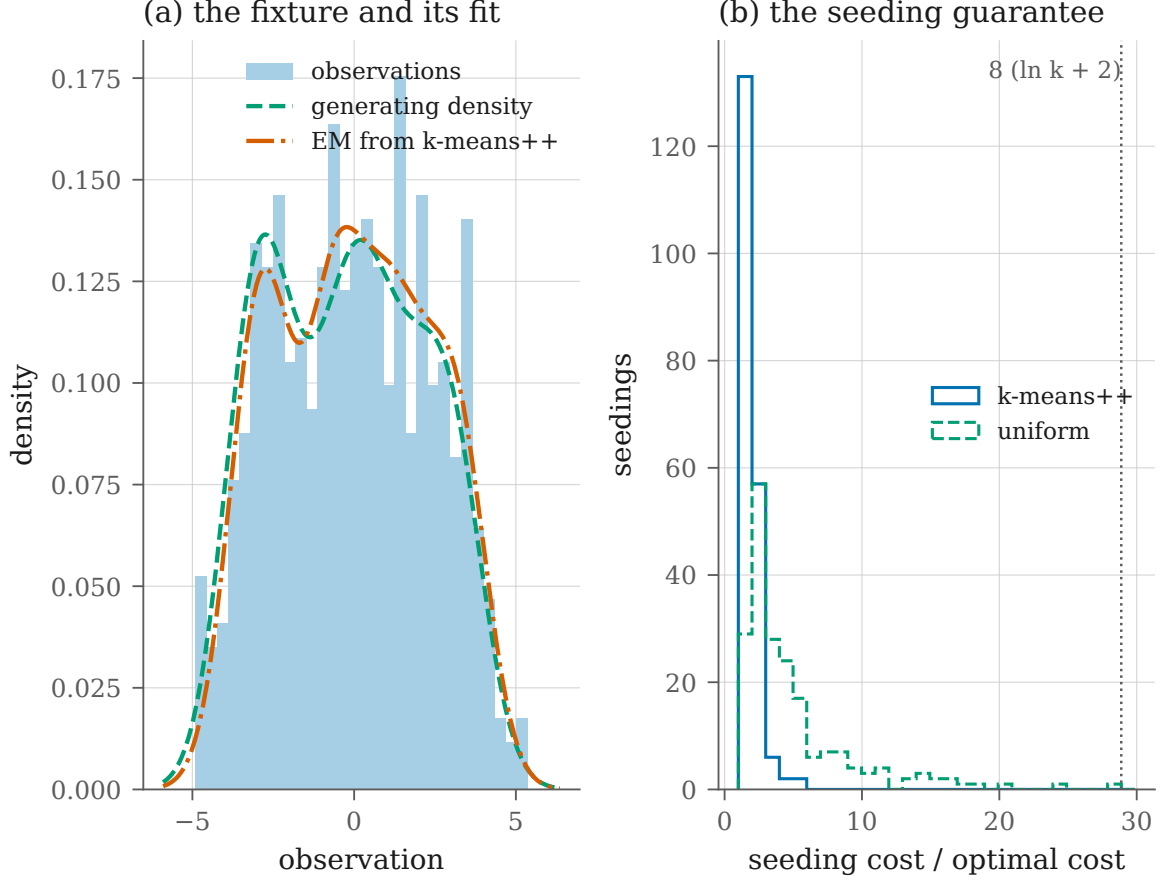


Figure 4: A Gaussian mixture of 5 components with means $-3, -1.5, 0, 1.5, 3$, unit scales and weights $0.3, 0.1, 0.25, 0.15, 0.2$: 500 observations at seed 20260908. (a) The observations, the generating density, and the density expectation-maximization reaches from one k-means++ start at the 500-iteration cap, still moving, at log-likelihood -1122.3 . (b) The k-means cost of a seeding divided by the exact optimal cost from the one-dimensional dynamic programme, over 200 independent seedings of the same observations each way: k-means++ averages 1.90 (worst 5.61) and uniform seeding 4.95 (worst 28.68), against the published bound of 28.88 on the k-means++ expectation. The bound is a statement about the mean, and the mean is what is held to it.

What is deliberately not attempted. A component count fitted from the data, a non-diagonal covariance, and any dimension above one — this section exists to referee an emission family and an initializer against exact quantities, and each of the three removes one. The collapsing component is kept: it is the standing example of a likelihood with no maximum, and regularizing it away would lose the case Section 34 needs.

5.5 The algorithms

Section 24 states them: the alternation of Algorithm 15 with no chain in it, the gradient fit over the unconstrained vector of Section 17, and the k -means++ seeding of Algorithm 6, which is the initializer of Section 18 in its data-sampled kind.

6 The Count-Emission Mixture

6.1 The model

Section 5’s mixture with its component family exchanged, which is the point: the mixture asks its components for a density and an M step and nothing else, so a family over a different observation drops in unaltered. Here the observation is a *pair*—a total count n_i and the successes y_i within it, a sequencing depth and its allele count—and each component is a negative binomial for the total together with a beta-binomial for the successes,

$$p(n_i, y_i) = \sum_{k=1}^K w_k \text{NB}(n_i; r_k, \mu_k) \text{BB}(y_i; m_{ik}, a_k, b_k), \quad \sum_k w_k = 1, \quad w_k > 0, \quad (9)$$

with the component label z_i latent and retained by the simulator. The trial count m_{ik} carries the whole difference between the family’s two forms and admits no default: it is a per-component constant $m_{ik} = m_k$ in the *independent* form, where the two channels factorize given the component, and it is the drawn total $m_{ik} = n_i$ in the *joint* form, where a deeper site carries a proportionally larger allele count. The joint form gives $y_i > n_i$ probability zero, so a pair outside the support is scored at $-\infty$ rather than left to the negative argument of a $\log \Gamma$.

Two properties of Section 34 are inherited rather than restated. The likelihood is bounded above by zero, both channels being discrete, so the mixture has no counterpart of the Gaussian’s collapsing component. What it has instead is two *flat* directions: the dispersion r_k toward the Poisson limit and the concentration $a_k + b_k$ toward the binomial one, each with a bound above which this much data cannot tell one value from infinity, and an estimate that reaches its bound is reported as a bound rather than raised on.

Supported sizes. K components in single figures; N observations in the hundreds to the low thousands. The declared instances are $K = 3$ at $N = 900$, where the posterior over labellings is enumerable, and $K = 10$, which is the size at which the fit stops being routine. The cost per iteration is not the mixture’s but the family’s: both shape parameters are solved for by bisection rather than by formula, at a few thousand score evaluations per component per M step, so the iteration count and not the sample count is what a budget is spent on. Enumerating the K^N labellings is out of reach at every declared size, and the enumeration oracle therefore runs over a prefix: the observations are independent, so the marginal at each of ten of them is the same quantity the responsibilities of the whole dataset carry, and 3^{10} is affordable where 3^{900} is not.

6.2 As a factor graph

N variables $x_i \equiv z_i$ with one unary factor each, $\psi_i(z_i) = w_{z_i} p(n_i, y_i \mid z_i)$, and no pairwise factor: Figure 5 is Figure 3 with a pair at each observation instead of a scalar. Equation 2 is again

a normalization per variable and the belief at i is again Equation 8, now with Equation 9’s density in place of the Gaussian’s. That the posterior factorizes across observations is what the enumeration oracle checks, and it is the property every step below rests on.

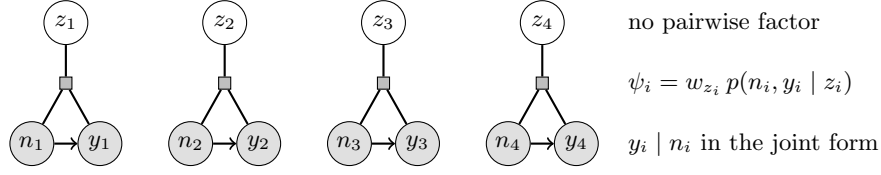


Figure 5: The count-emission mixture as a factor graph. It is Figure 3 with one change: the observation attached to each unary factor is a *pair*, a total n_i and the successes y_i within it. The arrow between them is the joint form’s conditioning—the beta-binomial’s trial count is the drawn total—and is absent in the independent form, where the trial count is a parameter and the two channels factorize given the component. The mixture itself is unchanged either way: Equation 8 is still one normalization per variable, and the M step is still the family’s own. A sketch of the structure and not a rendering of any instance.

6.3 Validation

- *The enumerated posterior over component labellings* referees the E step at the $K = 3$ instance. An exhaustive sum sharing no code with the normalization it checks, it establishes that the posterior factorizes across observations—the property the method rests on and the one an implementation can quietly break. It is available over a prefix only, and says nothing at the $K = 10$ instance.
- *The component M step is the emission family’s own*, unchanged from the hidden Markov case and from the Gaussian mixture, and the family carries its own referees: its density sums to one over the support, and its estimates recover planted parameters at a stated size. Being two of ours it is agreement, not truth.
- *Recovery of the planted mixture*, up to the permutation of components, is the referee for the fit. The permutation is resolved by ordering on the depth and the allele fraction rather than by enumerating it, since at ten components that enumeration is 10! and the ordering is exact wherever the instance separates its components—which a declared instance does by construction.
- *The seeded start against the uniform one*, on the same seeds and the same budget, is a comparison of two of ours and establishes nothing general; it is reported whichever way it falls, on the terms Section 5 states for k -means++.
- *An estimate at its identifiability bound* is reported and not raised on: the dispersion and the concentration each have a value above which the data cannot distinguish the family from its limiting case, and a fit that reaches one has found a bound rather than a maximum (Section 34).

6.4 Extensions, and what is not attempted

What it would take to go further. The flat directions are the sharpest gap. An estimate reaching its identifiability bound is reported as a bound, and saying more needs a profile likelihood in that direction — the curvature being absent, the interval must come from the objective’s own shape rather than from the observed information Section 17 pushes through the delta method. The M step is the second: both shape parameters are found by bisection, and a closed form

for either would move the cost of an iterate by the factor Section 25.2 says the budget is spent on. The third is structural and built one section over: a chain over the component labels is the coupled model of Section 12, so this section carries the same family with its sequential structure removed.

What is deliberately not attempted. More than two channels, since the trial count is what ties the pair and a third observation would need its own statement of how it is tied rather than a default. A dispersion shared across components, which would make the flat directions a single one and change what an identifiability bound means. And the K^N enumeration at the declared sizes: the prefix is the oracle, it establishes the factorization and nothing else, and extending it is not affordable at any size in this section.

6.5 The algorithms

Section 25 states them: the same alternation, Algorithm 15, over an emission family whose maximization step is itself an optimization, and the family-aware seeding that replaces the squared distance of Algorithm 6 by the family’s own divergence.

7 Hidden Markov Models

7.1 The model

A hidden path $\mathbf{Z} = (z_1, \dots, z_T)$ over k states evolves by an initial distribution $\boldsymbol{\pi}$ and a transition matrix $\mathbf{A}$; each position emits an observation from an emission family $\mathcal{E}$ indexed by its state.

A *batch* of such paths is not one longer path. Where the data is segmented into S chains of lengths $T_1, \dots, T_S$, each chain restarts at $\boldsymbol{\pi}$ rather than continuing through $\mathbf{A}$, so the batch takes $\sum_s (T_s - 1) = T - S$ transitions and not $T - 1$. The segmentation is therefore part of the model and not a detail of its storage: at a fixed total T , a single chain and twenty short ones assign different probabilities to the same observations, because a state change *across* a boundary is free where the same change one position earlier costs $\log A_{ij}$. A segment of one position carries an initial distribution and no transition, and is excluded as degenerate. The evidence marginalizes k^T paths.

Supported sizes. The exact referee is that k^T path enumeration, refused past 200,000 paths, so the oracle reaches eleven positions at three states and seventeen at two. The fixture is three states, four symbols and fifteen positions over 600 sequences: long enough that a fit recovers the transition matrix, short enough that any window enumerates. Six emission families run against the same recursions, exact at every length since the chain is a tree; the release tier fits sequences past the enumerable length, refereed by recovery of the simulated parameters alone, up to the permutation of hidden states.

Chains of unequal length. A batch need not be rectangular. Writing $y_{1:T_r}^{(r)}$ for the r th sequence, the likelihood of R sequences of lengths $T_1, \dots, T_R$ is the product

$$p(\mathbf{D} \mid \boldsymbol{\theta}) = \prod_{r=1}^R p(y_{1:T_r}^{(r)} \mid \boldsymbol{\theta}), \quad (10)$$

each factor the forward recursion of Equation 51 run to its own length T_r . Nothing in the model couples the sequences, so this is a statement about the data and not a second model: the parameters are shared and the lengths are not.

Padding is an implementation of Equation 10 and never a model. A batch padded to its longest member and masked computes the same product, and what it costs is the padding: the

work grows as $R \cdot \max_r T_r$ where the sum $\sum_r T_r$ is what the likelihood needs. The two agree to floating point and the gap between them is the padding fraction, which is a property of the batch rather than of either route. An unmasked padded position is a different likelihood, so every consumer masks — in the expectation step and in the emission re-estimation alike.

7.2 As a factor graph

The variables are the hidden states, $x_t \equiv z_t$. The factors are $\psi_1(z_1) = \pi_{z_1} \mathcal{E}_{z_1}(y_1)$ at the first position and, for $t > 1$, one pairwise $\psi_t(z_{t-1}, z_t) = \mathbf{A}_{z_{t-1}z_t}$ per transition and one unary $\mathcal{E}_{z_t}(y_t)$ per emission: the observation y_t is folded into the table, so a probability and a density enter the same way and the graph is over latent variables only. The chain is a tree, and Z is the evidence $p(\mathbf{D})$. An interior state sits on three factors, so in the Forney form it passes through an equality node—the drawing Section 12 uses for each class’s chain. Figure 6 is that structure on five positions.

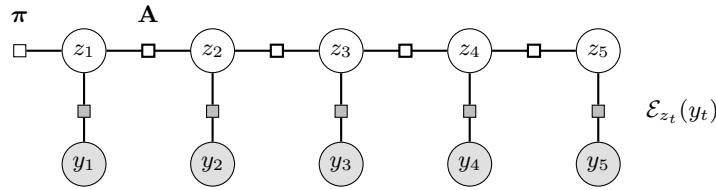


Figure 6: A hidden Markov model as a factor graph. Open circles are the hidden states z_t over k values, open squares the transition factors $\mathbf{A}$, the leftmost square the initial distribution π , and shaded squares the emission factors of the family $\mathcal{E}$, each folding in one observation y_t (shaded circle). The chain is a tree, so Equation 2 is exact and its two passes are Equation 51 and Equation 52. Removing the transition factors leaves the Gaussian mixture of Section 5; keeping them and gating the emissions on a spatial label gives the coupled model of Section 12. A sketch of the structure and not a rendering of any instance.

The emission family is the only part that knows what an observation is. Everything else—the recursions, the expectation step, path enumeration—needs three things from it: draw an observation given a state, score one, and re-estimate itself from posterior weights. Separating it lets one set of recursions serve a finite alphabet, a real observation, and a count.

A density is not a probability. For a family over a countable alphabet the evidence is a probability, so $\log p(\mathbf{D}) \leq 0$. For a family over the reals it is a *density* and carries no such bound. A check written against the first is false under the second, so what may be asserted is keyed on the declared support.

7.3 Validation

- *The sum over all k^T paths*, written out term by term with no recursion, referees the evidence, both decodings and every posterior at the lengths above. It establishes the recursions exactly; separating the two decodings, it cannot itself be checked by a decoder, and is pinned against quantities worked out by hand on a two-state, two-step chain.
- *Central differences* referee the gradient past the enumerable length. They establish that the differentiable path agrees with the recursion’s own value; they establish nothing about that value.
- *Monotonicity* referees expectation–maximization at every iterate. It establishes that the M step is an ascent, which is the one property the algorithm guarantees; it does not establish

that the fixed point is the maximum, which is why the fixed point is also required to agree with an independent gradient fit of the same likelihood.

- *Recovery up to the permutation of hidden states* is the referee where no oracle reaches. It establishes calibration on data from the model, the alignment an explicit part of the test rather than an accident of labelling; on a family whose M step is itself an optimization it also pins the boundary refusals, a dispersion the data cannot identify being refused and not clamped.
- *The declared support* decides what may be asserted at all: for a family over a countable alphabet the evidence is a probability and $\log p(\mathbf{D}) \leq 0$; for a family over the reals it is a density and carries no such bound. A check written against the first is false under the second.

7.4 Extensions, and what is not attempted

What it would take to go further. The chain has no missing oracle inside its ceiling and none past it, so the extensions are about the *start* and the *uncertainty* rather than the recursion. A method-of-moments start would give the fit what the distance start gives the tree: the observable moments of short windows invert to the parameters with a consistency guarantee and no surface to climb [38], and the argument carrying it to a tree [52] is cited in Section 27.4, which makes the chain the easiest case to referee. A posterior over parameters rather than a point estimate — variational, or sampled by Section A’s chain — would replace the delta-method interval whose asymptotics Section 17 is explicit about [15, 53]. And choosing the state count rather than declaring it is a model-selection problem this document does not pose.

What is deliberately not attempted. Continuous time, which replaces the transition matrix by a generator and the recursion’s product by a matrix exponential — the tree already carries that arithmetic, and duplicating it on the chain would buy no new referee. A semi-Markov duration model, whose state is not the position’s alone, so the factor graph is no longer the chain of Section 7 and the path enumeration that referees everything here does not apply. And an unbounded state space, since every oracle in this section counts k^T .

7.5 The algorithms

Section 26 states them: the two-pass marginalization of Equation 51 and Equation 52, which is Equation 2 on a chain; the alternation that fits it, Algorithm 15; the two decodings, of which Equation 53 is one; and the block draw of a whole path from its conditional, Algorithm 12.

Application: Phylogenetics. The two problems below are the factor graph of Section 2.1 on a tree, and the tree is what an application supplies. Everything application-specific in this document — a substitution model, a taxon, a parsimony score — enters here, in Section 12, and in Sections 13 and 14; the sections between them carry none of it.

8 Phylogenetic Inference

8.1 The model

Character evolution along a branch is a continuous-time Markov chain with rate matrix $\mathbf{Q}$; the transition probabilities over a branch of length t are the matrix exponential $\mathbf{P}(t) = \exp(\mathbf{Q}t)$ [25].

A time-reversible model factors as $\mathbf{Q} = \mathbf{S} \text{diag}(\boldsymbol{\pi})$ with $\mathbf{S}$ symmetric, and then satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$.

The simplest such model is Jukes–Cantor on k states: every substitution equally likely and $\boldsymbol{\pi}$ uniform, with $\mathbf{Q}$ scaled so that one unit of branch length is one expected substitution per site,

$$-\sum_{i \in \mathcal{A}} \pi_i \mathbf{Q}_{ii} = 1, \quad (11)$$

which puts the transition probabilities in closed form [25, 21]:

$$\mathbf{P}_{ij}(t) = \frac{1}{k} + \left(\delta_{ij} - \frac{1}{k} \right) \exp\left(-\frac{k}{k-1} t \right). \quad (12)$$

Rows sum to one, $\mathbf{P}(0) = \mathbf{I}$, and every entry tends to $1/k$ as $t \rightarrow \infty$; the general time-reversible model replaces the single rate by $\mathbf{S}$ and $\boldsymbol{\pi}$, keeping Equation 11.

Given a topology $\mathcal{T}$ over n taxa, branch lengths $\boldsymbol{\tau}$, and an alignment $\mathbf{D}$ of L sites, sites evolve independently and the likelihood marginalizes every assignment z of states to the internal nodes:

$$p(\mathbf{D} \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \prod_{s=1}^L \sum_z \pi_{z_\rho} \prod_{(u \rightarrow v)} \mathbf{P}_{z_u z_v}(\tau_v), \quad (13)$$

the product over the branches $(u \rightarrow v)$ of the rooted tree with the leaves fixed by the data— k^{n-2} terms per site for a binary tree. Summing it directly is the oracle every faster evaluation is checked against.

Supported sizes. The declared range is $n \in [10, 1000]$ taxa by $L \in [100, 11,000]$ sites, and the largest evaluation measured is 200 taxa by 11,000 sites. What is *refereed* is smaller, the referee being an enumeration: the $(2n-5)!!$ unrooted topologies number 15 at five taxa, 105 at six, 945 at seven and 10,395 at eight, so the per-pull-request tier carries the five- and six-taxon fixtures, the stress tier seven, and the release tier eight and the twenty- and fifty-taxon trees that only the distance start referees. Direct marginalization over the k^{n-2} internal assignments of Equation 13 referees the likelihood on the same fixtures at $k = 4$ and L from 200 to 200,000 sites. The Hadamard start is offered from three to twelve taxa, its spectrum having 2^{n-1} entries, and every enumeration here refuses more than 200,000 configurations rather than attempting them.

8.2 As a factor graph

One site is one instance of Equation 1. The variables are the states $x_u \equiv z_u \in \mathcal{A}$ at every node u of the rooted tree, leaves included. The factors are the root prior $\psi_\rho(x_\rho) = \pi_{x_\rho}$, one pairwise factor per branch, $\psi_v(x_u, x_v) = \mathbf{P}_{x_u x_v}(\tau_v)$ for u the parent of v , and one indicator per leaf, $\psi_\ell(x_\ell) = \mathbf{1}[x_\ell = \text{observed}]$, which folds the data in. The bipartite graph is a tree, so Equation 2 is exact, and Z is the site likelihood $p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$; sites being independent (Equation 13), the alignment’s likelihood is the product of L such Z . In the Forney form every internal node’s state is an edge through an equality factor of degree three—two branches and, at the root, the prior—which is the drawing Section 12 uses. Figure 7 is that structure on four taxa.

8.3 Validation

Each referee below establishes one thing and is silent about the rest; the list is what the section may claim.

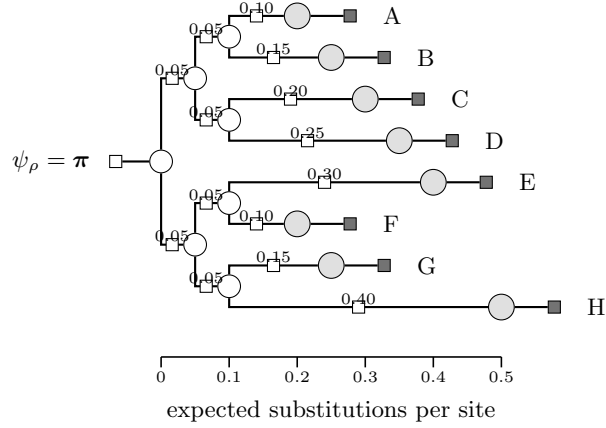


Figure 7: One site of the eight-taxon release fixture as a factor graph, drawn at the fixture’s own branch lengths. Open circles are the states of the internal nodes and shaded circles the states at the leaves; the small square on each branch is the pairwise factor $\mathbf{P}(\tau_v)$ of Equation 12, labelled with that branch’s length; the filled square at each leaf is the indicator folding in the observed character; and the square at the root is the prior π . Horizontal position is the sum of the branch lengths above a node, in the unit the axis carries, so the drawing is the topology and the lengths at once. The bipartite graph is a tree, so Equation 2 is exact and its leaf-to-root pass is Equation 54. Figure 8 is the same fixture with the characters this structure generates at its leaves.

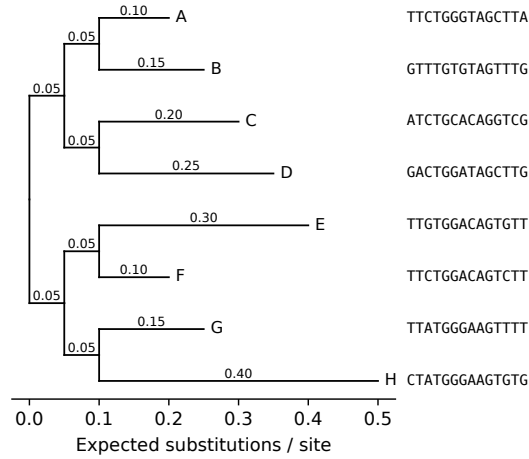


Figure 8: Simulated topology for 8 taxa under the Jukes-Cantor model (seed 20260904), branch lengths labelled in expected substitutions per site. The first 14 simulated sites are shown beside each leaf, in tree order, so the alignment the likelihood consumes can be read off against the topology that generated it.

- *Direct marginalization* over the k^{n-2} internal assignments, Equation 13, shares no traversal with Equation 54 and establishes that the recursion computes the marginal it claims, to a relative tolerance. It does not reach past the taxon count the supported sizes give, and says nothing about the rescaling at lengths where the unscaled product would underflow.
- *The closed form* Equation 12 referees the simulator through the empirical substitution frequencies along a branch. It establishes that the generative direction is the model; it does not check the likelihood, since a simulator checked against the evaluator fitted to its output referees nothing.
- *The pulley principle* — a reversible model’s likelihood is unchanged by where the root sits along a branch — and the gradient against central differences hold past every enumerable size. They are invariants, not answers: both are satisfied by a wrong likelihood that is wrong reversibly.
- *Topology enumeration* of the $(2n - 5)!!$ candidates referees the search, which must reach the enumerated maximum, and the neighbourhood counts $2(n - 3)$ and $2(n - 3)(2n - 7)$ are asserted exactly over every topology. Past those sizes nothing here says the climb succeeded, which is what a branch-and-bound bound would supply.
- *Recovery of the simulated truth* — the generating topology at a stated site count, and branch lengths covered by their intervals at the nominal rate — is the referee where no oracle reaches. It establishes that the pipeline is calibrated on data from its own model and nothing about data the model does not describe.
- *The path lengths of a known tree* referee neighbor joining exactly on an additive matrix, and Atteson’s radius bounds when an estimated matrix still recovers the topology. Outside that radius the start is a start and not an estimate, and only the fit that follows it is reported.

8.4 Extensions, and what is not attempted

What it would take to go further. The gap is the one Table 10’s last row names: past the enumerable taxon count nothing says a climb found the optimum. What closes it is a *bound*, not a faster climb — a branch and bound over the topology space, with Equation 42 or the plug-in of Equation 41 pruning a subtree of the search [58]. Three further directions each have a prerequisite. A posterior over topologies rather than a flat-prior weight over maximized likelihoods needs a chain that mixes over the move graph, whose mixing is studied rather than assumed [80, 2]; a differentiable topology needs a parameterization of the space itself, which the subsplit constructions supply and which would give Section 27.8 an oracle it lacks [84, 85]; and a learned ranking of the regraft neighbourhood is the surrogate Section 20 ranks with, learned rather than derived [8]. Richer substitution models — rate heterogeneity across sites, a general time-reversible matrix fitted rather than fixed — change the evaluator’s parameter vector and nothing about its referees [82]. Established implementations are a comparison not made here [51, 44]: they are surveyed, and admitting one is a decision on its own evidence.

What is deliberately not attempted. Empirical alignments, since there is no true tree behind one and a result on it can be reported but not refereed — the reason every fixture is simulated. Model misspecification, since every recovery statement here is calibration on the model’s own samples. Insertions and deletions, which break the site independence every referee in Section 8.3 rests on and would need a different likelihood rather than a longer one. And the four-state Hadamard conjugation over the Klein four-group: the two-state transform with the exact pairing reduction is carried, the general one stated and left unbuilt.

8.5 The algorithms

Section 27 states them: the pre-order simulation of Algorithm 1; the post-order marginalization of Equation 54 and the adjoint that differentiates it, Algorithm 2; the climb over a topological neighbourhood, Algorithm 14; the distance, agglomeration and spectral starts of Section 27.4, which are Algorithm 3 and Algorithm 7; and the tropical relaxation of Section 27.8.

9 Large Parsimony

9.1 The model

Parsimony is the phylogenetic criterion with no model: a topology is scored by the fewest state changes that explain the alignment, Equation 57 under unit costs and Equation 58 under a step matrix, with no branch lengths and no probabilities, so the score is an integer where the likelihood is a real number. *Small* parsimony scores a given tree; *large* parsimony is the search,

$$\mathcal{T}^* = \arg \min_{\mathcal{T}} F(\mathcal{T}), \quad F(\mathcal{T}) = \sum_{s=1}^L F_s(\mathcal{T}), \quad (14)$$

over the $(2n - 5)!!$ unrooted topologies [25]. NP-hard in general, and stated as the discrete half of Section 8 with the continuous half removed, so the same neighbourhoods and budget accounting serve it.

Supported sizes. The same sizes as Section 8, with the ceiling set by the same $(2n - 5)!!$ enumeration — five and six taxa per pull request, seven at the stress tier, eight at the release tier, where 10,395 topologies each cost one post-order pass rather than one optimization. A candidate being that much cheaper is the point: at an equal budget of candidates scored, the parsimony climb reaches a neighbourhood an order of magnitude larger than the likelihood climb, and the trade is that the criterion is the wrong one in the region Figure 10 names. The brute force over every labelling of the internal nodes is k^{n-2} per site and obeys the same 200,000-state refusal.

9.2 As a factor graph

One site is the tree factor graph of Section 8 in the min-plus semiring (Figure 9): the pairwise factor on a branch is the cost of a change, 0 on agreement and W_{ij} otherwise, a leaf's indicator is 0 at its observed state and ∞ elsewhere, and the root prior is constant. Max-product with the sum replaced by a minimum and the product by a sum is Sankoff's recursion, and under the unit matrix Fitch's set recursion carries the same minimum. The criterion is therefore the short-branch limit of Equation 54, and Equation 42 is where that limit re-enters the likelihood problem, as an upper bound on $\ell^*(\mathcal{T})$ a search can rank a neighbourhood by.

9.3 Validation

- *A brute force over every labelling of the internal nodes*, which shares no traversal with either recursion, referees the score at exact equality. It establishes both recursions on a given tree; it says nothing about the search.
- *Fitch against Sankoff under the unit step matrix*, on every topology of a fixture, referees the two implementations against each other. It establishes that the set recursion and the cost recursion are the same criterion; being two of ours, it is agreement and not truth, which the brute force above supplies.
- *Enumeration of Equation 14's domain* referees the climb from every start where it reaches. It establishes that the search found the minimum-score topology at those taxon counts.

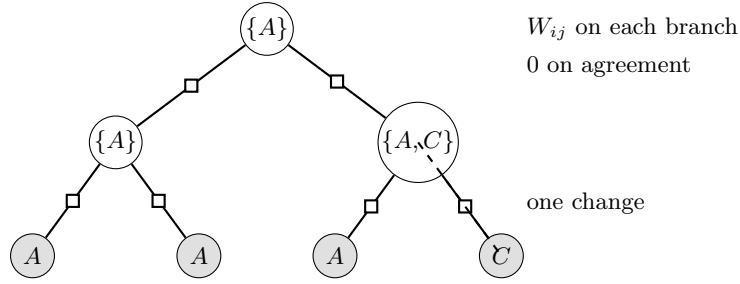


Figure 9: One site of large parsimony as the same tree factor graph in the min-plus semiring. Shaded circles are the observed characters, open circles the Fitch sets of Equation 57 carried up the tree, and squares the branch factors, whose table is 0 on agreement and W_{ij} otherwise (Equation 58). The right-hand cherry disagrees, so its parent takes the union and the site's score is one change, drawn dashed on the branch that pays it. The structure is Figure 7 with the probabilities replaced by costs and the branch lengths removed; the search over the $(2n - 5)!!$ topologies is what makes the problem large parsimony. A sketch of the structure and not a rendering of any instance.

- *The Felsenstein-zone fixture* referees the criterion rather than the code: its enumerated parsimony optimum is asserted to be the wrong tree while the likelihood optimum on the same alignment is the right one. It establishes that the implementation reproduces a known statistical inconsistency, the one case a merely broken implementation cannot pass, and nothing about accuracy outside that zone, for which the Farris-zone pair is kept.
- *The metric condition on the step matrix* is checked as a refusal, not as a result: an asymmetric matrix scores each rooting of an unrooted topology differently and is refused rather than scored at an arbitrary root.

9.4 Extensions, and what is not attempted

What it would take to go further. This is the one search here where an exact method past enumeration is plausibly affordable. The score is an integer and decomposes over sites, so a partial tree admits a lower bound on every completion and a branch and bound can discard a whole subtree of the topology space without scoring it [58, 19]; a likelihood offers no such bound, which is why an exact search would be built on the criterion first. A weighted criterion is the other direction: the step matrix is declared here, and estimating one from the alignment — subject to the metric condition the search needs — would make the criterion a fitted model rather than a fixed one. Compound moves are the third, and they belong to Section 21: a sacrifice and its payoff over this neighbourhood are two decisions whose credit an option would carry as one [74].

What is deliberately not attempted. A correction for the inconsistency of Figure 10. The Felsenstein zone is carried as a property of the criterion and asserted against, not repaired: the repair is the likelihood, which is Section 8 and already here. Ancestral-state uncertainty, since the criterion returns a minimum and not a posterior, and the object that carries uncertainty over internal states is the marginalization the likelihood already does. And an asymmetric step matrix, which is refused rather than scored at an arbitrary root.

9.5 The algorithms

Section 28 states them: the same climb, Algorithm 14, with one post-order pass in place of the fit, over the neighbourhoods Section 27.3 defines.

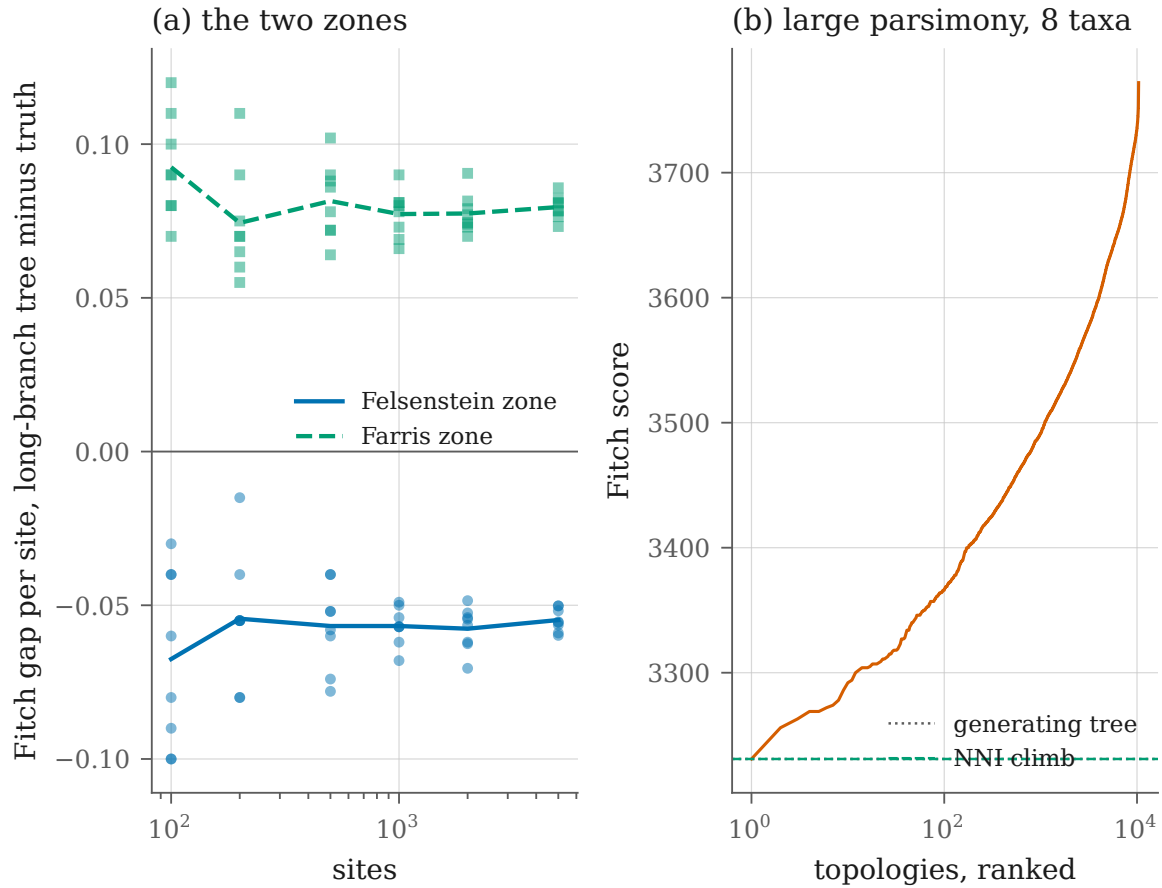


Figure 10: Parsimony in the two zones, and large parsimony on the eight-taxon fixture. (a) Four taxa, pendant branches 0.75 and 0.02 expected substitutions per site with internal branches 0.02, 4-state Jukes-Cantor, seed 20260904, 8 independent alignments at each of 6 site counts from 100 to 5000. Each marker is the Fitch score of the tree grouping the two long branches minus the generating tree's, per site; the line is the mean. In the Felsenstein zone the long branches are not a cherry and the mean gap is -0.068 at 100 sites and -0.055 at 5000: negative, so the criterion prefers the wrong tree, and not closing, which is statistical inconsistency. In the Farris zone the same branches are a cherry and the gap is 0.092 and 0.080. (b) Every one of the 10_395 unrooted topologies on the 8-taxon fixture, scored at 2000 sites (seed 20260904) and ranked; the generating tree scores 3231 and is the minimum. A nearest-neighbour-interchange climb from a random start reaches 3231 after scoring 26 candidates of a 400 budget, the generating tree; the enumeration is the oracle that says so.

The General Problems, Continued: Spins on a Graph. A Potts model in an external field is the general discrete problem at every size, and a frustrated lattice is the instance that separates the searches for it. Neither carries an application; both are cited by the two that do.

10 Potts Models in an External Field

10.1 The model

On a graph $\mathcal{G}$, each node i carries a spin $\sigma_i \in \{1, \dots, q\}$ and the energy is

$$\mathcal{H}(\sigma) = - \sum_{\langle i, j \rangle} J_{ij} \delta(\sigma_i, \sigma_j) - \sum_i h_i(\sigma_i), \quad (15)$$

with couplings J_{ij} and an external field h_i [50]. The Boltzmann distribution is $p(\sigma) \propto \exp(-\mathcal{H}(\sigma))$, and its normalizer sums over $q^{|V|}$ configurations.

Supported sizes. The exact referee is the $q^{|V|}$ enumeration, refused past 200,000 configurations, which puts the ceiling at eleven nodes at three states and seventeen at two. The fixtures sit under it: a chain of twelve sites at three states over 400 replicates, and two open 3×3 lattices at three states, one in a shared field and one in a per-site field, whose $3^9 = 19,683$ configurations give the exact normalizer and every marginal. Above it the transfer matrix referees a strip of any length at a width that fits — which is where the 12×6 per-site-field instance sits, at 3^{72} configurations — belief propagation is compared on the 8×8 lattice where nothing is exact, the minimum cut is checked at extents 8, 12 and 16, and the cluster moves are separated from single-site sweeps at extents 8 and 24, the 12×12 of them at the square-lattice transition of Section 29.5 being a declared instance. The largest per-site-field instance is a triangular 71×71 at ten states, 5,041 nodes, where no normalizer is available at any cost and what is asserted is that the covariate still tilts the labels. The general graph carries $G(n, p)$ instances to a hundred nodes, refereed by the planted energy of Section 11 rather than by any enumeration.

10.2 As a factor graph

The variables are the spins, $x_i \equiv \sigma_i$. The factors are one unary $\psi_i(\sigma_i) = e^{h_i(\sigma_i)}$ per node and one pairwise $\psi_{ij}(\sigma_i, \sigma_j) = e^{J_{ij} \delta(\sigma_i, \sigma_j)}$ per edge, so Equation 1 is the Boltzmann distribution and Z its normalizer. At a temperature T the factors are raised to the power $\beta = 1/T$, which is the same graph with J and h scaled; nothing else about the model changes, and that is why a tempered chain can be held to the unscaled model's enumeration. The graph is a tree exactly when $\mathcal{G}$ is, which is the regime every exact claim below is confined to. Figure 11 is that structure on the square lattice.

10.3 Why an odd cycle is the whole story

A two-state antiferromagnet ($J < 0$) wants every edge to disagree, possible exactly when $\mathcal{G}$ is 2-colourable. So every bipartite lattice—a chain, a square lattice—has a trivial ground state and says nothing about a hard problem. Odd cycles make the ground state non-trivial, and where a counting argument closes it fixes that ground state at every size: the difference between a fixture that exercises an optimizer and one that does not.

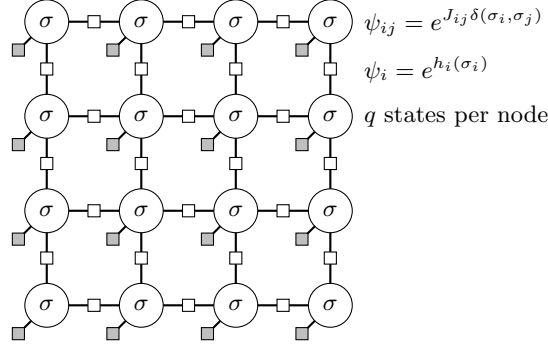


Figure 11: A Potts model in an external field as a factor graph, on the open 4×4 square lattice. Circles are the spins $\sigma_i \in \{1, \dots, q\}$; open squares on each edge are the pairwise factors of Equation 15 and shaded squares on a stub are the unary field factors. The graph has cycles, so Equation 2 on it is the Bethe approximation of Equation 68 and not the exact marginal; the square lattice is two-colourable, so it also carries no frustration, which is what Section 11 adds. A sketch of the structure and not a rendering of any instance.

10.4 Validation

- *Enumeration of the $q^{|V|}$ configurations* referees $\log Z$, every marginal, and the ground state at the sizes above. It establishes exactness there and nothing at all past them, which is why every claim about a larger lattice is either a structural invariant or a measurement.
- *The transfer matrix* referees the normalizer of a strip independently of the message passing, so a chain and a narrow lattice have an oracle past enumeration. It establishes agreement at a width, not a bound on the error at any width.
- *Zero coupling* is where belief propagation is exact on any graph. It establishes that the loopy implementation reduces correctly; the deviation away from it is reported as a measurement and asserted against nothing, since asserting agreement with the truth on a loopy graph would assert something false.
- *A minimum cut* referees the two-state submodular ground state exactly at any size max-flow reaches, and alpha expansion must reproduce it at two labels. It establishes the multi-label solver's construction; it does not establish the multi-label optimum, whose bound Equation 72 states as a theorem while the realized ratio is measured beside it.
- *A goodness-of-fit test against the enumerated Boltzmann distribution*, thinned by several autocorrelation times, referees a sampler. It establishes the distribution the chain converges to at an enumerable size; it establishes no mixing time, and its power is itself pinned by a paired negative — the same move set with the field accept step removed runs, mixes, and is rejected.
- *The unscaled model's enumeration*, with exchanges on, referees every replica's marginal in parallel tempering. It establishes that the exchange ratio does not contaminate the cold replica; whether tempering beats restarts at an equal budget is a property of the instance and is measured on the frustrated instances of Section 11, never asserted in general.

10.5 Extensions, and what is not attempted

What it would take to go further. The missing referee is an oracle for the normalizer of a lattice wider than the transfer matrix reaches. A boundary contraction supplies one: a strip's partition function is a product of transfer operators, and contracting it approximately

with a controlled bond dimension turns the width from a hard ceiling into a stated accuracy [65]. Two bounds would extend what can be claimed without an oracle — the tree-reweighted upper bound of Equation 45 has a maximum-a-posteriori counterpart bounding the ground state rather than $\log Z$ [79], and the characterization of which energies a cut can minimize says which multi-label instances reduce to the exact solver rather than to expansion [42]. For the search itself, population annealing carries an ensemble down a temperature schedule with resampling and is the natural comparison for Section B’s tempering at equal sweeps [46]. A learned ground-state proposal is admissible on the same terms as any other: a graph network trained to output a configuration is a method [67], and the demand that it be measured against a competently tuned greedy baseline is part of the method rather than a criticism [3].

What is deliberately not attempted. Continuous spins, which change the enumeration from a sum to an integral and leave every exact referee in Section 10.4 without a counterpart. A pairwise term that is not a metric on labels, since the expansion bound of Equation 72 assumes one and an instance that violates it is refused rather than approximated. And the thermodynamic limit: the critical couplings are used to *place* a finite instance, and no quantity here is claimed of the infinite lattice.

10.6 The algorithms

Section 29 states them: the message iteration of Equation 67 and the free energy of its fixed point; the minimum cut and the expansion cycle built on it, Algorithm 13; the single-site and cluster updates, Algorithms 9 and 10; and the replica exchange over them, Algorithm 11, whose ladder is Appendix B.

11 Frustrated Lattices

11.1 The model

Two instances of Equation 15 whose ground-state energy is known from outside, admitted because more than one algorithm is scored against them. The first is the two-state antiferromagnet, $J < 0$, on the periodic triangular lattice: N sites, $3N$ bonds, $2N$ triangles, every bond in exactly two triangles. A triangle is not two-colourable, so each keeps at least one agreeing bond, and counting (triangle, agreeing bond) pairs gives $2A \geq 2N$ for the agreeing bonds A of any configuration; with a zero field and $J = -|J|$,

$$E_{\min} = |J| N, \quad (16)$$

attained by many configurations: the residual entropy per site tends to Wannier’s 0.3231 in the thermodynamic limit, a value reported for orientation and never asserted at a finite size. The second is a planted Viana–Bray spin glass: couplings $\pm J$ on a sparse random graph, built around a state σ^* by setting each coupling to reward what the state does and then inverting a fraction f of them, so that with u inverted bonds

$$E(\sigma^*) = -J (|E| - 2u), \quad E_{\min} \leq E(\sigma^*), \quad (17)$$

a known energy at any size and an upper bound on the ground state, never the ground state itself: at $f = 0$ the instance is a gauge transform of the ferromagnet and trivial, and as f grows the planted state stops being near-optimal (Figure 13).

Supported sizes. Both instances state their answer at every size, so unlike every other section here the size is not bounded by an oracle. The periodic triangular lattice is carried at 3×3 , 3×4 , 4×4 and 9×9 ; the first three are inside the $2^{|V|}$ enumeration, which attains Equation 16

and measures its degeneracy — 42 ground states at 3×3 , 68 at 3×4 and 42 again at 4×4 , where neither extent divides by three — and the fourth is past it, where the same closed form still holds. The planted spin glass is carried from ten to a hundred nodes at mean connectivity three and four, with the inverted fraction f swept: at $f = 0$ the instance is a gauge transform of the ferromagnet and measures nothing, and the useful range is where the planted state stops being near-optimal.

11.2 As a factor graph

The factor graph of Section 10 with $\psi_{ij} = e^{J_{ij}\delta(\sigma_i, \sigma_j)}$ and $J_{ij} < 0$ on some or every bond. Nothing changes in the graph; what changes is that the odd cycles of Section 10 are now present, so the ground state is not a colouring and the bond probability $1 - e^{-\beta J_{ij}}$ of the cluster moves is not a probability. Figure 12 is the counting argument drawn on one triangle.

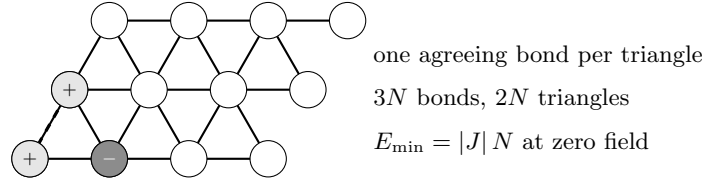


Figure 12: A patch of the periodic triangular antiferromagnet. Every bond wants its endpoints to disagree, and a triangle is not two-colourable, so at least one bond per triangle agrees — the dashed bond of the labelled triangle, where $+$ and $-$ are the two states. Counting (triangle, agreeing bond) pairs over the $2N$ triangles and $3N$ bonds gives Equation 16, a ground-state energy known at every size and attained by many configurations. The planted spin glass of Equation 17 is the same picture on a sparse random graph with a fraction of the couplings inverted. A sketch of the structure and not a rendering of any instance.

11.3 Validation

- *Equation 16*, a counting argument, referees the triangular ground state at every size. It establishes the minimum energy and that a search attaining it has attained the optimum; it does not identify *which* ground state, the optimum being massively degenerate, so the residual entropy is reported for orientation and never asserted at a finite size.
- *Enumeration over 2^N configurations* referees the same energy where it reaches, and the degeneracy with it. It establishes that the closed form and the code agree at the sizes both cover.
- *Equation 17* referees every search on the glass as an *inequality*: the planted energy is an upper bound on the ground state, never the ground state. It establishes that a search returning a higher energy has failed; it cannot establish that one reaching the planted energy has succeeded, and at a large inverted fraction a search is expected to go below it.
- *The refusals* are checked as results: the exact cut refuses a negative coupling, the energy not being submodular, and the cluster moves refuse it because $1 - e^{-\beta J}$ is not a probability there. They establish that the boundary between exact and approximate is drawn in the code and not only in the text.
- *An equal budget of sweeps* is what compares descent, annealing and tempering here. It establishes which wins *on this instance*, at that budget, over shared seeds; it establishes nothing general about the algorithms, which is why the comparison is reported as an experiment.

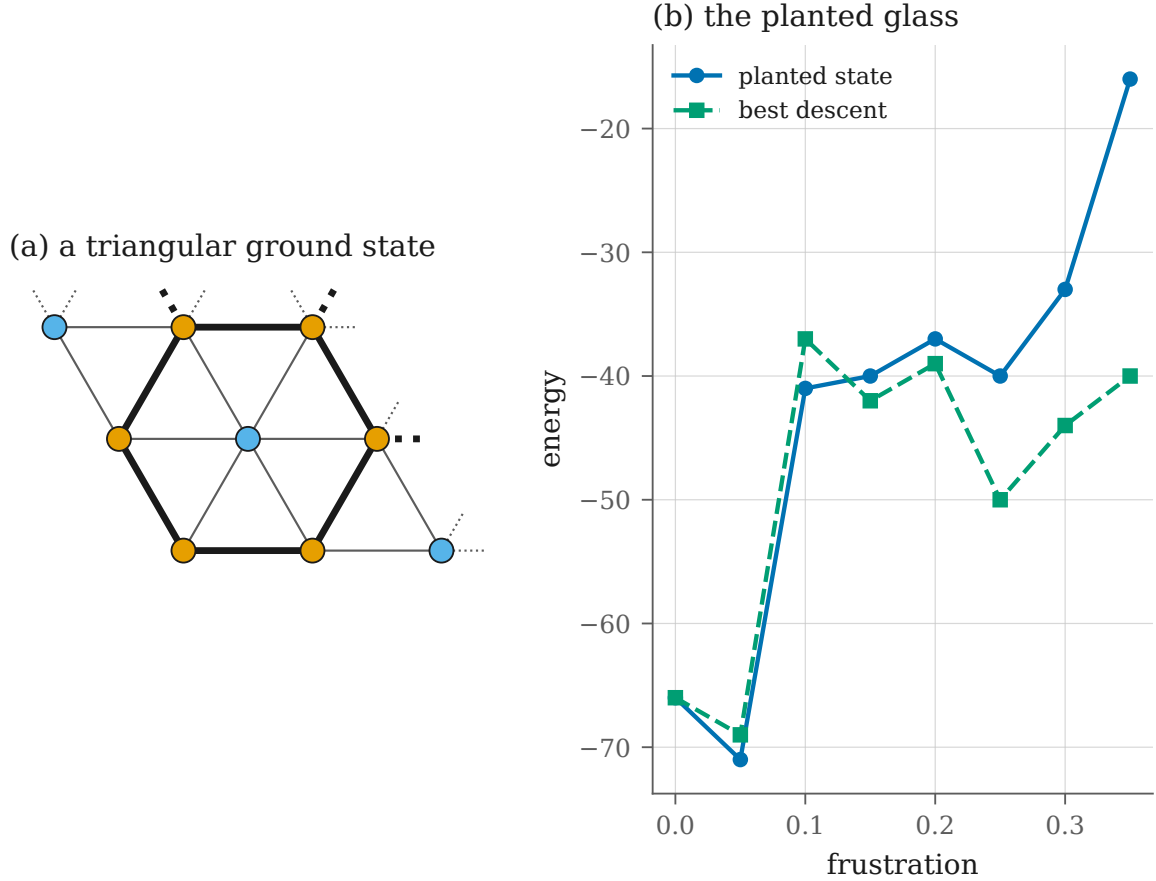


Figure 13: Two frustrated instances with an answer from outside. (a) The 3×3 periodic triangular Ising antiferromagnet, coupling -1 , 27 bonds (dotted: bonds wrapping the torus), with one of its enumerated ground states coloured by spin. Heavy bonds join agreeing spins: 9 of 27, one in three, which the double count over triangles fixes as the minimum at every size, so the ground-state energy is 9 exactly. (b) A planted Viana-Bray spin glass on 60 sites at mean degree 4, couplings of magnitude 1, one instance per frustration from seed 20260908: the energy of the planted state against the lowest energy reached by 20 single-site descents from random starts. Descent lands on the planted energy at 3 of 8 frustrations and goes below it at 5, where the planted state is an upper bound on the ground state and not the ground state.

11.4 Extensions, and what is not attempted

What it would take to go further. These instances are where a search is scored with no oracle, so what would extend them is anything turning a comparison into a certificate. The semidefinite relaxation of Equation 71 already computes one: its optimal value bounds the maximum cut from above on the instance in hand [32, 17], so reporting it beside a search’s result would convert “the best energy found” into a bracket. Population annealing is the search-side extension — an ensemble carried down a schedule with resampling rather than replicas exchanged along one [46] — and the comparison Section B would be made against at equal sweeps. A learned proposal is admissible on the terms Section 10.5 states, and these instances, having a known answer at every size, are where it would be measured.

What is deliberately not attempted. A disorder average. Every statement here is about one instance at one seed; averaging over the ensemble of couplings is a different quantity needing a different accounting, and the inverted fraction is swept so a per-instance result is reported with the disorder it was measured at. A phase diagram, for the reason the thermodynamic limit is out of Section 10: the residual entropy is quoted for orientation and asserted at no finite size. And the exact ground state of the glass, unavailable at any size here.

11.5 The algorithms

Section 30 states them, and they are Section 29’s unchanged: enumeration and the maximum cut where they reach, the semidefinite relaxation at any size, single-site descent, which is Algorithm 9 at zero temperature, and the annealing and replica exchange of Algorithm 11.

Application: Spatial Transcriptomics. The coupled model joins three of the general problems above into one graph, and is where a lattice, a chain and a mixture are fitted together.

12 The Coupled Spatio-Sequential Model

12.1 The model

Observations x_{sn} are indexed by a spatial node n of a graph $\mathcal{G}$ and a sequential position $s = 1, \dots, S$. Each node carries a class label $\ell_n \in \{1, \dots, M\}$; each class m carries its own hidden chain $k_{s,m} \in \{1, \dots, K\}$; and the observation at (s, n) is emitted by the chain of the class the node belongs to, under that class’s emission parameters θ_m . The joint is

$$\begin{aligned} \log p(\ell, k, x) = & \beta \sum_{\langle n, n' \rangle} J_{nn'} \delta(\ell_n, \ell_{n'}) - \log Z_{\text{Potts}} \\ & + \sum_m \left[\log \pi_m(k_{1,m}) + \sum_{s>1} \log A_m(k_{s-1,m}, k_{s,m}) \right] \\ & + \sum_n \sum_s \log P(x_{sn} \mid k_{s,\ell_n}, \theta_{\ell_n}), \end{aligned} \quad (18)$$

Equation 1 with pairwise, chain and degree-two gated factors, and no fourth code path.

Two of these terms admit a quantity the model conditions on rather than fits. An emission may be scored against a per-observation *covariate* c_{sn} — what the observation was measured against, an exposure for a rate or a number of trials for a bounded count — so the emission term reads $\log P(x_{sn} \mid k_{s,\ell_n}, \theta_{\ell_n}, c_{sn})$. A family whose observation has several channels takes one

covariate per channel, because the channels condition on different kinds of quantity and a single value names neither. Setting c_{sn} to each channel’s neutral value recovers the uncovared model exactly; the neutral value is a family’s to define and is not one for every channel. A covariate that varies and is not conditioned on is not ignored — it is absorbed, and the fit recovers the average of what it should have separated.

The chain term likewise admits $A_m(k_{s-1,m}, k_{s,m})$ varying with s : a kernel per transition rather than one for the chain. It carries $(S-1)K(K-1)$ free values against $S-1$ transitions per chain, so it is conditioned on and not estimated, and what a caller assembles it from — a distance between positions, a product over two latents — is outside this statement. Its three terms are the three problem classes above: a Potts prior on the labels,

$$\log p(\ell) = \beta \sum_{\langle n, n' \rangle} J_{nn'} \delta(\ell_n, \ell_{n'}) - \log Z_{\text{Potts}}, \quad J_{nn'} \geq 0, \quad (19)$$

ferromagnetic, so that neighbouring nodes prefer one class. Written as a penalty on disagreement it is $-\beta \sum J_{nn'} \mathbf{1}[\ell_n \neq \ell_{n'}]$, the same distribution up to a constant. Next, one hidden Markov chain per class, in the simplest case with a circulant transition,

$$\mathbf{A}_m(i, j) = t \delta_{ij} + \frac{1-t}{K-1} (1 - \delta_{ij}), \quad (20)$$

whose self-transition rate t and initial distribution Π_m are fitted; and a *gated* emission, $\mathbf{1}[\ell_n = m] \log p(x_{sn} | k_{s,m}, \theta_m)$, which couples the two. Marginalizing k recovers a Potts model in a field the chains define; marginalizing ℓ recovers a mixture of hidden Markov models. Neither marginal is tractable alone, and the structure of the joint dictates the estimator.

Supported sizes. The chains are per class and not per spot: each of the N spots carries one label $\ell_n \in \{1, \dots, M\}$, and each of the M classes carries one chain of S states drawn from K , shared by every spot the labels assign to it. So the labels contribute M^N and the chains K^{SM} , and the joint has $M^N K^{SM}$ states. The count is *not* K^{SN} , which is the different model in which every spot owns a private chain; there the classes would carry no information and the coupling this section is about would not exist. The gated emission above is what makes the distinction observable: a spot reads the profile of its own class, so two spots sharing a label share a profile exactly. The enumerable instance is therefore small: the canonical one is a 2×2 open lattice at $M = K = 2$ over $S = 6$ positions, $2^4 \cdot 2^{12} = 65,536$ joint states, inside the 200,000-state refusal and with its emissions separated far enough that a draw at $S = 6$ identifies the classes. Past it the model runs on the lattices of Section 10 at the sizes stated there. Under a categorical emission the instance stays small — a 10×10 lattice at $M = K = 2$ over four positions — refereed by the planted labels alone, the cell Section 16 marks as wanting an oracle. Under the paired count emission of Section 6 the sequential extent grows: an 8×8 lattice at $M = K = 2$ over 1,000 positions per pull request, and a triangular 71×71 at $M = K = 10$ over 20,000 positions past it. There the reference implementation of the posterior is the oracle, an accelerated one pinned to it, and the planted labels referee the fit above that.

The count-pair instance, and what a covariate conditions on. Under the paired count emission a spot’s observation at a position is two counts, a total and a successes, and each channel may carry a *covariate*: a per-observation quantity the model conditions on rather than fits. The two channels take different ones, and the difference is the whole of the contract:

- an *exposure* $c_{s,n} > 0$ on the total channel multiplies the rate the count is drawn at, so a negative-binomial total at mean μ becomes one at mean $c_{s,n}\mu$. The identity that makes this a conditioning and not a reparameterization is $c\Gamma(r, \mu/r) = \Gamma(r, c\mu/r)$: the shape is untouched, so the dispersion the fit estimates is the same parameter it would estimate at unit exposure;

- a *trial count* $m_{s,n}$ on the successes channel *replaces* the declared number of trials, so the beta-binomial is scored out of what was actually observed rather than out of a constant.

A covariate that varies and is not conditioned on is not neutral: the fit then estimates the rate averaged over the exposures. On the fixture below, planted rates of 24 and 72 with exposures spanning a factor of sixteen are recovered as 23.3 and 71.3 when the fit is told them, and as 47.9 and 151.7 when it is not — factors of 1.99 and 2.11 against the $\mathbb{E}[U(0.25, 4)] = 2.125$ the exposures average to.

Two instances are declared. The per-pull-request one is an 8×8 lattice at $M = K = 2$ over 1,000 positions; past it a triangular 71×71 at $M = K = 10$ over 20,000 positions, which is 1.008×10^8 count pairs and past every oracle in this document — there the reference implementation is the referee for the accelerated one, and the planted labels for the fit.

Chains of unequal length. The chains of Equation 18 need not share a length. Class m carries S_m states and the joint is the same product with S replaced by S_m per class, exactly as Equation 10 states for a batch of sequences: the classes are coupled through the labels, not through their lengths. Padding to the longest chain and masking computes the same joint and is an implementation of it, never the model — and what it costs is the padding fraction of the batch, not a property of either route.

12.2 As a factor graph

The variables are the N labels and the SM chain states. The factors are the pairwise Potts factors of Section 10 on the labels, the chain factors of Section 7 on each class's states, and one gated emission factor per (n, s, m) over the pair $(\ell_n, k_{s,m})$, equal to $p(x_{sn} | k_{s,m}, \theta_m)$ when $\ell_n = m$ and to one otherwise. A label sits on every emission factor of its node and on every Potts factor of its edges; in the Forney form it passes through an equality node of that degree, and the gated factors are the higher-degree nodes joining the lattice to the chains (Figure 14). That structure is for a labelling, which Figure 15 shows on a lattice past enumeration, planted beside recovered. The graph has cycles, so Equation 2 is the Bethe approximation on it, and the exact quantities below come from enumeration—a 2×2 lattice with $M = 2$, $K = 2$, $S = 3$ has $2^4 \cdot 2^{12}$ joint states.

12.3 Validation

- *The joint written out* on every assignment of the enumerable instance referees the constructed factor graph's log-density, at equality. It establishes that the three adapters compose into the model Equation 18 states; it establishes nothing about any inference on it.
- *Enumeration* of the same instance referees the simulator's label and state marginals and the per-class posteriors. It establishes exactness at $2^4 \cdot 2^{12}$ states and stops there.
- *Central differences* referee the gradient identity Equation 75. It establishes the M step's derivative, not the E step's posterior, which the enumeration above covers separately.
- *Monotonicity of the joint across every block*, for each of the three label solvers, referees the ascent. It establishes that no block decreases the likelihood; it does not order the solvers, and cannot, since a monotone ascent to a worse fixed point is still monotone.
- *The planted labels*, recovered up to permutation at an equal budget of sweeps, are the referee for the label solvers past enumeration. They establish which solver reaches the planted labelling on this fixture — where the measurement says it is the start and not the move that decides it — and they do not establish that the planted labelling is the

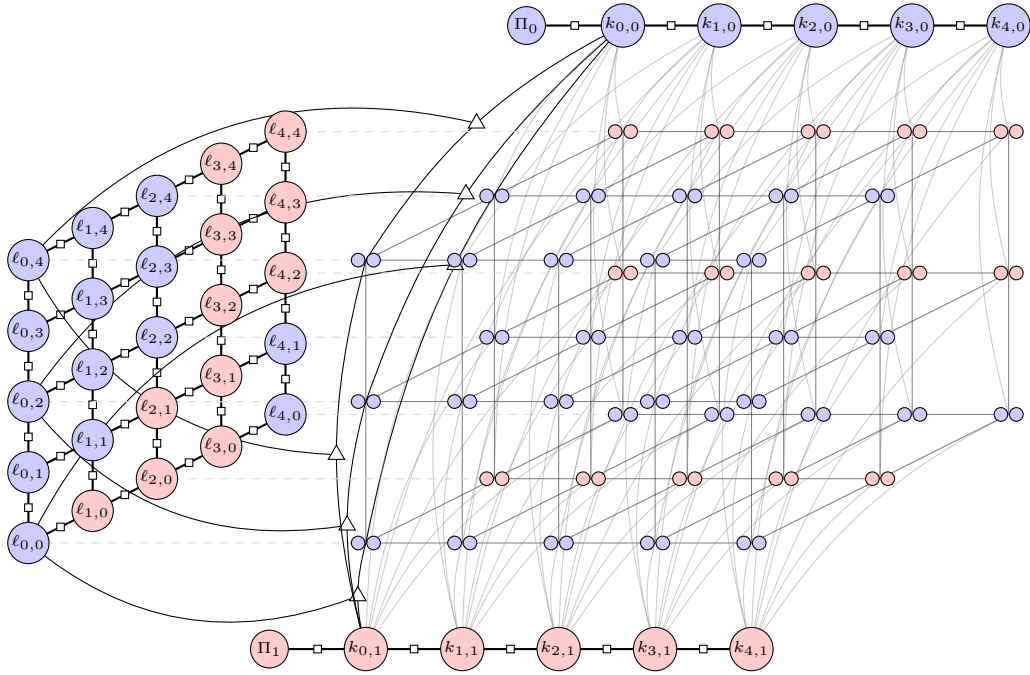


Figure 14: The coupled spatio-sequential model as a Forney-style factor graph. Each spatial node n carries a class label $\ell_n \in \{1, \dots, M\}$ (circles on the lattice, coloured by class for $M = 2$); each class m carries a hidden chain $k_{s,m} \in \{1, \dots, K\}$ with initial distribution Π_m (the two chains at the sides). Pairwise factors (squares) carry the Potts prior on the labels, Equation 19, and the chain prior, Equation 20. The gated emission factors (triangles) join one label to one chain state and contribute $\log p(x_{sn} | k_{s,m}, \theta_m)$ only when $\ell_n = m$; the observations x_{sn} are the small circles along each column. Every second observation column is drawn along each spatial axis, and an illustrative subset of the gated factors, so the structure is legible.

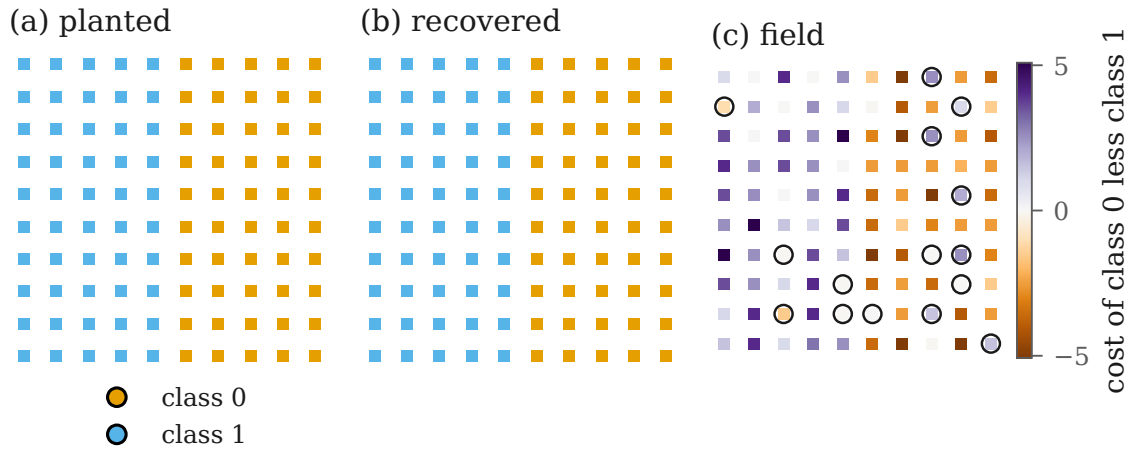


Figure 15: The coupled model's labelling on the instance past enumeration: a 10x10 open lattice, 2 classes, 2 chain states, 4 positions, spatial prior at inverse temperature 1 and coupling 1, observations drawn from seed 50 under the planted labelling and fitted from seed 0. (a) The planting: half the columns in each class, so the boundary is one cut of 10 bonds. (b) The labelling block ascent reaches from the annealed start, permuted onto the planted classes and ringed at every node where the two panels differ: 0 of 100 nodes are labelled otherwise, an agreement of 1.00. (c) The external field at the fit, as the cost of the first class less that of the second, with the sign carrying which class the observations at that node favour on their own. Taking each node at its cheaper class, with the spatial prior switched off, misses the planting at 15 of 100 nodes, ringed: at 4 positions the emissions overlap too far to label a node one at a time, and it is the prior that recovers them.

maximum of Equation 77, which is what the enumerated MAP labelling on the canonical instance would add.

12.4 Extensions, and what is not attempted

What it would take to go further. Section 12.3 names its own gap: the planted labelling is what a correct estimator recovers, and nothing establishes that it maximizes Equation 77. An enumerated maximum-posterior labelling on the canonical instance would turn that recovery statement into an optimality one at one size. Past it, a bound on the joint evidence would say how far a block ascent's fixed point sits from the best available — the mean-field bound of Equation 44 from below and the tree-reweighted bound of Equation 45 from above, both stated for a lattice and neither carried on the coupled graph. Two further directions are model rather than referee. The spatial coupling is declared and could be fitted, making it a parameter of the same block ascent and its identifiability a question the enumerable instance can answer. And the label update is a discrete search over a neighbourhood with a computable improvement — the decision process of Section 21 — so a learned proposal is admissible here on the terms it is admissible for a topology.

What is deliberately not attempted. A gate over more than one chain state. The degree-two factor is what makes conditioning on the labels leave independent chains, so a wider gate would leave the block structure standing and the expectation step no longer the posterior — the failure Section 31.3 records. Joint marginalization of labels and chains, which is intractable and is the reason the estimator is blocks at all. And a spatial prior that is not ferromagnetic: a negative coupling would put this model on the wrong side of Section 11's refusals and remove the cluster move the estimator relies on.

12.5 The algorithms

Section 31 states them: the block ascent the structure dictates, with forward-backward per class as its expectation step; one of the three label solvers of Section 29, of which the cluster move in a field is Algorithm 4; and the annealed, data-sampled start of Algorithm 5.

Application: Error Correction. A code is a factor graph with a hard constraint per check, and decoding is Equation 2 on it. The two problems below are the classical and the iteratively decoded case. A quantum code enters as the bicycle construction of Section 13.2, a second construction of the same problem rather than a problem of its own.

13 LDPC Decoding

13.1 The model

A binary linear code of length n is the null space of a parity-check matrix $\mathbf{H} \in \{0, 1\}^{m \times n}$ over $\text{GF}(2)$: the codewords are the c with $\mathbf{H}c = 0$, there are 2^k of them with $k = n - \text{rank } \mathbf{H}$, and the rate is k/n . A low-density code makes $\mathbf{H}$ sparse. Gallager's regular (d_v, d_c) ensemble [29] puts d_v ones in every column and d_c in every row: $\mathbf{H}$ is d_v bands of n/d_c rows, the first band's row i covering bits id_c to $(i + 1)d_c - 1$ and every other band the first under a uniformly random permutation of the columns, so n is a multiple of d_c and $m = nd_v/d_c$. The rows of one band sum to the all-ones vector, so at least $d_v - 1$ rows are dependent and $k \geq n - m + d_v - 1$. The

instance this repository carries is (3, 6) at $n = 19,998$, the multiple of six nearest the 20,000 the ticket names: $m = 9,999$ checks, 59,994 nonzeros, design rate $1/2$.

A codeword is sent through a memoryless binary-input channel and y is received. Three channels are supported, each seen only through its log-likelihood ratio,

$$L_i = \log \frac{p(y_i | c_i = 0)}{p(y_i | c_i = 1)} = \begin{cases} (1 - 2y_i) \ln \frac{1-p}{p} & \text{binary symmetric, flip } p, \\ 0 \text{ if erased, else } \pm \infty \text{ signed by } 1 - 2y_i & \text{binary erasure, erasure } \epsilon, \\ 2y_i/\sigma^2 & \text{binary-input Gaussian,} \end{cases} \quad (21)$$

positive when the received symbol favours zero, the Gaussian channel sending $y_i = (1 - 2c_i) + \sigma z_i$ for standard normal z_i . The posterior over the code is then

$$p(c | y) \propto \mathbf{1}[\mathbf{H}c = 0] \exp(-c \cdot L), \quad (22)$$

since $\log p(y | c) = \sum_i \log p(y_i | c_i) - \sum_i c_i L_i$ and the first sum is the same for every word. Decoding asks two questions of Equation 22: the *bitwise* MAP, each bit at the mode of its own marginal, minimizing the bit error rate and not necessarily a codeword; and the *blockwise* MAP, the single most probable codeword, minimizing the block error rate. They are different answers, as Section 7 says of the chain, and the fixture separating them is a test.

Supported sizes. Codes of length $n = 12, 24, 48, 96$ and 996 carry the refereed claims, the first being the length the codeword enumeration reaches, and the instance the roadmap names is (3, 6) at $n = 19,998$ with $m = 9,999$ checks and 59,994 nonzeros. Three ceilings separate what is refereed by what. Enumerating the 2^k codewords is refused past 200,000 words, so the exact decoding referee reaches $k \leq 17$, which at rate $1/2$ is a code of about 34 bits; Gauss–Jordan elimination gives a systematic encoder up to $n = 512$, past which the all-zero codeword is sent and the argument of Section 13 says nothing is lost; and past both, the referee is the density-evolution threshold of Appendix E.10, which exists for the erasure channel alone. The three channels of Equation 21 run at every length. The bicycle construction of Section 13.2 is carried at $n = 12, 96$ and 996 with $w = 3$, matching the first three in length and degrees, and the same three ceilings apply to it unchanged. The quantum code of Section 13.5 is carried at $n = 16$ and 96, at $m = 7$ and 40 rather than $n/2$, and replaces the first of the three ceilings: its oracle enumerates the 2^n errors rather than the 2^k codewords, so the same 200,000 configurations reach $n = 17$.

13.2 A second construction: bicycle codes

Gallager’s ensemble fixes the degrees and leaves the rest to a permutation. A bicycle code [48] fixes the matrix instead. Let A be the $n/2 \times n/2$ circulant whose first row carries w ones and whose row i is that row shifted by i , and take

$$\mathbf{H}_0 = \begin{bmatrix} A & A^\top \end{bmatrix}, \quad (23)$$

of shape $(n/2) \times n$. A column of the left half is a column of A and a column of the right half a row of it, and a circulant’s rows and columns carry the same weight, so every column of $\mathbf{H}_0$ has weight w and every row $2w$: the (3, 6) degrees at $w = 3$, and a matrix determined by w numbers rather than by $d_v - 1$ permutations. Rows are then deleted to reach $m < n/2$, which is how a rate above $1/2$ is set; *which* rows is a choice, and levelling the column weights is the one taken here, a bit in fewer checks being the bit the decoder has least to say about.

Two properties follow from the circulant and survive any deletion. First, A and $A^\top$ are both polynomials in the cyclic shift, so they commute and

$$\mathbf{H}_0 \mathbf{H}_0^\top = AA^\top + A^\top A = 0 \pmod{2}, \quad (24)$$

which is the condition a Calderbank–Shor–Steane quantum code puts on its two parity-check matrices and the reason the construction was written down [48]. Nothing in the classical problem needs Equation 24; it is exact, and is checked as one. Second, every row has even weight $2w$, so the all-ones word satisfies $\mathbf{H}\mathbf{c} = 0$ and the minimum distance is at most n whatever is drawn.

What the structure costs. The threshold of Appendix E.10 is a property of the ensemble a code is drawn from, and a bicycle code is not drawn from Gallager’s: it referees nothing here. The comparison at size is therefore against a Gallager draw of the same length, degrees and rate on shared channel realizations rather than against a limit, and at every setting measured the bicycle code is the weaker of the two under the decoder of Equation 78 — structure paid for in cycles, and the reason the construction is carried for what Equation 24 opens rather than for its classical performance.

13.3 The algebraic codes, and a decoder that does not iterate

Every code above is decoded by running Equation 78 to a fixed point, and refereed by enumerating its 2^k words, which stops where the exponent does. The codes here are decoded in closed form at every length, so each is a referee whose cost does not grow with the block [16, 47].

A code of minimum distance d corrects $t = \lfloor (d-1)/2 \rfloor$ errors, since the Hamming spheres of that radius around its words are disjoint. A *perfect* code is one whose spheres also cover, leaving nothing over:

$$2^k \sum_{i=0}^t \binom{n}{i} = 2^n, \quad (25)$$

an equality over integers rather than a bound, and the sharpest structural invariant available here — a dropped row or a permuted column fails it at once. The binary perfect codes are the odd repetition codes, the Hamming codes $(2^m - 1, 2^m - 1 - m)$ and the Golay code $(23, 12)$; the extended Golay $(24, 12)$ is not perfect, and buys $d = 8$, which corrects three errors and detects a fourth.

Reed–Solomon takes the other route. Over $\text{GF}(2^m)$ with $n = 2^m - 1$, the generator polynomial’s roots are $\alpha^1, \dots, \alpha^{n-k}$, and the code meets the Singleton bound

$$d \leq n - k + 1 \quad (26)$$

with equality, which no code above does: it is the first maximum-distance separable code here. Its decoder is algebraic — Berlekamp–Massey for the error locator, a Chien search for the locator’s roots, Forney for the magnitudes [16] — and is *bounded distance*: it returns a word within t symbols of what arrived, or refuses.

A bounded-distance decoder’s error rate is an equality, not a bound. Past t the transmitted word is farther than t from what arrived, so a decoder that only returns words within t cannot return it. The block error probability is therefore exactly the binomial tail

$$P_{\text{block}} = 1 - \sum_{i=0}^t \binom{n}{i} q^i (1-q)^{n-i}, \quad (27)$$

where q is the probability that one symbol is wrong: the crossover p for a binary code, and $1 - (1-p)^m$ for a code over $\text{GF}(2^m)$ whose symbols are sent as m bits. Equation 27 is also the *maximum-likelihood* rate for a perfect code, where every received word has a unique nearest codeword. That is what makes these codes oracles rather than examples, and it is what the implementations here are pinned against.

13.4 Capacity, which is nobody's decoder

Every rate above is what one decoder achieves. Capacity is what no decoder of that rate achieves: a converse, not a construction [20]. For the three channels of Section 13.1, in bits per channel use,

$$C_{\text{BEC}}(\epsilon) = 1 - \epsilon, \quad (28)$$

$$C_{\text{BSC}}(p) = 1 - H_2(p), \quad (29)$$

$$C_{\text{BIAWGN}}(\sigma) = 1 - \mathbb{E} \log_2 \left(1 + e^{-2y/\sigma^2} \right), \quad y \sim \mathcal{N}(1, \sigma^2), \quad (30)$$

with H_2 the binary entropy in bits. The first two are closed forms; the third is an integral, taken by Gauss–Hermite quadrature, and carries the published rate-1/2 limit $\sigma^* = 0.9787$ [61] as its pin — 0.49999606 against one half, and unmoved to 3×10^{-10} by halving or doubling the node count. Inverting each of the three for the noise at which capacity equals a rate is a bisection, on the same monotonicity argument as Appendix E.10's threshold.

What the gap says, and what it does not. A threshold and a capacity are the same kind of statement — both are about infinite length — and their ratio is the ensemble's. The (3,6) ensemble's erasure threshold $\epsilon^* = 0.4294$ sits below $C_{\text{BEC}}^{-1}(1/2) = 0.5$, leaving 14.1% of that channel unused; on the symmetric channel its threshold 0.084 against $p^* = 0.1100$ is a factor of 1.31. A finite-length code's crossing of a chosen error rate is *not* the same kind of statement, and the distance from it to the boundary is part length and part decoding: at a block error rate of 10^{-2} , the repetition, Reed–Solomon, Golay and Hamming pairs of Section 13.3 give up factors of 3.0, 5.6, 2.7 and 3.9 against Equation 29, on blocks of 3 to 23 bits. Lowering the rate does not buy the gap back: repetition has the mildest limit of the four, $p^* = 0.174$, and still gives up 3.0.

13.5 The quantum code those matrices define

Equation 24 is the condition a Calderbank–Shor–Steane code puts on its pair of parity-check matrices, so $\mathbf{H}_X = \mathbf{H}_Z = \mathbf{H}$ is a quantum code on n qubits with

$$k = n - 2 \text{rank } \mathbf{H} \quad (31)$$

logical qubits [48]. Everything below is one sector of it: X errors against Z checks, which is what a single matrix gives without a symplectic representation of the full Pauli group.

The first thing to check is that there is anything to decode. Equation 31 is zero whenever $\mathbf{H}$ has full row rank at $m = n/2$, which is the matrix Equation 23 gives before any deletion. At the two lengths Section 13.2 carries at $m = n/2$ the rank is m and $k = 0$: the code encodes nothing, every correction is trivially right, and a measured error rate on it would be a measurement of the zero. Deleting rows raises the rate *and* the dimension, and the instances used here have $m < n/2$ strictly.

What a decoder has to get right, and it is not the error. The stabilizer group is the row space of $\mathbf{H}$; the operators commuting with it are $\ker \mathbf{H}$; and Equation 24 is exactly the statement that the first sits inside the second. A residual $r = e \oplus \hat{e}$ in the row space acts as the identity on the codespace, so the decode *succeeds* although $\hat{e} \neq e$; a residual in $\ker \mathbf{H}$ outside the row space is a logical operator and the decode fails. The criterion is therefore membership of the quotient $\ker \mathbf{H} / \text{rowspace } \mathbf{H}$, of dimension k , and agreement with the error is neither necessary nor sufficient for it. This is *degeneracy*, and it is why the referee of Section 13.7 — the maximum-likelihood codeword — does not transfer: the quantity that replaces it is a coset.

Maximum likelihood is a sum over a coset. With a channel giving each error e a probability $P(e)$, the decoder that minimizes the logical error rate returns

$$\hat{L}(s) = \arg \max_L \sum_{e: \mathbf{H}e=s, [e]=L} P(e), \quad (32)$$

the coset L of the quotient whose members carry the most probability between them, and not the single likeliest error. The two are different decoders, and the second is the one a classical intuition reaches for. Summing a coset is the optimum here, so its logical error rate is the floor any other decoder is measured against, and the gap between the two is a property of the code rather than of an implementation.

Short cycles are forced, not drawn. Equation 24 says every pair of rows of $\mathbf{H}$ overlaps in an even number of positions. A pair overlapping in exactly two is two checks covering two common bits, which is a four-cycle of the Tanner graph. So the condition that makes the quantum code exist puts short cycles into the very graph Equation 78 needs free of them, and the count is reported beside every decoding result rather than cited: at the enumerable instance the graph carries thirty of them, and at the larger one a hundred and sixty.

Three regimes again, and the ceiling has moved. The oracle here enumerates *errors* and not codewords, so its ceiling is 2^n rather than 2^k : the two hundred thousand configurations of Section 16 reach $n = 17$, where the classical oracle at rate one-half reached about thirty-four bits. The enumerable instance therefore carries an exact floor and a measured gap to it; past it the decoder's rate is reported with nothing exact behind it, which the fixture states by declaring no oracle.

13.6 As a factor graph

The variables are the bits, $x_i \equiv c_i \in \{0, 1\}$. The factors are a unary $\psi_i(c_i) = e^{-c_i L_i}$ per bit, the channel folded in as an observation is everywhere in this document, and a parity factor $\psi_a(c_{\partial a}) = \mathbf{1}[\bigoplus_{i \in \partial a} c_i = 0]$ per row of $\mathbf{H}$, a hard constraint of degree d_c whose log table is zero on even-parity assignments and $-\infty$ elsewhere. This is the Tanner graph of the code; a bit sits on d_v parity factors and one unary, so in the Forney form every bit passes through an equality node of degree $d_v + 1$ (Figure 16). The sketch states the structure; Figure 17 draws one member of the ensemble, where which bits a check covers is the draw and not the design. The graph has cycles—a random draw does not avoid short ones—so Equation 2 is the Bethe approximation on it, and the exact quantities below come from enumerating the 2^k codewords where $2^k \leq 200,000$ (a $(3, 6)$ code at $n = 24$ has $k \geq 14$) and from the tree schedule on a cycle-free code, where message passing is exact.

13.7 Validation

- *Enumeration of the 2^k codewords* referees both decodings on a code short enough to enumerate: the bitwise posterior and the maximum-likelihood codeword, the latter with the margin over the runner-up pinned so that no tie is being broken. It establishes exactness there and reaches no further than the ceiling above.
- *The tree schedule of the general sum-product* referees the decoder on a cycle-free code, where message passing is exact. It establishes that the log-domain specialization of Equation 78 is the same computation as the general implementation; on a loopy code the two agree because they are the same Bethe approximation, which is agreement and not truth.

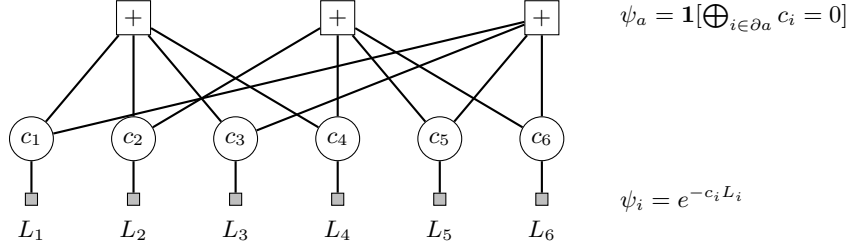
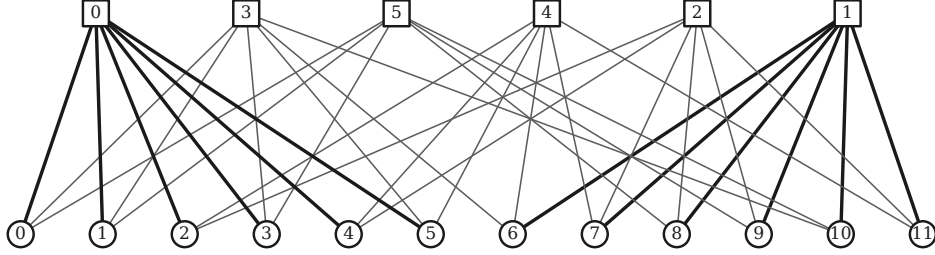


Figure 16: A Tanner graph: bits $c_i \in \{0, 1\}$ as circles, the parity factors of a row of $\mathbf{H}$ as squares, and the channel factor of Equation 21 as a shaded square carrying the log-likelihood ratio L_i at each bit. The drawn graph is six bits and three checks of degree four; the ensemble this repository carries is regular $(3, 6)$, so every bit sits on three parity factors and every parity factor on six bits. The graph has cycles — a random draw of the ensemble does not avoid short ones — so Equation 2 on it is the Bethe approximation and Equation 78 is that approximation in the log domain. A sketch of the structure and not a rendering of any instance.

(a) bits, checks, and the nonzeros between them



(b) the same nonzeros as a matrix, 3 bands

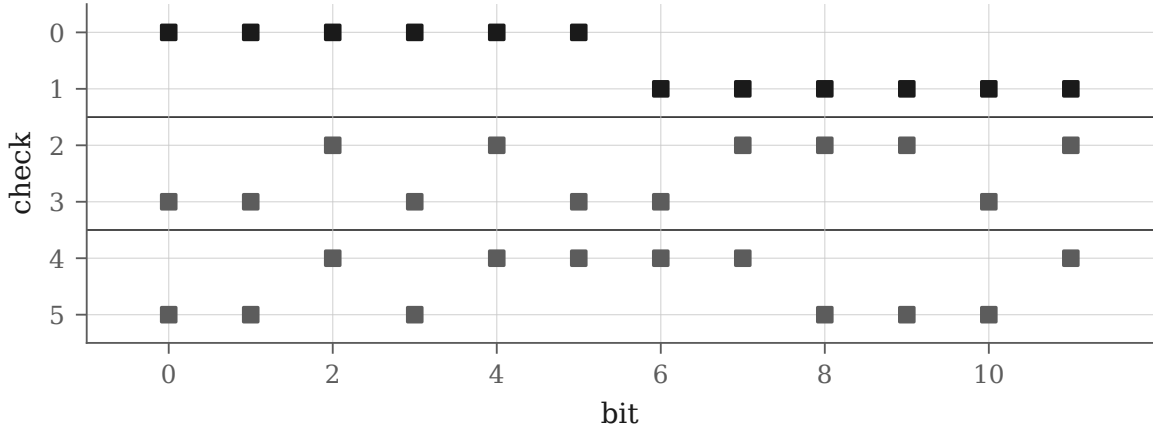


Figure 17: The Tanner graph of one draw from the regular $(3, 6)$ Gallager ensemble: the enumerable instance, 12 bits under seed 340, decoded on a binary symmetric channel at crossover 0.15. (a) Bits as circles, the 6 parity checks as squares, and one edge per nonzero of the parity-check matrix: 36 of them, every bit on 3 checks and every check on 6 bits. (b) The same nonzeros as the matrix, its 3 bands of 2 rows separated by a rule. The first band is drawn heavy in both panels: its checks cover consecutive bits by construction, and every later band is that band under a uniform random permutation of the columns, which is the whole of the randomness in the draw. The construction fixes the degrees at every block length, so the longer instances are this structure at a size no page holds; this one is drawn because a page holds it.

- $\mathbf{H}c = 0$ *on every realization* referees the encoder, and the dense matrix product referees the sparse offsets layout it stands in for. Each establishes one construction and nothing about decoding.
- *The channel symmetry argument* is checked rather than assumed: where an encoder exists, the error pattern under a non-trivial codeword is compared per realization with the pattern under the zero word. It establishes that the all-zero shortcut is sound at the lengths an encoder reaches, which is what licenses it past them.
- *The erasure threshold* $\epsilon^* = 0.4294$ of Appendix E.10 referees the 19,998-bit code, which is held to bracketing it. It establishes the asymptotic behaviour of the ensemble on one channel; it establishes nothing at finite length on the other two, which is the cell Section 16 marks and the extension past the erasure channel would fill.
- *Equation 25* referees the algebraic constructions as an equality over integers, and Equation 27 referees their decoders as a closed form no length outruns: the implementations are held to it by Monte Carlo at a stated draw count, agreeing within 1.7 standard errors on every pair measured. It establishes that each code is the one named and that its decoder achieves what bounded distance guarantees; it establishes nothing about what a better decoder would achieve on the same code, which for a perfect code is the same number and for Reed–Solomon is not.
- *Capacity* referees every threshold and operating point from above, and is the only referee here that does so without running anything: Equation 29 and its two siblings forbid a rate at a noise level, so a threshold that passed one would be a mistake in the threshold or in the capacity. It establishes that the numbers reported are not impossible, and nothing about any of them being right.
- *The algebra of Equation 23* referees the second construction, exactly and with no tolerance: the degrees against w , the halves against a circulant built by rolling its own first row, $k = n - \text{rank } \mathbf{H}$ against the rate each deletion count asks for, Equation 24 against a dense product over $\text{GF}(2)$, and the minimum distance against an enumeration of the code where one reaches. It establishes what the construction is and nothing about how it decodes.
- *A Gallager draw of the same length, degrees and rate* referees the bicycle code’s decoding on shared channel realizations, and the two enumerated decodings bound it where they reach. It establishes where one construction sits against the other; it establishes nothing about either against the best a decoder could do at that length, which no oracle here reaches.
- *The rank of $\mathbf{H}$* referees the quantum code’s dimension, Equation 31, and a basis of the quotient is built and checked to lie in $\ker \mathbf{H}$ and outside the row space, no combination of it excepted. It establishes that there is a code to decode; at $m = n/2$ it establishes the opposite, which is why those instances carry no decoding claim.
- *The criterion is refereed from both sides before a decoder runs*: a nonzero stabilizer added to a correct correction must read as a success and a logical operator must read as a failure, since an implementation that answers one way always passes one of the two. The membership test is then pinned a second way, against a label map built from a different computation, over every word of $\ker \mathbf{H}$ at the enumerable instance. It establishes that the quotient is the one Equation 32 is written in, which every number below depends on and none of which would visibly break if it were wrong.
- *Enumeration of the 2^n errors* referees decoding under degeneracy: the partition carries exactly 2^k cosets per reachable syndrome and the whole probability, and Equation 32 is

evaluated on it exactly. It establishes the floor at that length and nothing past it, the ceiling being $n = 17$.

- *The coset symmetry* referees the decoder being a decoder of the syndrome: two errors differing by a codeword are checked to give the same correction, which is the same argument the all-zero word rests on rather than a second assumption.

13.8 Extensions, and what is not attempted

What it would take to go further. One extension is named by the validation itself: the threshold of Appendix E.10 exists on the erasure channel alone, so the symmetric and Gaussian channels have no asymptotic referee at the length the roadmap names. Carrying the same fixed-point argument to them means tracking a message *density* rather than a single erasure probability, either discretized or reduced to one parameter by a Gaussian approximation [61]; that is the marked cell of Section 16 and the single most valuable thing this section is missing. Two further directions are the code rather than the referee. An irregular ensemble, with a degree distribution optimized against the threshold, is what the modern construction does and what a regular $(3, 6)$ code deliberately is not. And a decoder whose Tanner-graph edges carry learned weights is the same message passing with the update's parameters fitted [54]: it is admissible here because it changes nothing about the referees — a short code still enumerates, and a learned decoder that beats the tanh rule on a loopy code must still equal it on a cycle-free one.

What is deliberately not attempted. The rest of the Pauli group. Section 13.5 carries one CSS sector, X errors against Z checks, which is what $\mathbf{H}_X = \mathbf{H}_Z = \mathbf{H}$ gives without a symplectic representation; correlated X and Z errors — a depolarizing channel rather than two independent ones — couple the two sectors, and a decoder for them decodes a quaternary alphabet on one graph rather than a binary alphabet on two. Codes that are not CSS are further out again: there the stabilizer generators are not separable into two classical matrices at all, and nothing in this section transfers unchanged. Non-binary codes, since a binary variable's message being one number is what makes Equation 78 a closed form rather than a sum. A channel with memory, which breaks the per-bit factorization the unary factors rest on and with it the symmetry argument that licenses the all-zero word. And an encoder past the elimination's reach: past it the shortcut is used rather than extended, and the section's job is to say what that costs, which is that the symmetry is checked below the reach and assumed above it.

13.9 The algorithms

Section 32 states them: Equation 2 specialised to one number per message, run in the log domain by Algorithm 16, with the min-sum approximation of Equation 79 beside it.

14 Turbo Decoding

14.1 The model

A rate- $1/2$ *recursive systematic convolutional* encoder is a linear feedback shift register of memory m over $\text{GF}(2)$, stated by two generator polynomials written in octal: a feedback $a(D)$ and a feedforward $b(D)$, both of degree m with $a_0 = b_0 = 1$. With register contents $s_1 \dots s_m$, most recent first, one step on input u_t forms the feedback value, emits the input unchanged and one parity bit, and shifts:

$$d_t = u_t \oplus \bigoplus_{i \geq 1} a_i s_i, \quad p_t = b_0 d_t \oplus \bigoplus_{i \geq 1} b_i s_i, \quad (s_1, \dots, s_m) \leftarrow (d_t, s_1, \dots, s_{m-1}). \quad (33)$$

The state is the register, so the encoder is a deterministic Markov chain on 2^m states with two edges leaving each: a *trellis*. Setting $u_t = \bigoplus_{i \geq 1} a_i s_i$ makes $d_t = 0$, so m such *tail* steps empty the register from any state — for a recursive encoder the tail is a function of the state and not a string of zeros, which is the whole of what termination costs.

A *turbo* code [14] is two such registers fed the same K -bit message in two orders. The first reads u ; the second reads Πu , where Π is an interleaver, a permutation of $\{0, \dots, K-1\}$. Both are terminated, and the transmitted word is

$$c = \left(\underbrace{u, t^1}_{K+m}, \underbrace{p^1}_{K+m}, \underbrace{t^2}_m, \underbrace{p^2}_{K+m} \right), \quad (34)$$

with t^1, t^2 each encoder's tail inputs and p^1, p^2 its parity stream: $n = 3K + 4m$ bits, rate K/n , unpunctured. The second encoder's K message bits are already carried by the first's systematic stream in a different order, which is the whole of what the interleaver buys; its tail inputs are not, so they are transmitted. The channel and the log-likelihood ratio convention are Section 13's unchanged, Equation 21, and the posterior over messages is Equation 22 with the code taken as the image of the encoder.

Supported sizes. Message lengths $K = 12, 256$ and $1,024$ carry the refereed claims, all with the memory-2 (7, 5) register, so $n = 44, 780$ and $3,080$. Three ceilings separate what is refereed by what. Enumerating the 2^K messages is refused past 200,000 configurations, so the exact bitwise MAP referee reaches $K \leq 17$; a dense GF(2) nullspace turns the code into a parity-check matrix, and so into an instance of Section 13, up to $n = 512$; and past both the referees are the output symmetry of Equation 21's channels — checked per realization where an encoder is affordable — and the uncoded closed form $Q(\sqrt{2E_b/N_0})$, which every coded curve must fall below at its operating point. The convolutional decoders themselves are linear in K at every length: 2^m states times $K + m$ steps, with no ceiling of their own.

14.2 As a factor graph

The variables of one constituent code are the $K + m + 1$ register states, and one factor per step carries the whole of that step — the transition the register allows and the channel's score for the two bits it transmits:

$$\psi_t(s_t, s_{t+1}) = \begin{cases} \exp(-u(L_t^s + L_t^a) - pL_t^p) & \text{the edge exists,} \\ 0 & \text{otherwise,} \end{cases} \quad (35)$$

where the edge is the transition $s_t \rightarrow s_{t+1}$, u and p are the input and parity bits labelling it, L^s and L^p are the channel ratios of the systematic and parity bits, and L^a is an a priori ratio on the input, zero for a stand-alone decoding. The input bit needs no variable of its own: the two edges leaving a state enter different states, so u_t is recoverable from the pair. This is a chain and the terminated boundary pins both ends, so Equation 2 on the tree schedule is exact on it. That is the difference between one constituent code and the pair. The turbo graph joins the two chains at the message bits through Π (Figure 18), which makes each message bit a variable of degree two shared between two chains and puts a cycle around every pair of positions the permutation reorders; no exact algorithm is available there at the lengths the code is used at, and what is run instead is Equation 2 on a serial schedule.

14.3 Validation

- *Enumeration of the 2^K messages* referees the BCJR posterior, the Viterbi decision and the size of the turbo iteration's gap. It establishes exactness for one constituent code and an ordering, not an equality, for the pair; it reaches $K \leq 17$ and no further.

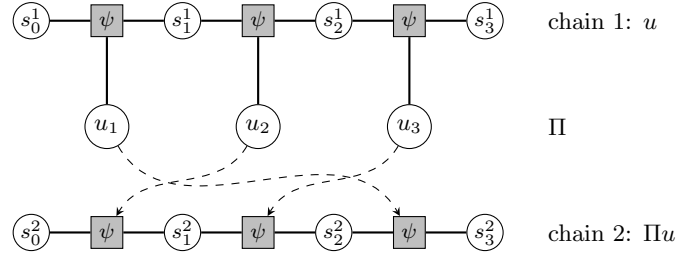


Figure 18: A turbo code as a factor graph: two trellis chains sharing the message bits through a permutation. Each ψ is one trellis factor of Equation 35, carrying both the transition the register allows and the channel's score for the two bits that transition transmits; each s_t^c is a register state. The message bit u_i is read by the first chain at position i and by the second at position $\Pi^{-1}(i)$, so every u_i has degree two and the graph has one cycle per pair of message bits the permutation reorders — which is why Equation 2 on it is the Bethe approximation and why the serial schedule of Algorithm 17 is a choice rather than a derivation. Three message bits and a memory-1 register are drawn; the instances this repository carries are $K = 12, 256$ and $1,024$ with the memory-2 $(7, 5)$ register. A sketch of the structure and not a rendering of any instance.

- *The tree schedule of the general sum-product* referees the trellis decoders on the factor graph of Equation 35. It establishes that the specialization is the same computation as the one every other problem class here runs, and nothing about whether either is the right model.
- *The polynomial division $b(D)/a(D)$* referees the trellis arrays, run as an explicit long division over $\text{GF}(2)$ sharing no code with them; $\mathbf{H}c = 0$ through a nullspace matrix referees the encoder, and the tail is refereed by the state it leaves. Each establishes one construction and nothing about decoding.
- *The interleaver as a permutation* is checked rather than assumed: $\Pi^{-1}\Pi$ and $\Pi\Pi^{-1}$ are the identity, and the fixture records the seed the permutation was drawn under, so the code is reproduced by drawing it again rather than by a written-out array no reader could check.
- *The channel symmetry argument* of Section 13 is re-checked here for the recursive encoder and the turbo message map, per realization: the decoded error pattern under a codeword equals the decoded word under the all-zero message on the same noise. It establishes that the all-zero shortcut is sound, which is what licenses every error rate measured past $K = 17$.
- *The uncoded bit error rate $Q(\sqrt{2E_b/N_0})$* referees the waterfall at $K = 256$ and $K = 1,024$: the coded curve is held to falling below it at every declared point and to falling with E_b/N_0 . It establishes that the code buys something and bounds nothing about how much; the block error rate at a stated frame count is what carries the resolution, and a curve is reported with the count it rests on.

Where the iteration departs. The strong claim — that each turbo iteration improves on the last — is *not* made, because the measurement refuses it. Over the six declared points at $K = 256$ the ensemble bit error rate rises between consecutive iterations at four of them, the largest rise 2.3×10^{-3} at 0 dB between iterations 7 and 8, and every rise is inside one binomial interval of the 12,800 message bits the point rests on. What is asserted is the weaker pair the sample supports: the last iteration beats the first, and no rise exceeds twice that interval. The same non-monotonicity is visible at $K = 12$ with a shorter interleaver, where it is larger relative to the rate; it is a property of loopy sum-product on a serial schedule and not a defect of this implementation.

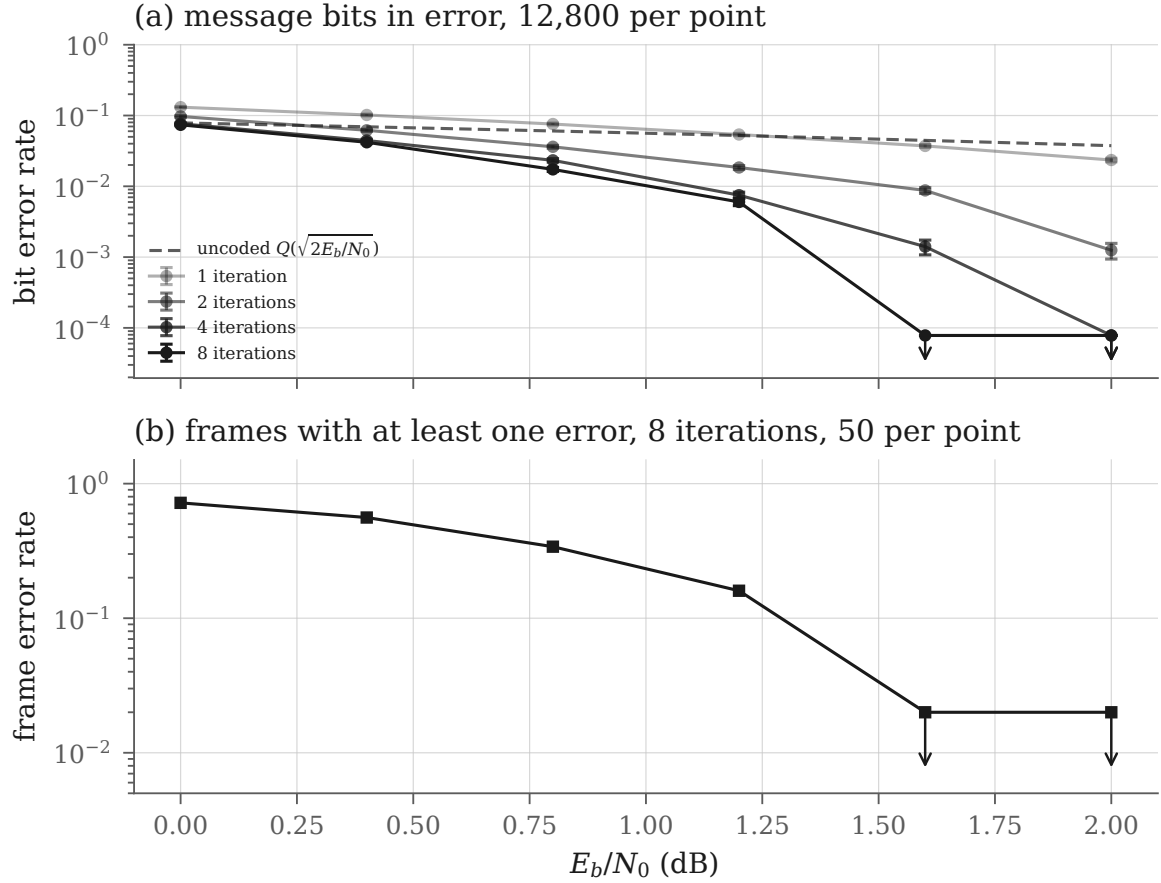


Figure 19: The waterfall of the memory-2 (7, 5) turbo code at $K = 256$: rate 0.330 unpunctured, both constituent encoders terminated, a uniformly random interleaver drawn under seed 233, on the binary-input Gaussian channel. (a) Bit error rate at 1, 2, 4 and 8 iterations of the extrinsic exchange, each point 50 frames and 12_800 message bits, with the one-sigma binomial interval that sample supports; the dashed line is the uncoded antipodal closed form $Q(\sqrt{2 E_b/N_0})$, which the coded curve falls below at every point, reaching below $8e-05$ at 2 dB against 0.038 uncoded. (b) Frame error rate at 8 iterations on the same axis. A point at which no frame erred is drawn at one over the sample size with an arrow: the sample bounds the rate from above and says nothing below it. The interval understates the spread of the estimate, since bit errors inside one frame are correlated; it is the bar the sample size supports if the bits were independent, and a floor on the true one.

14.4 Extensions, and what is not attempted

What it would take to go further. The section’s own statement is that no exact algorithm reaches the pair at the lengths it is used at, and the referee past $K \leq 17$ is a bound from one side. What would give the pair an ensemble statement is the analogue of Appendix E.10: track how the extrinsic information one constituent decoder produces depends on what it is given, and read the iteration’s convergence off the pair of such curves — the same fixed-point reasoning as the erasure threshold, applied to two decoders in series rather than to one graph. That would turn the measured waterfall into a predicted one. Two code-side directions follow from Section 33.4’s observation that the cycles, not the length, are what cost the iteration: an interleaver designed to avoid short cycles rather than drawn at random, and puncturing, which raises the rate and is what the transmitted word of Equation 34 deliberately does not do.

What is deliberately not attempted. Monotonicity in the iteration count, which is not an open question here but a refused claim — the measurement contradicts it and the section states the weaker pair the sample supports. A blockwise decoding of the *pair*: the maximum over messages is well defined and no algorithm reaches it, so what is carried is the constituent code’s blockwise answer and the pair’s bitwise one. And non-binary constituent codes, for the reason Section 13.8 gives: the channel and the ratio convention are binary throughout, and every referee here is stated in those terms.

14.5 The algorithms

Section 33 states them: the BCJR recursion of Equation 81 on each constituent trellis, which is Equation 51 in the log domain, and the extrinsic exchange between them, Algorithm 17.

15 Polar Codes

15.1 The model

A polar code is the image of one fixed matrix under a choice of which inputs carry information [5]. With $F = \begin{bmatrix} 1 & 0 \\ 1 & 1 \end{bmatrix}$ and $N = 2^n$,

$$G_N = F^{\otimes n}, \quad c = uG_N \pmod{2}, \quad (36)$$

where u carries the k message bits on an *information set* $\mathcal{I}$ and zeros on its complement, the *frozen set*. F is its own inverse over $\text{GF}(2)$ and so is every Kronecker power of it, which is how a decoder recovers u from c — by applying G_N a second time rather than inverting it.

Which inputs carry information is the whole construction. Under a decoder that decides u_i in order, knowing $u_0, \dots, u_{i-1}$, input i sees a *synthetic* channel W_i : the same physical channel, N uses of it, and a different effective quality per input. The transform moves quality between inputs and creates none,

$$\sum_{i=0}^{N-1} I(W_i) = N I(W), \quad (37)$$

so a rule that freezes the worst $N - k$ inputs spends the channel where it is worth spending. On the erasure channel the Bhattacharyya parameter obeys Arıkan’s exact recursion, one kernel application at a time,

$$Z(W^-) = 2Z(W) - Z(W)^2, \quad Z(W^+) = Z(W)^2, \quad (38)$$

with $I = 1 - Z$ there exactly, which makes Equation 37 checkable rather than cited. Off the erasure channel no closed recursion exists and the standard Gaussian approximation is used instead: one mean per input, $Z = e^{-m/4}$. It is an approximation, and only the *order* it induces matters, since the frozen set depends on nothing else.

Reed–Muller is the same transform under a different rule. $\text{RM}(r, n)$ freezes by row weight — keep the rows of G_N of weight at least 2^{n-r} — where a polar code freezes by reliability. At $N = 8$, $k = 4$ the two rules pick the same information set, $\{3, 5, 6, 7\}$, which is the extended Hamming $(8, 4)$ code — the same object Section 13’s algebraic constructions reach by a different route; at $N = 16$ they part. One transform, two rules, and no second implementation of either.

Successive cancellation is not maximum likelihood, and that is the point. It decides each input in order and cannot revisit one, so a single early mistake propagates. The synthetic channels of Equation 37 are *defined* by that decoder, which is what makes the construction coherent and also what caps it: the decoder is weaker than the code. A list decoder carries L candidate prefixes and prunes by an exact path metric [77], and at $L = 2^k$ it prunes nothing and is therefore maximum likelihood.

Supported sizes. Block lengths $N = 16$, 256 and 2,048 at rate $1/2$ carry the refereed claims. Three ceilings separate what is refereed by what. Equation 38 is closed form at *every* length, so the construction is exact at all three. Enumerating the 2^k codewords is refused past 200,000 configurations, so the maximum-likelihood referee — Section 13’s enumeration, reached by taking the dual of Equation 36’s information rows — covers $N = 16$ at rate $1/2$ and no more, and the exhaustive list $L = 2^k$ with it. A dense G_N is refused past $N = 4,096$; the decoders need none, being recursive over $\log_2 N$ stages. $N = 8$ was the first candidate for the enumerable size and is too small to measure the gap this section is about: at $k = 4$ it is the extended Hamming code, where successive cancellation already agrees with enumeration on most draws.

15.2 As a factor graph

The graph is Equation 36 written out: $N(n + 1)$ binary variables in $n + 1$ columns, and one parity factor per butterfly joining a pair in one column to a pair in the next (Figure 20). It is loopy for $n \geq 2$, so Equation 2 on it is the Bethe approximation and not exact — and successive cancellation is not that approximation either. It is a *schedule*: decide u_0 from a leaf-to-root pass, hold it fixed, decide u_1 , and so on, each pass exact given the decisions before it. The two operations the pass needs are the check-node rule of Equation 78 on a pair, and a sign correction by the decision already made,

$$f(a, b) = 2 \operatorname{artanh}(\tanh(a/2) \tanh(b/2)), \quad g(a, b, \hat{u}) = b + (1 - 2\hat{u})a. \quad (39)$$

15.3 Validation

- *Equation 37* referees the construction at every length, as an equality to 10^{-12} at $n = 3, 5, 8, 11$: the recursion is closed form, so there is nothing to sample. It establishes that the transform conserves what it redistributes and nothing about any decoder.
- *Enumeration of the 2^k codewords* referees both decoders at $N = 16$: the maximum-likelihood codeword is what successive cancellation is measured against, and the gap is *reported* rather than asserted away — at $\sigma = 1$ over 200 shared draws, 98 block errors for successive cancellation, 64 for a list of 4, and 62 for maximum likelihood. It establishes the size of what the decoder gives up, and it reaches no further than $k \leq 17$.

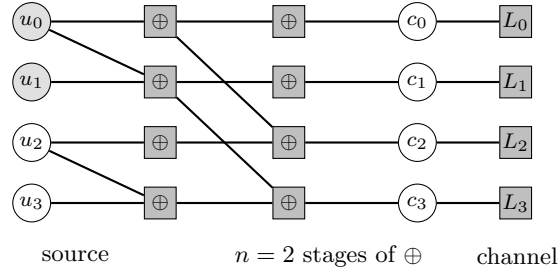


Figure 20: The transform of Equation 36 at $N = 4$ as a factor graph. Each $\oplus$ is one application of the kernel: a parity factor over the pair it joins. Shaded source variables are frozen to zero and carry no message, and which they are is what Equation 38 chooses. Successive cancellation is a schedule on this graph and not a separate algorithm — it decides u_0 , then u_1 given that decision, and so on, each pass exact given the ones before it.

- *A naive recursive referee* referees the list implementation’s layout: same arithmetic, same order, one Python call per node and no path axis, pinned *bitwise* over 900 draws across the three channels. It establishes that the path-array form computes the same decisions; a tolerance there would hide the one bug it exists to catch, a gather that reorders the traversal.
- *The two ends of the list* referee the fork-and-prune machinery: $L = 1$ must reproduce plain successive cancellation bit for bit, and $L = 2^k$ prunes nothing and must therefore equal the enumerated maximum-likelihood answer. Each establishes one end and neither the middle, which is what the monotonicity below covers.
- *Monotonicity in L over shared seeds* referees the metric’s ordering: a larger list explores a superset only if the metric ranks candidates consistently, which is a property of this implementation and not of the theory. It establishes an ordering and no rate.
- *A measured ordering of the synthetic channels* referees the Gaussian approximation, which has no closed form to be held to: per-input error rates of successive cancellation on the rate-one code agree with it at Spearman 0.897, against 0.903 for the exact erasure recursion measured the same way, and the best and worst inputs agree exactly. It establishes that the approximation orders the inputs as a decoder does; it is not the genie-aided ordering the synthetic channels are defined by, so the agreement is high rather than perfect and is reported as such.

15.4 Extensions, and what is not attempted

What it would take to go further. A cyclic redundancy check appended to the message and carried through the list turns the metric’s best survivor into the best *passing* survivor, which is what makes list decoding competitive in practice; the check is polynomial division over $\text{GF}(2)$, which the elementary codes carry, and the combination is the next step here. Beyond it: a systematic encoder, so the message is readable off the codeword without inverting anything; puncturing and shortening, which is how a length that is not a power of two is reached; and the Gaussian approximation against Monte Carlo at $N \geq 256$, where a disagreement would change the frozen set and the fixture would have to carry the measured one — a decision, since it doubles the construction cost, and not a default.

What is deliberately not attempted. The polarisation theorem, which is asymptotic: what is reported instead is the fraction of inputs that are neither good nor bad at each length, falling from 0.75 at $N = 8$ to 0.063 at $N = 16,384$, and the trend is asserted monotone rather than

the limit claimed. And a min-sum f : the hardware approximation of Equation 39, which would need the exact form kept beside it as its oracle, and nothing here asks for it yet.

15.5 The algorithms

Section 33.3 states them: the recursive pass of Equation 39 per input, Algorithm 18, and the list that carries L prefixes through it, Algorithm 19.

16 The Problems, and What Applies to Each

Tables 2 and 3 are generated from the problem catalogue and the regression suite, not written: a row is a problem, a column an algorithm family or a kind of referee, and a cell says whether the pair exists and what pins it. Tables 4 to 7 are the same reading taken one method family at a time, in the grouping Appendix G uses, and add to each pairing the tier it is validated at and a sentence on when the family wins on that problem and when it does not — the one hand-written part of the three, naming the experiment where a measurement decided it. A pairing the catalogue carries and no test of either significant kind names is marked *untested* rather than omitted, so the table cannot go quiet about a method that exists. Two readings are load-bearing. The first is the standard: where an oracle is affordable at a size, the method is pinned to it; where none is, recovering the simulated truth—the generating parameters or structure of a seeded fixture, to a stated tolerance or at a stated coverage—is sufficient, and the oracle remains desired. The second is the marks saying where that second case holds, per method and per size tier, so the oracle each would need is named: a branch-and-bound bound for trees past the enumerable taxon count, a boundary contraction for lattices past the transfer-matrix width, and for the code the density-evolution threshold of Appendix E.10 extended past the erasure channel. Table 9 is the third reading, method against method rather than method against problem: per problem the algorithms ordered from exact to efficient, each rung beside the rung it is pinned against and the test pinning the pair, and a test cell reading *ticket* is an issue rather than a silence.

A blank is not built, not inapplicable. The tables are generated from the catalogue and the suite, both of which are records of what the tree holds, so an empty cell says that this problem carries no method of that family *here* and says nothing about whether one could exist. The two are different claims and only the first is generated: not built is a gap with a ticket behind it, and not applicable is a statement about the mathematics that has to be argued rather than read off a table. Four pairings are the second, and they are the only four. A surrogate for the continuous test functions stands in for an evaluation that is already three arithmetic operations, so there is nothing for it to save (Section 20). A cut-based ground state needs a submodular energy, which the frustrated lattice’s mixed-sign couplings do not give (Section 29.3). A single-spin cluster move built on the Fortuin–Kasteleyn bond needs a ferromagnetic coupling, so Swendsen–Wang and Wolff are defined on Section 10’s lattice and not on Section 11’s. And a branch-length optimizer has nothing to optimize under parsimony, whose score is a count and carries no continuous parameter (Section 9). Every other blank is work not done.

Table 8 then says what each problem is declared *at*. Every claim in this document is made on an instance, and an instance is a size and a set of parameters rather than a name: the tiers of one problem differ in more than one quantity at once — extent and state count, taxa and sites, chain length and transition rate — so a reader who knows only that a result was taken at the smaller of two instances knows almost nothing. The table states each in the words its own declaration uses, and it is generated, so an instance cannot be described here and declared otherwise.

	simulation	exact evaluation	message passing	gradient fit	expectation-maximization	initializer	hill climbing	sampling and tempering	descent and expansion	exact ground state	relaxation	policy learning
Phylogenetic tree, Jukes–Cantor	✓	✓	✓	–	–	✓	✓	✓	✓	–	✓	✓
Phylogenetic tree, general time-reversible	✓	✓	–	–	–	–	✓	–	–	–	–	–
Phylogenetic tree, parsimony	✓	✓	–	–	–	–	✓	–	–	–	–	–
Potts chain in a field	✓	✓	–	✓	–	–	–	–	–	–	✓	✓
Potts lattice and Markov random field	✓	✓	✓	–	–	–	–	✓	✓	–	–	–
Potts lattice in a per-site field	✓	✓	–	–	–	–	–	✓	✓	✓	–	✓
Frustrated Potts: triangular antiferromagnet	✓	–	✓	–	–	–	✓	✓	–	–	–	–
Planted spin glass, enumerable ground state	–	–	–	–	–	–	–	–	✓	–	–	–
Low-density parity-check code, Gallager (3,6)	✓	–	✓	–	–	–	–	–	–	–	–	–
Low-density parity-check code, bicycle	✓	–	✓	–	–	–	–	–	–	–	–	–
Calderbank–Shor–Steane code over a bicycle matrix	✓	–	✓	–	–	–	–	–	–	–	–	–
Turbo code, two (7,5) recursive systematic encoders	✓	✓	✓	–	–	–	–	–	–	–	–	–
Hidden Markov model, six emission families	✓	✓	✓	✓	✓	–	✓	✓	–	–	✓	✓
Coupled spatio-sequential model	✓	✓	✓	–	✓	✓	✓	✓	–	–	–	–
Count-pair coupled spatio-sequential model	✓	✓	–	–	✓	–	–	–	–	–	–	–
Gaussian mixture	✓	✓	–	✓	✓	✓	–	✓	–	–	–	–
Mixture of two-channel count emissions	✓	✓	–	–	✓	✓	–	–	–	–	–	–
Continuous test functions	–	✓	–	–	–	–	–	–	–	–	–	–

Table 2: The algorithms that apply to each problem, read from the problem catalogue, and what referees each: ✓ a test pinned to an oracle, ✓[†] a test pinned to the simulated truth only, ○ neither kind of test, and – the problem carries no method of that family. The tables report what the tree holds, so – reads as *not built* and never as *not applicable*; Section 16 says which pairings are the second. A dagger or a circle is a cell where an oracle is wanted and absent.

	enumeration or brute force	closed form or published value	independent algorithm	planted truth	referee, ci tier	referee, stress tier	referee, release tier
Phylogenetic tree, Jukes–Cantor	✓	✓	–	–	oracle	oracle	oracle
Phylogenetic tree, general time-reversible	✓	–	–	–	oracle	oracle	oracle
Phylogenetic tree, parsimony	✓	–	–	–	oracle	oracle	oracle
Potts chain in a field	✓	–	–	–	oracle	–	oracle
Potts lattice and Markov random field	✓	✓	✓	✓	oracle	oracle	oracle
Potts lattice in a per-site field	✓	–	✓	–	oracle	oracle	oracle
Frustrated Potts: triangular antiferromagnet	✓	✓	–	–	oracle	oracle	oracle
Planted spin glass, enumerable ground state	–	–	–	✓	oracle	–	oracle
Low-density parity-check code, Gallager (3,6)	✓	–	–	–	oracle	truth [†]	truth [†]
Low-density parity-check code, bicycle	✓	–	–	–	oracle	truth [†]	truth [†]
Calderbank–Shor–Steane code over a bicycle matrix	✓	–	–	–	oracle	truth [†]	truth [†]
Turbo code, two (7,5) recursive systematic encoders	✓	✓	–	–	oracle	oracle	oracle
Hidden Markov model, six emission families	✓	✓	✓	–	oracle	oracle	oracle
Coupled spatio-sequential model	✓	✓	✓	–	oracle	oracle	oracle
Count-pair coupled spatio-sequential model	–	–	–	–	oracle	–	oracle
Gaussian mixture	–	–	✓	–	oracle	–	oracle
Mixture of two-channel count emissions	✓	–	–	–	oracle	–	oracle
Continuous test functions	–	✓	–	–	oracle	–	–

Table 3: The oracle each problem’s evaluators and searches are pinned to, and, per size tier, what referees the tests that pin them: an oracle, the simulated truth only (dagger: an oracle is wanted), or no test of either kind at that tier (–). A planted truth is an upper bound on the optimum rather than the optimum.

Problem	Tier	Referee	When it wins, and when it does not
Phylogenetic tree, Jukes–Cantor	ci	oracle	Ties the objective’s own start at five and six taxa, every start converging in 22 to 27 evaluations (experiment 006); the joining start is the only one available past the enumerable taxon count, where nothing referees it.
Phylogenetic tree, general time-reversible	–	untested	The log-det distance is additive under any Markov process, so it starts a fit the Jukes–Cantor distance cannot; no test of either significant kind names it as a start, which is why the pairing reads untested.
Phylogenetic tree, parsimony	–	–	–
Potts chain in a field	–	–	–
Potts lattice and Markov random field	–	–	–
Potts lattice in a per-site field	–	–	–
Frustrated Potts: triangular antiferromagnet	–	–	–
Planted spin glass, enumerable ground state	–	–	–
Low-density parity-check code, Gallager (3,6)	–	–	–
Low-density parity-check code, bicycle	–	–	–
Calderbank–Shor–Steane code over a bicycle matrix	–	–	–
Turbo code, two (7,5) recursive systematic encoders	–	–	–
Hidden Markov model, six emission families	–	–	–
Coupled spatio-sequential model	ci	oracle	The annealed data-sampled burn-in beats a uniform start at an equal budget on the planted lattice, where it is the start and not the move that decides the labelling; it returns the enumerated maximum-posterior labelling on 9 of 12 instances and is within 1.05 nats on the rest.
Count-pair coupled spatio-sequential model	–	–	–
Gaussian mixture	ci	oracle	The seeding cost is within the published factor of the exact clustering cost, and on the enumerable instance the seeded start lands in the maximum-posterior assignment over all 65,536 on 20 of 20 seeds where uniform seeding lands in it on 10; what it buys past the basin is nothing, since expectation–maximization reaches the same optimum from either start.
Mixture of two-channel count emissions	ci	oracle	A component is placed on an observed pair rather than on a coordinate, so the seeding has a distance where a squared one would compare a depth with an allele count; against a uniform start on shared seeds the two reach the same optimum at the three-component instance and differ at ten, and the comparison is reported whichever way it falls rather than asserted in a direction.
Continuous test functions	–	untested	Multi-start is what makes Himmelblau’s four basins reachable at all; on a unimodal surface it spends the budget for nothing. Every fit from every restart lands within 6.2e-07 of one of the four published minimizers, so the closed form referees where in a basin the fit stopped and not only which basin it found.

Table 4: The initializers each problem carries: the size tier at which the pairing is validated, the kind of referee that validates it, and what the pairing is worth. Tier and referee are read from the suite, as in the two tables above — *oracle* a test pinned to an independent exact answer, *truth*[†] a test pinned to the simulated truth only, *untested* a family the problem carries that no test of either kind names, and – a family it carries no method of. The last column is the one hand-written part of these tables, and names the experiment that measured the comparison where one did.

Problem	Tier	Referee	When it wins, and when it does not
Phylogenetic tree, Jukes–Cantor	ci	oracle	Annealing over topologies returns the flat-prior weight over maximized likelihoods, which hill climbing cannot; it pays a full branch-length fit per step, so it loses wherever only the optimum is wanted.
Phylogenetic tree, general time-reversible	–	–	–
Phylogenetic tree, parsimony	–	–	–
Potts chain in a field	–	–	–
Potts lattice and Markov random field	ci	oracle	Cluster moves slow by $1.9\times$ between extent 8 and 24 where single-site sweeps slow by $3.2\times$, a gap of $2.1\times$ and widening (experiment 001); they refuse a negative coupling, where the single-site sweep is all there is.
Potts lattice in a per-site field	ci	oracle	The lattice’s sampler applies unchanged: a per-site field changes the energy and not the move. What the row adds is the rung the sampler is scored at — the exact normalizer at nine and at 72 sites, and at 5,041 the energy a discrete solver reaches, since nothing enumerates there. No experiment: a statement about where the sampler is refereed, not about where it wins.
Frustrated Potts: triangular antiferromagnet	ci	oracle	No sampler of its own: the lattice’s samplers apply unchanged, and the cluster moves refuse the negative couplings these instances are built from. What the row adds is the ground-state energy every search is scored against.
Planted spin glass, enumerable ground state	–	untested	Annealing is the sampler that separates from descent here, and the reason the instance is declared: single-site descent reaches the enumerated ground state from 0.079 of starts, so a sampler that escapes a local minimum has something to win against and an exact answer to win it against.
Low-density parity-check code, Gallager (3,6)	–	–	–
Low-density parity-check code, bicycle	–	–	–
Calderbank–Shor–Steane code over a bicycle matrix	–	–	–
Turbo code, two (7,5) recursive systematic encoders	–	–	–
Hidden Markov model, six emission families	ci	oracle	The block sweep draws a whole path from its conditional in one pass, where a single-site sweep would move one position at a time against the chain; over eight positions its draws agree with the enumerated path posterior site by site to 0.0146 and jointly over the lumped cells.
Coupled spatio-sequential model	ci	oracle	The factor-graph sweep is one algorithm over the whole coupled graph and is refereed by enumeration on the canonical instance; it is slower per sweep than the specialized Potts kernel it generalizes.
Count-pair coupled spatio-sequential model	–	–	–
Gaussian mixture	ci	oracle	Annealed Hamiltonian proposals are the only sampler the row declares, and the continuous parameters are why: the discrete problems’ single-site sweep has no counterpart over a mean and a covariance. No experiment separates it from the restarts and tempering the optimizer note reports, so the pairing is here because the sampler runs on this instance, not because it is known to win on it.
Mixture of two-channel count emissions	–	–	–
Continuous test functions	–	untested	The annealed Hamiltonian chain with a polishing fit is beaten by plain restarts on the mixture at 3,000 evaluations (experiment 004); on Rastrigin it is the only method that crosses the ripple, and which wins is a property of the surface.

Table 5: The samplers each problem carries: the size tier at which the pairing is validated, the kind of referee that validates it, and what the pairing is worth. Tier and referee are read from the suite, as in the two tables above — *oracle* a test pinned to an independent exact answer, *truth*[†] a test pinned to the simulated truth only, *untested* a family the problem carries that no test of either kind names, and – a family it carries no method of. The last column is the one hand-written part of these tables, and names the experiment that measured the comparison where one did.

Problem	Tier	Referee	When it wins, and when it does not
Phylogenetic tree, Jukes–Cantor	ci	oracle	Hill climbing reaches the enumerated optimum from 12 of 12 starts at six taxa; the learned policy ties it over one feature and wins over seven (experiment 005), while random restarts reach 1.000 at the same budget.
Phylogenetic tree, general time-reversible	ci	oracle	The same quasi-Newton fit over a longer parameter vector, refereed by the same brute-force marginalization; it recovers the rate matrix where the alignment identifies it and refuses an interval where it does not.
Phylogenetic tree, parsimony	ci	oracle	One post-order pass per candidate rather than one optimization, so an equal budget reaches a neighbourhood an order of magnitude larger; it climbs the wrong criterion in the Felsenstein zone.
Potts chain in a field	ci	oracle	The transfer matrix makes the chain’s normalizer exact at every length, so the gradient fit is refereed wherever it runs; nothing here separates it from enumeration, which is also exact at these sizes.
Potts lattice and Markov random field	ci	oracle	The minimum cut is exact and polynomial where the couplings are submodular; above two states only alpha expansion applies, with a factor-of-two bound rather than an optimum, and Max-Cut is bounded and never solved.
Potts lattice in a per-site field	ci	oracle	The minimum cut is exact at 5,041 sites, where a per-site field leaves the couplings submodular and the alphabet is two; above two states only alpha expansion applies. Its factor-of-two bracket is stated for a non-negative energy and this one is negative, so the bracket is checked shifted, and only at nine sites where the optimum is enumerated rather than bounded.
Frustrated Potts: triangular antiferromagnet	ci	oracle	No optimizer of its own. The exact cut refuses a non-submodular energy, so these instances are where descent, annealing and tempering are separated at an equal budget of sweeps and nothing is solved exactly.
Planted spin glass, enumerable ground state	ci	oracle	Descent is the baseline the instance defeats. The sizes are chosen so the ground state is enumerated rather than bounded, which is what lets a failure be a failure rather than an unknown.
Low-density parity-check code, Gallager (3,6)	ci	oracle	Sum-product decodes to the bitwise posterior and min-sum to the most probable codeword; both are exact on a cycle-free code and the Bethe approximation on a loopy one, where min-sum is the cheaper and the weaker and the gap is measured.
Low-density parity-check code, bicycle	ci	oracle	The same two updates on a different matrix. What the construction changes is the graph they run on: a circulant puts short cycles where a permutation would have spread them, and the decoder is the weaker of the two at every length and channel setting measured against a random draw of the same degrees.
Calderbank–Shor–Steane code over a bicycle matrix	ci	oracle	The same decoder, asked a different question: the residual it leaves is scored on whether it is a stabilizer rather than on whether it is zero. It is measured against exact degenerate maximum likelihood, which sums a coset rather than maximizing over one error, and the gap is the price of the four-cycles the self-orthogonality $\mathbf{H}\mathbf{H}^\top = 0$ forces into the graph.
Turbo code, two (7,5) recursive systematic encoders	ci	oracle	BCJR is exact on one constituent code, since a terminated trellis is a chain, and Viterbi over the same branch metrics returns the most likely message rather than the most likely bits — two answers, and a received word separates them. The pair is loopy sum-product on a serial schedule and is exact nowhere: at $K = 12$ its bit error rate is 1.26 to 3.20 times the enumerated bitwise MAP’s, and it is not monotone in the iteration count, every rise lying inside one binomial interval of the sample. The iteration wins where the interleaver is long: at $K = 256$ it is under the uncoded closed form at every declared point and at $K = 1,024$ the curve turns between 0.4 and 1.2 dB.
Hidden Markov model, six emission families	ci	oracle	Expectation–maximization re-estimates in closed form for every family whose M step has one and wins on cost there; the gradient fit is the only route where it does not, and the two must reach the same optimum. Interval coverage is nominal from two standard deviations of separation upward (experiment 002).
Coupled spatio-sequential model	ci	oracle	Block-coordinate ascent does not decrease the joint at a 55 block, for every label solver; iterated conditional modes freezes into disjoint clusters where the cluster move does not, and which wins is measured at an equal budget rather than assumed.
Count-pair coupled spatio-sequential model	ci	oracle	Block-coordinate ascent recovers the planted labelling stream declared in faster on the 5,041 system in

Problem	Tier	Referee	When it wins, and when it does not
Phylogenetic tree, Jukes–Cantor	ci	oracle	A surrogate-ranked neighbourhood reaches its optimum at 5 fits against 312, and changes only what is fitted; it wins nothing where the neighbourhood is small enough to fit exhaustively.
Phylogenetic tree, general time-reversible	–	–	–
Phylogenetic tree, parsimony	–	–	–
Potts chain in a field	ci	oracle	The relaxation is exact at every corner to 1e-11 and supplies a gradient the discrete objective has not; it buys nothing at a size enumeration reaches, and the learned policy beats greedy on the same landscape (experiment 003).
Potts lattice and Markov random field	–	untested	Belief propagation is exact on a tree and the Bethe approximation otherwise, so it stands in for the normalizer at sizes enumeration cannot reach; on a loopy graph its error is reported and asserted against nothing.
Potts lattice in a per-site field	ci	oracle	The rung decides which surrogate is even defined. Enumeration and the column transfer matrix give the exact normalizer at nine and at 72 sites, so a fit is scored against it there; at 5,041 neither runs and the target is the energy a discrete solver reaches. The mean-field and spanning-tree bounds bracket the first two rungs and the spanning-tree bound does not reach the third, where a decoupled pair costing one pass over the sites and one over the edges brackets instead — looser by an order of magnitude, and the only bracket available.
Frustrated Potts: triangular antiferromagnet	–	–	–
Planted spin glass, enumerable ground state	–	–	–
Low-density parity-check code, Gallager (3,6)	–	–	–
Low-density parity-check code, bicycle	–	–	–
Calderbank–Shor–Steane code over a bicycle matrix	–	–	–
Turbo code, two (7,5) recursive systematic encoders	–	–	–
Hidden Markov model, six emission families	ci	oracle	The relaxation of the state path is exact at every corner and gives a gradient over paths; it says nothing at a length past which the path enumeration that referees it cannot run.
Coupled spatio-sequential model	–	–	–
Count-pair coupled spatio-sequential model	–	–	–
Gaussian mixture	–	–	–
Mixture of two-channel count emissions	–	–	–
Continuous test functions	–	–	–

Table 7: The surrogates each problem carries: the size tier at which the pairing is validated, the kind of referee that validates it, and what the pairing is worth. Tier and referee are read from the suite, as in the two tables above — *oracle* a test pinned to an independent exact answer, *truth*[†] a test pinned to the simulated truth only, *untested* a family the problem carries that no test of either kind names, and – a family it carries no method of. The last column is the one hand-written part of these tables, and names the experiment that measured the comparison where one did.

Table 8: Every instance the registry declares, and the shape each file states in its own words. The tiers of one problem differ in more than one quantity at once — extent and state count, taxa and sites, length and rate — so the column is a shape and never a size. A field a fixture declares appears here the day it is written; a transition matrix, a tree and a size distribution are the instance’s parameters rather than its extent and are left to the file.

Problem	Tier	Declared shape
Low-density parity-check code, bicycle	ci	n.bits 12; n_checks 6; circulant_weight 3; flip_probability 0.15; erasure_probability 0.4; noise_scale 1
Low-density parity-check code, bicycle	release	n.bits 996; n_checks 498; circulant_weight 3; flip_probability 0.05; erasure_probability 0.4; noise_scale 0.8
Low-density parity-check code, bicycle	stress	n.bits 96; n_checks 48; circulant_weight 3; flip_probability 0.05; erasure_probability 0.4; noise_scale 1
Calderbank–Shor–Steane code over a bicycle matrix	ci	n.bits 16; n_checks 7; circulant_weight 3; flip_probability 0.05
Calderbank–Shor–Steane code over a bicycle matrix	stress	n.bits 96; n_checks 40; circulant_weight 3; flip_probability 0.02
Mixture of two-channel count emissions	ci	n.samples 900; weights 0.5, 0.3, 0.2; family count-pair-joint; dispersion 6, 10, 15; mean 30, 80, 200; alpha 1.8, 6, 9.6; beta 10.2, 6, 2.4
Mixture of two-channel count emissions	stress	n.samples 3000; weights 10 values; family count-pair-joint; dispersion 10 entries of 60; mean 10 values; alpha 10 values; beta 10 values
Frustrated Potts: triangular antiferromagnet	ci	shape 3, 3; boundary periodic; coupling -1; glass_nodes 60; glass_mean_degree 4; glass_magnitude 1; glass_frustrations 8 values; glass_restarts 20
Hidden Markov model, six emission families	ci	n.states 3; n_symbols 4; lengths 600 entries of 15; initial 0.5, 0.3, 0.2
Low-density parity-check code, Gallager (3,6)	ci	n.bits 12; column_weight 3; row_weight 6; flip_probability 0.15; erasure_probability 0.4; noise_scale 1
Low-density parity-check code, Gallager (3,6)	release	n.bits 996; column_weight 3; row_weight 6; flip_probability 0.05; erasure_probability 0.4; noise_scale 0.8
Low-density parity-check code, Gallager (3,6)	stress	n.bits 96; column_weight 3; row_weight 6; flip_probability 0.05; erasure_probability 0.4; noise_scale 1
Gaussian mixture	ci	n.samples 500; weights 5 values; means 5 values; scales 5 entries of 1; variance_floor 1e-12
Gaussian mixture	release	n.samples 20164000; weights 10 values; variance_floor 1e-12
Planted spin glass, enumerable ground state	ci	shape 3, 3; boundary periodic; coupling -1; glass_nodes 18; glass_mean_degree 4; glass_magnitude 1; glass_frustrations 0.3; glass_restarts 50
Potts chain in a field	ci	n.states 3; chain_length 12; n_chains 400; coupling 0.75; field 0.4, -0.1, -0.3
Potts lattice and Markov random field	ci	shape 3, 3; boundary open; n_states 3; coupling 0.6; field 0.3, -0.1, -0.2; n_samples 2000; burn_in 500
Potts lattice and Markov random field	stress	shape 12, 12; boundary open; n_states 3; coupling critical; field 0, 0, 0; n_samples 1500; burn_in 300
Hidden Markov model, six emission families	ci	n.states 3; n_symbols 4; lengths 5 values; initial 0.5, 0.3, 0.2
Potts lattice in a per-site field	ci	geometry square; shape 3, 3; boundary open; n_classes 3; coupling 0.6; alpha 1, -1, 0; n_samples 2000; burn_in 500
Potts lattice in a per-site field	release	geometry triangular; shape 71, 71; boundary open; n_classes 10; coupling 0.7; alpha 10 values; n_samples 8; burn_in 60
Potts lattice in a per-site field	stress	geometry square; shape 12, 6; boundary open; n_classes 3; coupling 0.6; alpha 1, -1, 0; n_samples 2000; burn_in 500
Coupled spatio-sequential model	ci	shape 2, 2; boundary open; coupling 1; n_classes 2; n_states 2; n_positions 6; beta 0.6; self_transition 0.7

Problem		Tier	Declared shape
Coupled spatio-sequential model		stress	shape 10, 10; boundary open; coupling 1; n_classes 2; n_states 2; n_positions 4; beta 1; self_transition 0.7
Count-pair coupled spatio-sequential model		ci	shape 8, 8; boundary open; coupling 1; n_classes 2; n_states 2; n_positions 1000; beta 1; self_transition 0.7
Count-pair coupled spatio-sequential model		stress	shape 71, 71; boundary open; coupling 1; n_classes 10; n_states 10; n_positions 20000; beta 1; self_transition 0.7
Coupled spatio-sequential model		ci	shape 2, 2; boundary open; coupling 1; n_classes 2; n_states 2; n_positions 6; segments 2, 4; beta 0.6; self_transition 0.7
Continuous test functions		ci	grid 81; at_minimum 0.0001
Phylogenetic general time-reversible; Parsimony	tree, tree,	ci	n_sites 20000; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic general time-reversible; Parsimony	tree, tree,	release	n_sites 200000; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic general time-reversible; Parsimony	tree, tree,	stress	n_sites 200000; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic Jukes–Cantor	tree,	ci	n_sites 2000; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic Jukes–Cantor	tree,	release	n_sites 2000; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic Jukes–Cantor	tree,	stress	n_sites 2000; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic Jukes–Cantor	tree,	ci	n_sites 1200; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic Jukes–Cantor	tree,	release	n_sites 2000; k 4; pi 0.25, 0.25, 0.25, 0.25
Phylogenetic Jukes–Cantor	tree,	stress	n_sites 1500; k 4; pi 0.25, 0.25, 0.25, 0.25
Turbo code, two (7,5) recursive systematic encoders		ci	message_length 12; memory 2; feedback 7; feedforward 5; iterations 8; eb_n0_db 0, 1, 2, 3; frames 100
Turbo code, two (7,5) recursive systematic encoders		release	message_length 1024; memory 2; feedback 7; feedforward 5; iterations 8; eb_n0_db 6 values; frames 200
Turbo code, two (7,5) recursive systematic encoders		stress	message_length 256; memory 2; feedback 7; feedforward 5; iterations 8; eb_n0_db 6 values; frames 50

Table 9: The oracle ladder per problem, generated from its one declaration: each rung, the callable it names, the rung it is pinned against, and the test pinning the pair. A rung with nothing under it is pinned to an exact referee outside the ladder — an enumeration at its foot, a closed form, a second implementation — which the test names. A rung reading *ticket* is one no test pins yet; it is a gap with an issue behind it and never a blank, and a rung appears once per rung it is pinned against, so a method pinned at the exact end and wanted against a cheaper rung is two rows.

Problem	Rung	Callable	Pinned against	Pinned by
potts	enumeration	likelihood.potts.enumerate_potts	–	search/test_search_support: test_the_enumerated_labelling_weight_is_enumerate_potts_s_boltzmann_weight_at_beta_one
potts	transfer matrix	likelihood.potts.strip_log_partition	enumeration	likelihood/test_potts_exact: test_the_transfer_matrix_reproduces_exhaustive_enumeration
potts	transfer matrix	likelihood.potts.strip_log_partition	sum-product / BP	likelihood/test_potts_exact: test_the_transfer_matrix_is_sum_product_on_the_strip_that_is_a_tree
potts	sum-product / BP	likelihood.message_passing.sum_product	enumeration	likelihood/test_message_passing: test_sum_product_on_the_potts_tree_is_the_enumeration
potts	flooding schedule	likelihood.belief_propagation.belief_propagation	sum-product / BP	likelihood/test_message_passing: test_flooding_on_the_loopy_lattice_is_belief_propagation
potts	Kikuchi / region graph	sandbox.region_graph.kikuchi_free_energy	enumeration	sandbox/test_region_graph: test_the_free_energy_is_the_exact_log_partition_on_a_tree
potts	dual / LP bound	search.tightening.dual_bound	enumeration	search/test_tightening: test_the_bound_never_exceeds_the_enumerated_ground_state
potts	dual / LP bound	search.tightening.dual_bound	sum-product / BP	search/test_tightening: test_the_dual_bound_is_the_zero_temperature_belief_propagation_energy
potts	variational bounds	likelihood.surrogate.mean_field_log_partition	enumeration	likelihood/test_surrogate: test_mean_field_and_spanning_tree_bounds_sandwich_log_z
potts	exact cut / max-flow	search.maxflow.ising_ground_state	enumeration	search/test_maxflow: test_the_cut_finds_the_enumerated_minimum_with_a_per_node_field
potts	declined max-flow kernels	sandbox.maxflow_declined.min_cut	exact cut / max-flow	sandbox/test_maxflow_declined: test_every_declined_kernel_returns_the_python_cut_on_seeded_networks
potts	max-cut (SDP)	search.max_cut.goemans_williamson	enumeration	search/test_max_cut: test_the_rounded_cut_reaches_the_enumerated_optimum_on_a_lattice
potts	max-cut (SDP)	search.max_cut.goemans_williamson	alpha-expansion	search/test_max_cut: test_the_rounded_cut_is_the_gauged_alpha_expansion_optimum_at_two_labels
potts	alpha-expansion	search.alpha_expansion.alpha_expansion	exact cut / max-flow	search/test_alpha_expansion: test_two_labels_reproduce_the_exact_minimum_cut
potts	alpha-expansion backends	search.alpha_expansion.alpha_expansion	alpha-expansion	search/test_alpha_expansion: test_the_rust_cut_reproduces_the_python_expansion
potts	ICM	search.alpha_expansion.iterated_conditional_modes	exact cut / max-flow	search/test_potts_sizing: test_the_zero_field_optimum_is_a_closed_form_three_ways

Problem	Rung	Callable	Pinned against	Pinned by
potts	Gibbs / heat bath	search.gibbs.gibbs_sweep	enumeration	sim/test_potts_simulate: test_gibbs_sampling_matches_brute_force_enumeration_on_a_loopy_lattice
potts	Potts MCMC, Wolff, Swendsen–Wang	search.potts_mcmc.sample_potts	enumeration	search/test_potts_mcmc: test_the_chain_is_drawn_from_the_exact_boltzmann_distribution
potts	cluster moves as RL arms	search.potts_keyed.cluster_moves	Potts MCMC, Wolff, Swendsen–Wang	learn/test_cluster_arms: test_a_wolff_step_is_potts_mcmc_own_sweep_bitwise
potts	annealing	search.potts_mcmc.anneal_potts	–	search/test_potts_mcmc: test_annealing_reaches_the_closed_form_ground_energy_where_descent_does_not
potts	parallel tempering	search.potts_mcmc.parallel_tempering	annealing	search/test_potts_mcmc: test_tempering_reaches_the_ground_energy_annealing_reaches
potts	learned policy	learn.potts_nd.PottsNDEnvironment	exact cut / max-flow	learn/test_potts_nd: test_two_labels_agree_with_the_exact_cut
potts	learned lattice surrogate	search.surrogate.lattice_examples	enumeration	search/test_search_lattice_surrogate: test_a_surrogate_learns_the_gap_above_the_mean_field_bound_at_nine_sites
potts	run_tempering, run_greedy, run_max_product	search.ground_state.run_tempering	–	search/test_ground_state: test_the_runners_record_the_energy_their_kernels_return
tree	projection	search.projection.project	–	search/test_projection_seeding: test_the_projected_draw_is_the_closed_form_mixture_of_the_flattened_families
tree	topology enumeration	search.topology.enumerate_topologies	–	search/test_search_exhaustive: test_enumeration_produces_every_topology_exactly_once
tree	topology equality	search.topology.robinson_foulds	–	search/test_search_topology_rustworkx: test_two_topologies_our_code_calls_equal_are_isomorphic
tree	parsimony, Fitch	likelihood.parsimony.fitch_score	–	likelihood/test_likelihood_parsimony: test_fitch_matches_exhaustive_enumeration_over_internal_labellings
tree	parsimony, Sankoff	likelihood.parsimony.sankoff_score	parsimony, Fitch	likelihood/test_likelihood_parsimony: test_sankoff_with_the_unit_matrix_is_fitch_on_every_five_taxon_topology
tree	parsimony against likelihood	likelihood.parsimony.fitch_score	pruning, NumPy	likelihood/test_likelihood_parsimony: test_the_short_branch_likelihood_ranks_the_topologies_as_the_fitch_score_does
tree	pruning, NumPy	likelihood.pruning.log_likelihood	–	likelihood/test_likelihood_pruning: test_pruning_matches_brute_force
tree	pruning, Rust	likelihood.pruning_rust.log_likelihood	pruning, NumPy	likelihood/test_pruning_rust: test_rust_matches_numpy_oracle
tree	pruning, Torch	likelihood.pruning_torch.log_likelihood	pruning, NumPy	likelihood/test_pruning_torch: test_torch_matches_numpy_oracle
tree	pruning, analytic gradient	likelihood.pruning_analytic.log_likelihood	pruning, Torch	likelihood/test_pruning_gradient: test_every_route_agrees_with_the_taped_gradient

Problem	Rung	Callable	Pinned against	Pinned by
tree	Hadamard / spectral	likelihood.hadamard.hadamard_conjugation	–	likelihood/test_likelihood_hadamard: test_the_conjugation_returns_the_true_split_weights_on_the_exact_spectrum
tree	distance start (NJ)	search.neighbor_joining.neighbor_joining	–	search/test_search_neighbor_joining_scipy: test_the_two_return_the_same_tree_on_an_ultrametric_matrix
tree	distance start (NJ)	search.neighbor_joining.neighbor_joining	topology enumeration	search/test_neighbor_joining: test_the_joined_tree_is_the_least_squares_optimum_over_the_enumerated_topologies
tree	tropical / Grassmannian	sandbox.tropical.resolutions	–	sandbox/test_sandbox_tropical: test_the_combinatorial_resolution_is_the_tropical_plucker_argmin
tree	tree search (NNI/SPR)	search.infer.infer	topology enumeration	search/test_search_exhaustive: test_hill_climbing_reaches_the_enumerated_maximum
tree	cheaper searches	search.infer.infer	topology enumeration	search/test_search_exhaustive: test_the_cheaper_searches_reach_the_enumerated_maximum
tree	analytic surrogates	likelihood.surrogate.PlugInLikelihood	pruning, NumPy	likelihood/test_surrogate: test_plug_in_bound_is_below_every_fitted_likelihood
tree	surrogate-ranked search	search.surrogate.LearnedTreeSurrogate	tree search (NNI/SPR)	search/test_search_surrogate: test_surrogate_ranked_search_reaches_what_the_full_search_reaches
tree	surrogate-ranked search	search.surrogate.LearnedTreeSurrogate	analytic surrogates	search/test_search_surrogate: test_the_learned_surrogate_ranks_the_topologies_the_plug_in_bound_ranks
tree	learned GNN surrogate	learn.surrogate.GraphSurrogate	–	learn/test_learn_surrogate_pyg: test_pyg_s_gin_reproduces_the_graph_surrogate_on_tied_weights
tree	RL policy / PPO	search.rl.TreeEnvironment	topology enumeration	search/test_search_rl: test_greedy_search_reaches_the_enumerated_optimum
tree	support / tempering over topologies	search.support.neighbourhood_support	topology enumeration	search/test_search_support: test_the_nni_neighbourhood_of_four_taxa_is_the_whole_space_so_the_two_supports_agree
tree	Gibbs / annealing over topologies	search.gibbs.topology_step	topology enumeration	search/test_gibbs: test_the_topology_move_at_temperature_one_samples_the_enumerated_flat_prior_weight
hmm	path enumeration	likelihood.hmm.paths.enumerate_hidden_paths	–	likelihood/test_hmm_paths: test_a_marginal_is_the_summed_joint_over_paths_through_that_state
hmm	forward / forward-backward	likelihood.forward_backward.forward_backward	path enumeration	opt/test_opt_hmm: test_forward_matches_brute_force_path_enumeration
hmm	path sampling	likelihood.forward_backward.sample_path	path enumeration	likelihood/test_hmm_paths: test_the_sampled_paths_are_drawn_from_the_enumerated_path_posterior
hmm	forward via sum-product	likelihood.message_passing.sum_product	forward / forward-backward	likelihood/test_message_passing: test_the_tree_schedule_on_a_deep_chain_is_the_forward_recursion
hmm	Viterbi	likelihood.message_passing.max_product	path enumeration	likelihood/test_message_passing: test_max_product_on_the_chain_is_viterbi

Problem	Rung	Callable	Pinned against		Pinned by
hmm	Viterbi	likelihood.message_passing.max_product	coupled E and M steps		likelihood/test_message_passing: test_max_product_decodes_the_mode_of_the_coupled_e_step_s_path_law
hmm	relaxed MAP	learn.relaxed.optimize	Viterbi		learn/test_learn_relaxed: test_the_relaxed_optimum_of_the_hmm_is_the_viterbi_path
hmm	Baum–Welch	opt.hmm.baum_welch	path	enumeration	opt/test_opt_hmm: test_baum_welch_reaches_the_enumerated_path_evidence_and_its_fixed_point
hmm	Baum–Welch	opt.hmm.baum_welch	–		opt/test_opt_hmm: test_baum_welch_ascends_and_settles_on_the_re_estimation_equations
hmm	ragged HMM	likelihood.ragged_rust.posteriors	forward	forward-backward	opt/test_ragged_hmm: test_equal_lengths_reproduce_the_conserved_route_bitwise
hmm	RL over paths	learn.hmm.optimum	path	enumeration	learn/test_learn_hmm: test_hill_climbing_reaches_the_enumerated_optimum
hmm	coupled model, exact	likelihood.spatio_sequential.enumerate_spatio_sequential	–		likelihood/test_spatio_sequential: test_the_enumerated_evidence_equals_the_per_class_forward_route
hmm	coupled E and M steps	likelihood.spatio_sequential.class_posteriors	coupled	model, exact	likelihood/test_spatio_sequential_fit: test_the_class_e_step_is_the_conditional_posterior_by_enumeration
hmm	coupled, Rust	likelihood.spatio_sequential_rust.class_posteriors	coupled E and M steps		likelihood/test_spatio_sequential_rust: test_the_rust_e_step_and_field_match_the_numpy_oracle
hmm	coupled label search	search.spatio_sequential.label_step	coupled	model, exact	search/test_spatio_sequential_fit: test_the_label_step_reaches_the_enumerated_map_from_the_planted_labels
hmm	coupled Gibbs	search.gibbs.sample_factor_graph	coupled	model, exact	search/test_gibbs: test_the_coupled_sweep_draws_labellings_from_the_enumerated_joint_law
codes	brute-force ML	likelihood.ldpc.enumerate_codewords	–		likelihood/test_ldpc: test_the_enumeration_oracle_on_a_single_parity_check_by_hand
codes	BCJR	likelihood.convolutional.bcjr	brute-force ML		likelihood/test_convolutional: test_bcjr_posteriors_are_the_exact_bitwise_map
codes	Viterbi on the trellis	likelihood.convolutional.viterbi	brute-force ML		likelihood/test_convolutional: test_viterbi_returns_the_maximum_likelihood_message_with_a_pinned_margin
codes	turbo	likelihood.turbo.decode_turbo	brute-force ML		likelihood/test_turbo: test_the_joint_posterior_is_the_enumerated_one_where_the_two_chains_agree
codes	turbo	likelihood.turbo.decode_turbo	BCJR		likelihood/test_turbo: test_the_turbo_posterior_is_bcjr_on_the_first_constituent_alone
codes	LDPC sum-product / min-sum	likelihood.ldpc.decode	brute-force ML		likelihood/test_ldpc: test_sum_product_is_exact_on_a_cycle_free_code
codes	bicycle, loopy	sim.ldpc.bicycle_code	brute-force ML		likelihood/test_bicycle: test_the_exact_decodings_bound_belief_propagation
codes	CSS / syndrome	likelihood.css.decode_syndrome	brute-force ML		likelihood/test_css: test_summing_a_coset_beats_maximizing_over_one_error

Problem	Rung	Callable	Pinned against		Pinned by
codes	CSS / syndrome	likelihood.css.decode_syndrome	LDPC product	sum-min-sum	likelihood/test.css: test_the_syndrome_decode_is_belief_propagation_on_the_component_code
codes	polar SC / SCL	sandbox.polar_decoding.decode_sc	–		sandbox/test.polar_decoding: test_successive_cancellation_is_the_reference_implementation_bitwise
codes	polar SC / SCL	sandbox.polar_decoding.decode_sc	LDPC product	sum-min-sum	sandbox/test.polar_decoding: test_successive_cancellation_against_min_sum_on_the_same_parity_check
codes	polar construction	sandbox.polar.polar_information_set	–		sandbox/test.polar: test_the_polar_and_reed_muller_rules_coincide_at_eight_and_part_at_sixteen
codes	elementary codes	sim.elementary_codes.single_parity_check	–		sim/test.elementary_codes: test_the_single_parity_check_posterior_is_the_tanh_rule
codes	hamming_correct, golay_code	sim.elementary_codes.hamming_correct	brute-force ML		sim/test.elementary_codes: test_syndrome_correction_is_the_nearest_codeword_enumeration_returns
codes	Reed–Solomon	sim.reed_solomon.decode	–		sim/test.reed_solomon: test_reed_solomon_meets_the_singleton_bound_with_equality
codes	Reed–Solomon	sim.reed_solomon.decode	brute-force ML		sim/test.reed_solomon: test_the_algebraic_decode_is_the_nearest_codeword_enumeration_returns
mixture	assignment enumeration	likelihood.mixture_assignments.enumerate_mixture_assignments	–		opt/test.opt_mixture: test_the_evidence_and_the_e_step_match_the_enumerated_assignments
mixture	GMM EM	opt.mixture.expectation_maximization	assignment	enumeration	opt/test.opt_mixture: test_the_evidence_and_the_e_step_match_the_enumerated_assignments
mixture	emission-mixture EM	opt.emission_mixture.expectation_maximization	assignment	enumeration	opt/test.opt_emission_mixture: test_the_responsibilities_are_the_enumerated_posterior
mixture	k-means++ seeding	opt.mixture.kmeans_plus_plus	–		opt/test.opt_mixture: seeding: test_the_shipped_rule_is_k_means_plus_plus_on_a_gaussian
mixture	optimal clustering cost	opt.mixture.optimal_clustering_cost	assignment	enumeration	opt/test.opt_mixture: test_the_optimal_clustering_cost_is_the_minimum_over_the_enumerated_assignments
mixture	projection seeding	search.projection.euclidean_seeding	k-means++	seeding	search/test.projection_seeding: test_euclidean_seeding_is_one_dimensional_kmeans_plus_plus_on_a_flat_channel
mixture	emission_seeding, data_seeding	search.projection.emission_seeding	k-means++	seeding	search/test.projection_seeding: test_the_non_euclidean_seedings_draw_the Law_of_the_metric.they_declare
mixture	HMC	opt.hmc.sample	–		opt/test.opt_hmc: test_the_chain_recovers_an_analytic_gaussian
mixture	HMC	opt.hmc.sample	assignment	enumeration	opt/test.opt_hmc: test_the_chain_recovers_the_enumerated_assignment_posterior_of_a_mixture

mixture	FromChain, FromAnnealing, FromTempering	opt.initialize.FromChain	—	opt/test_opt_initialize: test_ the_sampled_starts_are_their_ runs_own_records_and_leave_ the_cell_descent_cannot
---------	---	--------------------------	---	---

Part II

The Algorithms

Every algorithm this document uses is stated once. Where one serves several problems it is stated in general terms in Appendix G and specialised here, problem by problem, in the order the problems were stated. Where one serves a single problem it is stated in that problem’s section below.

17 Factor Graph Optimization

Let the objective be a differentiable scalar $\mathcal{L}(\boldsymbol{\theta})$ over an unconstrained vector $\boldsymbol{\theta} \in \mathbb{R}^d$, with a map ϕ carrying $\boldsymbol{\theta}$ to the constrained parameters a model is stated in: positive branch lengths through a logarithm, a distribution over k states through a softmax on $k - 1$ free values, a positive scale through a logarithm.

Why the map and not a projection. An optimizer that must be *stopped* from leaving the feasible set will eventually leave it, and a constrained parameter at its boundary has no interval worth reporting. Constraining by construction removes both: every point in $\mathbb{R}^d$ is feasible, and $\nabla_{\boldsymbol{\theta}}\mathcal{L}$ is defined everywhere.

Why the map must be injective. Where a reparameterization leaves $\mathcal{L}$ unchanged, the direction along it is not estimable: the observed information is singular and an interval undefined. A softmax on k free values has that flat direction—adding a constant to every value changes nothing—which is why $k - 1$ are used. Removing the freedom is the constraint map’s job, not the optimizer’s.

What follows for inference. Standard errors come from the observed information at the optimum, pushed through ϕ by the delta method: $\text{Var}(\phi(\boldsymbol{\theta})) \approx J\Sigma J^\top$ with J the Jacobian of ϕ . An *asymptotic* statement, exact only in the large-sample limit, which is why Section 34 makes realized interval coverage a measurement.

18 Initializers

Every search and every fit below starts somewhere, and where it starts is a declared part of the method rather than a detail of a run. Four kinds are carried, and they differ in what they read: a start that reads nothing, a start that reads the data, a start that reads a moment of the data and inverts it, and a start that reads the objective through a sampler. The kinds are not interchangeable, because what each can be held to is different.

Model-free restarts. A draw from a declared generator around a declared point, repeated, Algorithm 8. It reads nothing and assumes nothing, so it applies to every problem here and is the baseline every other start is measured against at an equal budget of objective evaluations. On a multimodal surface it is also the only start with a *rate* attached — the fraction of starts

from which the fit reaches the global optimum, which Section 23 measures rather than asserts. Its cost is the restart count, and that count is part of the budget the comparison charges.

Data-sampled seeds. A start drawn from the observations, proportionally to how badly the current parameters explain them: *k*-means++ on a mixture, Algorithm 6, and its family-aware form on an emission mixture, where the family’s own negative log-density replaces the squared distance. What it inherits from the published bound is exactly the case the bound was proved for: the $8(\ln K + 2)$ expected-cost guarantee is a statement about squared Euclidean distance [6], so a seeding under another divergence is measured against a uniform start over seeds and reported whether it wins or not [11].

Moment and spectral starts. A start that inverts an observable moment to the parameters, with a consistency guarantee and no surface to climb: the pairwise distances and the agglomeration over them for a tree (Algorithm 3), the exact two-state spectrum of Equation 64 (Algorithm 7), and the window moments of a chain [38, 52]. This is the only kind carrying an argument rather than a measurement — Atteson’s radius says when the agglomeration recovers the topology, and the conjugation is exact on an exact spectrum — and the only kind that stops existing above a size, which is why Section 27 states the taxon count at which it does.

Annealed and sampled starts. A start produced by running the sampler itself at a temperature, and lowering it: the data-sampled burn-in of Algorithm 5, which anneals in β from zero so the labels are nearly free while the emission parameters are fitted, and simulated annealing read as an initializer for a descent that polishes what it returns. A schedule is one object with both endpoints reached exactly at the declared step count (Appendix B), so an annealed start is reproducible in the sense the others are: one seed, one schedule, one result. Whether annealing beats restarts of descent at an equal budget of sweeps is a property of the instance and is measured on Section 11’s, never asserted in general.

A fifth kind is available and deliberately not used as a start: a tempered ensemble estimates a *distribution* rather than a point, so its cold replica’s weights, Equation 86, are an answer and not a starting point. Reading one as the other is the standing error about Appendix B.

19 Relaxing a Discrete Decision

A third route past a discrete search is to make it continuous. Write a configuration over L sites and k states as a row-stochastic q , one simplex per site, and read the discrete score on it. Two demands separate a relaxation from a second model, and both are checkable.

It must be an extension of what it relaxes. At every vertex — every one-hot q — the relaxed value must equal the discrete score exactly, asserted over every configuration of an enumerable instance. A relaxation merely close at the corners optimizes a different problem and nothing measured on it transfers — the demand Section 27.8 makes of the tropical relaxation, and the reason Equation 66 is a derived tolerance rather than a constant.

Its boundary is multilinearity, not the shape of the graph. If every term of the score involves at most one factor from each site, then under a factorized q distinct sites are independent and each term’s expectation is that term at the marginals:

$$\mathbb{E}_q[\text{score}(x)] = \text{score}(q), \quad (40)$$

exactly, so the form evaluated at the marginals *is* the expected discrete score and not an approximation. The condition is easy to state wrongly. It is not that the model must be a chain — a lattice is as multilinear — nor that the terms must be pairwise, a term coupling three *distinct* sites being one factor per site. What breaks it is a term reading one site twice, since for an

indicator $\mathbb{E}[X^2] = \mathbb{E}[X] \neq \mathbb{E}[X]^2$. Two things follow. A multilinear function on a product of simplices attains its maximum at a vertex, so the relaxation introduces no optimum the discrete problem lacks and what a relaxed search loses is lost to the ascent. And the deterministic relaxation needs no sampling, which sharpens the question about the stochastic one: since Equation 40 is exact, whatever bias an estimator shows belongs to the estimator.

Sampling a relaxed configuration is the Gumbel-softmax: perturb each site’s log-probabilities by independent Gumbel noise and take a softmax at temperature τ where an $\arg \max$ would give a draw, which is reparameterized — differentiable in the probabilities — and biased at every $\tau > 0$, exact only in a limit where the gradient vanishes. The bias is measured against the exact gradient enumeration supplies and never assumed small, and the score-function estimator of Equation 98 is the unbiased alternative at higher variance: Section C.7’s trade in a second place. Carrying the construction from one categorical variable to a *structured* object, where the $\arg \max$ is a combinatorial solver rather than a coordinate, is the stochastic-softmax-trick family [59] — what a differentiable topology would need, and the direction Section 27.8 approaches from the other side.

20 Surrogates and Bounds

The evaluation a discrete search pays for—a fit of τ per topology, or $\log Z$ for a lattice—is what makes the search expensive, so each problem carries *surrogates*: cheaper functions of the structure standing in for it. A surrogate makes a claim — a lower bound, an upper bound, or a point prediction — and the claim is part of the object. An analytic bound is proved (Appendix F) and *certified*: checked against the exact value on every structure an oracle can score, with no violation allowed. A learned predictor is *calibrated* instead, shifted by a quantile of its held-out residuals, and certified to the coverage it claims. The search ranks a neighbourhood by a surrogate and fits only the top candidates (Algorithm 20); the accepted move is always evaluated in full, so a surrogate changes what is fitted and never what is reported.

For a topology the maximized log-likelihood $\ell^*(\mathcal{T}) = \max_{\tau} \ell(\mathcal{T}, \tau)$ is bracketed by two one-pass quantities. Any feasible lengths give a lower bound, and lengths $\hat{\tau}$ solved by non-negative least squares from the pairwise Jukes–Cantor distances $\hat{d}_{ab}$ are feasible:

$$\ell(\mathcal{T}, \hat{\tau}) \leq \ell^*(\mathcal{T}), \quad \hat{\tau} = \arg \min_{\tau \geq 0} \sum_{a < b} \left(\sum_{e \in \text{path}(a,b)} \tau_e - \hat{d}_{ab} \right)^2. \quad (41)$$

Under Jukes–Cantor $\mathbf{P}(t) = a\mathbf{I} + (1-a)\mathbf{J}/k$ with $a = e^{-kt/(k-1)}$, so a site likelihood is multilinear in the a_e and its maximum over the box $[0, 1]^{|E|}$ sits at a vertex, where every branch either forbids a change or forgets the parent’s state at cost $1/k$. With the tree rooted at the first leaf, the vertex maximum of site s is $\pi(x_{1,s}) k^{-F_s(\mathcal{T})}$ for the site’s Fitch score F_s , whence

$$\ell^*(\mathcal{T}) \leq \sum_s \log \pi(x_{1,s}) - F(\mathcal{T}) \log k. \quad (42)$$

Both apply to a subtree as they do to the tree.

A third trades exactness for cost by the site count rather than the tree. Site-pattern compression, Equation 56, is exact and saturates; blocks of $N > 1$ consecutive sites repeat too rarely for the same identity to save anything, so a block partition buys a bracket instead. Partition the L columns into $\lceil L/N \rceil$ blocks of N consecutive sites, and let $\mathcal{F}$ be the blocks occurring at least m times. Evaluate those exactly, by Equation 56 over their columns, and bound the remaining B sites by the extremes of a single site’s log-likelihood over the whole column alphabet (Algorithm 21):

$$\ell_{\mathcal{F}} + B \underline{\lambda}(\mathcal{T}, \tau) \leq \ell(\mathcal{T}, \tau) \leq \ell_{\mathcal{F}} + B \bar{\lambda}(\mathcal{T}, \tau), \quad \begin{aligned} \underline{\lambda} &= \min_{x \in \mathcal{A}^n} \log p(x \mid \mathcal{T}, \tau, \mathbf{Q}), \\ \bar{\lambda} &= \max_{x \in \mathcal{A}^n} \log p(x \mid \mathcal{T}, \tau, \mathbf{Q}), \end{aligned} \quad (43)$$

with $\ell_{\mathcal{F}}$ the frequent blocks' own log-likelihood. Both $\underline{\lambda}$ and $\bar{\lambda}$ are had in one pass over the tree and independently of L , by Equation 54 with each leaf's message replaced by the extremes of its transition row (Appendix F); relaxing them to those one-pass values makes the bracket cheap and makes it loose. The width is exactly $B(\bar{\lambda} - \underline{\lambda})$, so the cutoff m sets the trade: $m = 1$ leaves $B = 0$ and returns the exact value, and raising m moves blocks into the tail and never out. Only the lower end bounds $\ell^*(\mathcal{T})$, by the argument Equation 41 rests on. Measured at five taxa and 2,000 sites, $N = 1$ and $m = 8$ leaves a width of $1.0|\ell|$ and $N \geq 3$ leaves the whole alignment in the tail, so neither end orders topologies and what ranks is $\ell_{\mathcal{F}}$ itself, a point prediction over a fixed subsample that claims nothing.

For a lattice the partition function is bracketed by two variational quantities. The naive mean-field bound is Gibbs' inequality at a product distribution $q = \prod_i q_i$, tightest at its fixed point:

$$\log Z \geq \sum_i \sum_{\sigma} q_i(\sigma) h_i(\sigma) + \sum_{\langle i,j \rangle} J_{ij} \sum_{\sigma} q_i(\sigma) q_j(\sigma) - \sum_i \sum_{\sigma} q_i(\sigma) \log q_i(\sigma). \quad (44)$$

Because $\log Z$ is convex in the parameters, Jensen's inequality over a distribution ρ on spanning trees T , with parameters $\theta(T)$ supported on T and averaging to θ , gives an upper bound each term of which is exact by one leaf-to-root pass:

$$\log Z(\theta) \leq \sum_T \rho(T) \log Z(\theta(T)), \quad \sum_T \rho(T) \theta(T) = \theta, \quad (45)$$

the tree-reweighted bound of Wainwright et al. [78]. From any bracket $L \leq \log Z(\beta) \leq U$ at inverse temperature β , and $e^{-\beta E_{\min}} \leq Z(\beta) \leq k^N e^{-\beta E_{\min}}$,

$$-\frac{U}{\beta} \leq E_{\min} \leq \frac{N \log k - L}{\beta}. \quad (46)$$

Both of those cost one exact tree pass per edge, which a lattice of thousands of sites does not admit. Two looser ones cost $O(N + |E|)$ and so reach any size. Write

$$E_{\text{dec}} = - \sum_i \max_{\sigma} h_i(\sigma) - \sum_{\langle i,j \rangle} \max(J_{ij}, 0) \leq E_{\min} \leq \mathcal{H}(\sigma) \quad \text{for every } \sigma, \quad (47)$$

each term of the energy being at least its own minimum, and any configuration's energy being at least the least. The same quantity carries a bracket on the partition function, the left end holding where every coupling is non-negative:

$$\sum_i \log \sum_{\sigma} e^{h_i(\sigma)} \leq \log Z \leq N \log k - E_{\text{dec}}. \quad (48)$$

21 Learned Proposals for a Discrete Search

The structural searches above choose a move from a neighbourhood by a rule fixed in advance — take the best, or take a random one. A learned proposal replaces the rule by a policy trained on the searches already run, and enters this document in one position only: it chooses *what to evaluate*, and the thing evaluated is still the objective the problem defines. That is what keeps it refereeable. A proposal that fails is slower; an answer that fails is wrong.

The search is a Markov decision process [73]: the state is the current structure with its fitted parameters, an action is a move from the neighbourhood $\mathcal{N}(\cdot)$, and the reward is the improvement in the objective. An episode ends on a step budget or when no move improves the score — a property of the state, and the stopping rule the greedy baseline obeys, which is what makes the two comparable. Because the reward is a difference, the undiscounted return

telescopes to the improvement between the first and last state whatever path joined them, Equation 89: a discount would prefer improvement found early over the same improvement found late, and a truncation landing between a sacrifice and its payoff teaches the opposite of the truth, so a truncated episode is recorded as one. Appendix C states the algorithms that train such a policy, from dynamic programming to the actor–critic, and which of them a state space of this shape admits.

Four things about *this* decision process are application-specific, and each is a place a measurement goes wrong quietly. The policy is a softmax over the scores of the *available* actions, $\pi(a \mid s) \propto \exp f(s, a)$ for $a \in \mathcal{N}(s)$, so a neighbourhood of any size is admissible. A scorer with a single weight on the improvement a move buys is an inverse temperature, and greedy search is its zero-temperature limit; such an agent can learn only how much to accept a worsening move to reach a better optimum, which is the credit-assignment question this search makes sharp. That class cannot rank two moves differently from hill climbing, so the tree environment also scores a move by what it computes without a fit: the change in parsimony score, the fraction of sites compatible with the split the move breaks and with the split it makes, and the sizes of the two subtrees it exchanges, each standardized within the neighbourhood. A feature constant over a neighbourhood is cancelled by the softmax and excluded; the Robinson–Foulds distance from the current tree, the same for every NNI neighbour, is the standing example. Whether the wider class learns more is measured at a matched budget, never inferred from its width. It is trained by the score-function estimator of Equation 98 with a baseline, and the comparison against the greedy baseline is at a matched budget of objective evaluations per decision, never in wall-clock time.

The scorer, and what is identifiable in it. The score $f(s, a)$ is a function of the action’s features, and two parameterizations serve. A *linear* scorer, $f = w^\top x(s, a)$, is the reference, because its weights are what a recovery test can state a truth against: on the lattice environment the improvement a move buys decomposes exactly into two features, so the greedy searcher *is* a particular w inside the policy class, and “did the agent learn the landscape” becomes a statement about w rather than about a loss curve. A *multilayer-perceptron* scorer — two hidden layers over the same features, one output per action — is for a reward not linear in them. It is measured against the linear one at a matched budget and never assumed better: a wider class that is not needed costs sample efficiency and gives up the recovery statement. Both share the softmax’s gauge. Adding a constant to every score in a state changes nothing, so a feature constant over the neighbourhood carries no identifiable weight and is excluded, and neither scorer has a bias term. The critic of Section C.7 shows the rule is about the softmax and not about features: a value is not a softmax, a constant there *is* identifiable, and the critic carries one.

Exploration is declared, not learned. A softmax over improvements ends its episode at the first state no move improves, so an agent chooses which local optimum to enter and can never leave one. The remedy is to collect episodes under a *behaviour* policy that takes the trained policy’s best action except with probability ϵ , where it draws uniformly from the neighbourhood — the only way a *worsening* move is taken. It is not trainable and exposes no weights, so a measured escape rate is attributable to ϵ and not to a training run alongside it; ϵ follows a declared schedule over the iterations, as a temperature does in Section B. At $\epsilon = 0$ the agent still walks, deterministically: past a local optimum the best available action is the least-bad move, not the same object as a climb that would have stopped.

Two consequences for the accounting, both easy to get wrong in the direction that flatters the agent. An episode allowed to continue past a local optimum is scored on the *best* state it saw and not its last, or a better searcher looks worse the longer it runs. And the matched baseline is then greedy *with restarts*: a single greedy run stops after a few decisions and leaves the rest of the budget unspent, so a result stated against it overstates itself. One failure produces plausible

numbers rather than an error — wrapping an *untrained* policy is not hill climbing, since with equal scores the best action is whichever the neighbourhood lists first, and what is measured is exploration around a fixed arbitrary move.

What pins it. Equation 89 is an identity and is asserted exactly on every recorded episode. Equation 98 is an expectation, so its estimator is pinned against the gradient of the exact expected return, computable by enumerating every episode on a small environment, and the baseline’s unbiasedness by the same enumeration: the expected gradient with and without b agree. An environment small enough to enumerate is the oracle for the learning machinery, as a tree small enough to enumerate is for the search.

Compound moves. A move set fixes the granularity of a decision, and a search whose neighbourhood holds one interchange must find a sacrifice and its payoff as two decisions — the credit assignment a truncated episode destroys. An *option* is a policy over primitive moves with an initiation set and a termination rule, so a sequence that pays off only as a unit becomes one decision and is credited as one [74]; the termination rule can be learned beside the policy that calls it [9]. Nothing here builds one, and the budget rule says what it would have to survive: a compound move evaluates every primitive it composes, so an option reaching a better optimum by evaluating more candidates than hill climbing has not won.

22 Reading a Comparison

Every problem statement below ends with the same question: given more than one method that applies, which is worth a budget at the sizes that section supports. Three axes answer it, asked in the same order.

The precondition is a structural property of the instance deciding whether a method may run at all, and it is not a matter of degree: a non-negative coupling, a metric on labels, a tree, a terminated trellis, an output-symmetric channel. A method whose precondition fails is refused rather than run badly, which separates the comparisons here from a ranking — a cell of Section 16 is empty where a precondition failed as often as where nobody wrote the code.

The cost is counted in the unit that problem counts: objective evaluations for a discrete search, sweeps for a sampler, passes over the graph for a marginalization, configurations for an enumeration. It is never counted in seconds. A cost in operations and in the live set a recursion holds is a property of the algorithm and transfers between machines; a wall clock is a property of neither, and the language a kernel is expressed in — and the boundary between it and the language that calls it — measures a small fraction of a call rather than the term that decides a method. No choice here turns on it, and the measurements belong to the companion paper.

The reach of the referee is the size past which nothing exact scores the answer. It changes what may be *claimed* rather than what may be run: inside it a method is scored against a truth, outside it against a recovered structure, a bound, or another method — agreement and not truth (Section 34). Two methods that agree inside the reach and diverge outside it have not been separated, and a comparison crossing the reach without saying so has changed its subject mid-sentence.

Where the three leave a genuine choice, the answer is a measurement at an equal budget in the problem’s own unit over shared seeds, and a statement about that instance. Where they do not, the section says which of the three decided.

23 Algorithms: Continuous Test Functions

23.1 The algorithms that apply

Each function is an instance of the objective abstraction with the identity as its constraint map, so everything that runs on a likelihood runs on it: the quasi-Newton fit, the Hamiltonian sampler of Section A read as an energy, and annealing with Hamiltonian proposals on a schedule (Section B). They are the continuous counterpart of the frustrated lattices: an equal budget of objective evaluations is spent by restarts of the fit and by an annealed chain with a polishing fit, and which wins is a property of the surface rather than of the algorithms.

23.2 The initializers

The three model-free strategies, Algorithm 8: the function’s own start, that start tilted by a fixed alternating amount for a surface whose nominal start is stationary, and Gaussian restarts around it from a passed-in generator, every start fitted and every fit reported with its spread rather than the best alone (Figure 2). A multi-start that reports one answer for a surface with four basins is the failure the abstraction exists to expose.

What pins it. The fit against the analytic minimizer to a stated tolerance, in several dimensions for Equation 4; the automatic gradient against the hand-written closed form, exactly; every one of Himmelblau’s minima reachable from its own basin; and on Equation 5 a *rate*—the fraction of starts from which a converged fit is the global minimum—asserted low rather than asserted to succeed, which is the measurement that bounds what a single fit of any multimodal surface here may claim.

23.3 Choosing among these at the supported sizes

Four methods are assumed and none stated here: the quasi-Newton fit of Section 17, the Hamiltonian sampler of Section A read as an energy, annealing over that sampler on a schedule (Section B), and the restarts of Algorithm 8. The functions are carried to separate those methods rather than to be solved, so the question is asked in reverse: which failure does each surface expose.

One surface per failure. Conditioning alone, with a single basin: a second-order method arrives and every global method spends its budget for nothing, so a comparison run only on this surface reports that the cheapest local method is the best one. A quadratic bowl under a ripple, with about 10^d local minima: only a method that can cross a barrier arrives at all, restarts of a local fit convert into a rate rather than an outcome, and a fit that stopped at a stationary point satisfies every first-order test while being wrong. Four equal minima: no method is right, and the failure exposed is in the *reporting* — a multi-start that returns one answer is describing its start, and the spread is the result. Together they are the continuous counterpart of Section 11: an equal budget of objective evaluations, and which method wins is a property of the surface.

The one problem with no ceiling. The minimizers are closed forms, so the referee costs nothing at any dimension and there is no per-tier size limit. What grows with d is the number of basins a multi-start must find, so the quantity that scales is a *rate* — the fraction of starts reaching the global minimum — asserted low rather than asserted to succeed. That bounds what a single fit of any multimodal surface here may claim, and it is measured here because here it can be.

They are not likelihoods, and that is the point. The Hessian at a minimum is not an observed information, so nothing here supports an interval and the delta method of Section 17 has no meaning on these surfaces. That makes them a clean test of an optimizer and useless as a test of inference, which is the separation they exist for: on a likelihood, a fit that stopped early and a parameter the data identify weakly produce the same symptom, and only a surface whose answer is known can tell the two apart.

What the validation assumes. The published minimizers, and that the automatic derivative and the hand-written closed form are derivatives of the same function. It is the shortest such list in this document, and the reason these functions are carried at all.

24 Algorithms: The Gaussian Mixture

24.1 Expectation–maximization and the gradient fit

Expectation–maximization, Algorithm 15 with the chain removed, alternates Equation 8 with the weighted re-estimate of each component’s mean and variance and of the weights as the mean responsibility, and increases the likelihood at every iterate [47]; the same objective is also fitted by the quasi-Newton optimizer of Section 17 over an unconstrained vector, the weights through a softmax and the scales through a logarithm. The two share the model and nothing else, which makes their agreement evidence. Where the components overlap the fixed point is approached linearly, so a raw iterate stopped at a count sits above its basin’s optimum by an amount a gradient polish removes, and an equal-budget comparison charges the polish to every method.

24.2 The k -means++ start

The first initializer here that reads the data, Algorithm 6. The first centre is an observation drawn uniformly; each subsequent one is an observation y drawn with probability proportional to its squared distance from the nearest centre already chosen,

$$\Pr(c_{j+1} = y) = \frac{D(y)^2}{\sum_{y'} D(y')^2}, \quad D(y) = \min_{j' \leq j} |y - c_{j'}|, \quad (49)$$

and the expected k -means cost of the seeding, before any refinement, is within a factor of the optimum [6]:

$$\mathbb{E}[\phi(c)] \leq 8(\ln K + 2) \phi^*. \quad (50)$$

In one dimension ϕ^* is computable exactly—an optimal clustering partitions the sorted observations into contiguous runs, and a dynamic programme over the cut positions finds it—so the bound has something to be checked against, which it does not in higher dimension where the problem is NP-hard. Seeding is a statement about the cost and not about the likelihood: whether a seeded start reaches a better optimum of Equation 7 than a uniform one is measured over seeds and reported whichever way it falls.

The divergence the squared distance stands for. Equation 49 is stated with a squared distance because the components are isotropic Gaussians. The general rule replaces $D(y)^2$ by the Bregman divergence of the family’s log-partition [11], which for that family *is* the squared distance over twice the variance—and the sampling rule normalizes its scores, so a positive factor cancels and the two are one rule. **An additive constant is not a positive factor.** The family’s negative log-density is its Bregman divergence plus the log normalizer, and adding a constant to every score dilutes Equation 49 toward the uniform draw in proportion to the constant’s size against a typical divergence. The two spellings are therefore different algorithms wherever the normalizer is not small, which is a property of the family and has to be measured per family rather than assumed.

What pins it. The component M step against the hidden Markov model’s on the same responsibilities, at equality; expectation–maximization against the gradient fit at the same optimum, and its iterates monotone; the seeding cost against the exact dynamic programme, with the mean over seedings held to Equation 50; and recovery of the generating parameters up to the permutation of components, with the alignment part of the test. A component whose variance reaches the floor of Section 34 is a refusal, in either algorithm.

24.3 Choosing among these at the supported sizes

Three algorithms are assumed: the alternation of Algorithm 15 with the chain removed, the quasi-Newton fit of Section 17 over the unconstrained vector, and the seeding of Algorithm 6. The interesting size is neither N nor K but the dimension, which is one by choice.

Why one dimension is a decision about the referee. Enumerating the K^N labellings is out of reach at every declared size, so the expectation step is refereed by the chain recursion of Section 7 on the same responsibilities — the one problem here whose oracle is borrowed from a neighbouring class. The seeding’s referee is the exact clustering cost, a dynamic programme over cut positions in the sorted observations, which exists because the observations are totally ordered. In higher dimension the same quantity is NP-hard, so raising the dimension would not make the fit harder: it would delete the initializer’s only oracle and leave Equation 50 with nothing to check against. Growing N buys precision and changes no referee.

Which fit. The alternation is cheaper per iterate and monotone; the gradient fit is neither, and is carried because two routes sharing the model and nothing else are evidence where one route’s convergence is not. Where the components overlap the alternation approaches its fixed point linearly, so an iterate stopped on a count sits above its basin’s optimum by an amount a polish removes — and an equal-budget comparison charging the polish to one method alone compares stopping rules.

What the model does not have. There is no maximum. A component contracting onto a single observation sends the likelihood to infinity, so what any method here finds is a local optimum by construction, and the floor refusing such a fit is a modelling decision rather than a numerical guard: it decides which optima are admissible. The count-emission family of Section 6 is the same mixture without the pathology, both channels being discrete.

What the validation assumes. Independence across observations, which makes the responsibilities a per-observation normalization; and separation enough that ordering the components resolves the permutation. Neither transfers: a mixture whose components overlap past the ordering’s reach fails the recovery test for a property of the instance and not a defect of the fit, so the separation is declared with the instance rather than discovered from it.

25 Algorithms: The Count-Emission Mixture

25.1 Expectation–maximization and a seeded start

Expectation–maximization, Algorithm 15 with the chain removed, exactly as for the Gaussian mixture—what differs is that the M step it calls is itself an optimization. The negative binomial’s dispersion is a one-dimensional root find and the beta-binomial’s (a, b) an alternating one, each bracketed rather than stepped toward and each reporting whether it settled; a mixture discarding that report would surface an unsettled inner solve several iterations later as a non-monotone outer likelihood, so it is propagated and refused on.

The start is `Emission_Mixture++`: Algorithm 6 with the family’s own Bregman divergence in place of the squared distance of Equation 49. A component is placed *on* an observation— its depth becomes the component’s negative-binomial mean and its allele fraction, smoothed by a half-count, the beta-binomial’s rate—and the two shape parameters, which no single observation carries, start at declared values shared across components. The seeding is drawn over observation *indices* rather than over observation values, because a pair drawn from a flattened array of pairs is half of one.

What pins it. The responsibilities against the posterior enumerated over component labellings, at the size that admits one; the fitted weights and component parameters against the planted ones, up to the permutation of components; and the seeded start against the uniform one over shared seeds, reported whichever way it falls.

25.2 Choosing among these at the supported sizes

Everything Section 5 assumes is assumed here, with one difference that moves every cost in the section: the component’s re-estimate is a bracketed solve rather than a formula (Algorithm 15, with the family’s own maximization step). The seeding is Algorithm 6 with the family’s Bregman divergence in place of the squared distance, which is not a refinement but a necessity — a squared distance between a depth and an allele count compares two quantities that share no units. The divergence and the negative log-density are different rules, on the terms Section 5 states, and the beta-binomial is not an exponential family in its success count, so what stands in for the divergence there is the unit deviance it generalizes to: the log-density gap to the best rate the family admits at the observation, which is non-negative by construction.

The cost is the family’s, not the sample’s. Both shape parameters are found by bisection, at a few thousand score evaluations per component per iterate, so the budget is spent on iterations and components and barely on observations: the ten-component instance is a hard case rather than a larger one. It also decides where a failure appears. An inner solve that did not settle is a local event, and a mixture discarding its report would surface it as a non-monotone outer likelihood several iterates later, with nothing in the trace saying which component produced it — so the report is propagated and refused on rather than summarized.

Two forms, and no default between them. The trial count is the whole difference between the family’s independent form, in which the two channels factorize given the component, and its joint form, in which the second channel’s size is the first channel’s draw. They are different models over the same parameters: a fit under one scores nothing about the other, and the joint form additionally puts a whole region of the observation space at probability zero, which is a support statement and is handled as one rather than left to a logarithm of a negative argument.

What the enumeration oracle is doing. It runs over a prefix of ten observations, licensed by one property: the posterior factorizes across observations, so the marginal at each of ten is the same quantity the responsibilities of the whole dataset carry. The prefix is therefore the test itself rather than a sample of it, and is sharpest when the factorization is in doubt — the property every step in this section rests on, and the one an implementation can break without another check noticing.

What the validation assumes, beyond that. The declared form of the trial count, since the two forms are refereed by different enumerations; and separation on depth and allele fraction sufficient for an ordering to resolve the permutation, which a declared instance supplies by construction and which makes an ordering admissible where enumerating the alignments of ten

components would not be. Two flat directions bound what a fit may report: the dispersion toward the Poisson limit and the concentration toward the binomial one each have a value above which this much data cannot distinguish one value from infinity, so an estimate arriving there has found a bound rather than a maximum.

26 Algorithms: Hidden Markov Models

26.1 The forward–backward algorithm

The forward recursion is Equation 54 on a chain:

$$\alpha_t(i) = \left[\sum_j \alpha_{t-1}(j) \mathbf{A}_{ji} \right] \mathcal{E}_i(y_t), \quad p(\mathbf{D}) = \sum_i \alpha_T(i). \quad (51)$$

Pruning on a caterpillar tree carrying one observed leaf per internal node *is* this recursion [21, 41]: the three problem classes are one marginalization over three structures, not three unrelated models sharing an optimizer; Appendix E.2 derives both passes from it. In the terms of Equation 2, α_t is the message arriving at z_t from the left multiplied by the emission factor, the backward recursion β_t is the same pass from the right, and the posterior at a position is their normalized product,

$$p(z_t = i \mid \mathbf{D}) = \frac{\alpha_t(i) \beta_t(i)}{p(\mathbf{D})}, \quad \beta_t(i) = \sum_j \mathbf{A}_{ij} \mathcal{E}_j(y_{t+1}) \beta_{t+1}(j), \quad (52)$$

which is the belief b_t of Section 1. Max-product on the same chain is Viterbi,

$$\delta_t(i) = \left[\max_j \delta_{t-1}(j) \mathbf{A}_{ji} \right] \mathcal{E}_i(y_t), \quad (53)$$

with the path read back from the argmax at each step; on a chain with a unique maximum the max-marginals’ argmax at every position is that path [21].

Regime. Exact for every length, the chain being a tree. The one hazard is the underflow pruning meets, with the same remedy: the recursion runs in the logarithm.

What pins it. The sum over every one of the k^T paths, written out term by term with no recursion, referees the evidence, the posteriors and the Viterbi path at the lengths it allows; a gradient against central differences past them. The enumeration separates the two decoders, so no decoder can validate it; it is pinned against algebra instead, on quantities worked out by hand on a two-state, two-step chain.

26.2 Expectation–maximization

Baum–Welch, Algorithm 15, alternates a posterior over hidden states given the parameters with a re-estimate of the parameters given that posterior, and increases the likelihood at every iteration [47]. The initial distribution and the transitions re-estimate in closed form for every family; the emission re-estimate is the family’s own, and *whether it is a formula* separates the families. A weighted mean is; a dispersion parameter whose score involves the digamma function is not, and its M step is itself an optimization—the case that says whether an interface built around closed forms was built correctly.

What pins it. Monotonicity, asserted at every iterate; agreement of the fixed point with a gradient fit of the same likelihood; and recovery of simulated parameters up to the permutation of hidden states, with the alignment part of the test (Section 34). Where the family’s M step is itself an optimization, its refusals at the boundary of the parameter space are pinned too: a dispersion the data cannot identify is refused, not clamped.

26.3 Two decodings, and they are different answers

Viterbi returns the single path of highest joint probability; posterior decoding returns, at each position independently, the state of highest marginal probability. Where they disagree, the posterior-decoded sequence can be a path the model assigns zero probability to, nothing constraining consecutive choices to be compatible. They agree on almost every instance, which is why an implementation computing one and reporting the other survives a test suite with no case where they diverge. Equation 53 and the argmax of Equation 52 are those two decodings; the fixture on which they differ is constructed and kept, being the only case that tells an implementation of one from the other. Sampling the path posterior rather than decoding it is Algorithm 12, the block sweep the coupled model’s E step also uses.

26.4 Choosing among these at the supported sizes

Three algorithms are assumed and stated elsewhere: the two-pass marginalization on a chain (Appendix E.2), the alternation that fits it (Algorithm 15), and the block draw of a whole path from its conditional (Algorithm 12). Unlike every other section here, none has a size at which it stops being exact: the chain is a tree, so the recursions cost $O(Tk^2)$ and are exact at every length. The size question is therefore which referee is available, and the two ceilings move for different reasons.

Below the path enumeration — eleven positions at three states, seventeen at two — the term-by-term sum scores the evidence, both decodings and every posterior, and it is the only thing that can, since it is what separates a decoder computing the most probable path from one computing the per-position modes. That is why the fixture on which the two decodings differ lives inside this ceiling. Past it only recovery remains, and recovery needs *length* for a reason unrelated to cost: a transition matrix is identified by the number of transitions observed, so the fixture’s length is set by the variance of the estimate and its width by the state count, while the recursion’s cost stays linear in the length.

Which fit, and why both are carried. The alternation costs one pass per iterate and increases the likelihood at every one, so it is what to run wherever the emission family’s re-estimate is a formula. Where the re-estimate is itself an optimization — a dispersion whose score involves a digamma — the guarantee is inherited only if the inner solve converged, and an unsettled one surfaces as a non-monotone outer likelihood several iterates later. There the gradient fit over the unconstrained vector of Section 17 is the direct route. Where both apply they must reach the same optimum, and the evidence is the agreement of two routes sharing only the model, never the convergence of either.

Which decoding, and the choice is the caller’s. The two answers are not an accuracy ordering. The most probable path minimizes the probability of getting the whole path wrong; the per-position modes minimize the expected number of wrong positions and may return a path the model gives probability zero. Neither dominates, and the choice states what the caller is scored on. Sampling the path posterior is the third answer and the one the coupled model’s expectation step needs, a decoded path being a point estimate.

What the validation assumes. Two properties, both stated in the section and neither inherited from a neighbouring model. The declared support decides what may be asserted at all: over a countable alphabet the evidence is a probability and is bounded by one, over the reals it is a density and is not, so a check written against the first is false under the second, and it is the family’s own declaration and not the recursion that says which of the two holds. And the parameters are identified only up to the permutation of hidden states, so recovery carries its

alignment as part of the test rather than as a post-processing step — an alignment chosen after seeing the fit would referee nothing.

27 Algorithms: Phylogenetic Inference

27.1 The simulation algorithm

For a given rate matrix $\mathbf{Q}$ and branch lengths $\boldsymbol{\tau}$, the generative direction of Equation 13 is a walk in *pre-order* — a traversal visiting each node before any of its children, so that a node’s parent always carries a state by the time the node is reached. The root is drawn from $\boldsymbol{\pi}$ and every other node from the row of $\mathbf{P}(\tau_v)$ its parent’s state indexes. One site is Algorithm 1; L sites are L independent runs, and the empirical substitution frequencies along any branch converge to Equation 12 (Section 8.3).

Algorithm 1 Forward simulation of one site under $(\mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$

- 1: draw $z_\rho \sim \boldsymbol{\pi}$
 - 2: **for** each non-root node v in pre-order, with parent u **do**
 - 3: draw $z_v \sim \mathbf{P}_{z_u, \cdot}(\tau_v)$
 - 4: **end for**
 - 5: **return** the leaf states $(z_\ell)_{\ell \in \text{leaves}(\mathcal{T})}$
-

27.2 The pruning algorithm

Felsenstein’s pruning evaluates that marginal in $O(nLk^2)$ rather than $O(Lk^{n-2})$, by a post-order recursion carrying a partial likelihood $L_u(i)$ —the probability of the data below node u given that u is in state i :

$$L_u(i) = \prod_{v \in \text{children}(u)} \sum_{j \in \mathcal{A}} \mathbf{P}_{ij}(\tau_v) L_v(j), \quad (54)$$

with $L_u(i) = \mathbf{1}[u \text{ observed in state } i]$ at a leaf, and the site likelihood read off at the root,

$$p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \sum_{i \in \mathcal{A}} \pi_i L_\rho(i) \quad (55)$$

[25, 21]; Appendix E.1 derives the recursion from the marginalization. Products of many small numbers underflow, so each node’s partials are rescaled and the logarithm of the scaling accumulated separately.

Site patterns. Equation 13 reads one column per term, so two identical columns contribute identical terms and the sum over L columns is a weighted sum over the $P \leq \min(L, k^n)$ distinct ones:

$$\ell(\mathcal{T}, \boldsymbol{\tau}) = \sum_{s=1}^L \log p(\mathbf{D}_s \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}) = \sum_{p=1}^P w_p \log p(\mathbf{D}_p \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q}), \quad \sum_p w_p = L, \quad (56)$$

with w_p the number of columns equal to pattern p . The identity is exact: the arithmetic is reassociated, not approximated, and only the floating-point summation order distinguishes the two sides. It is the standard first step of a maximum-likelihood implementation [25], and it saturates, since $P \leq k^n$ however long the alignment—at $k = 4$ and four taxa $P \leq 256$ whatever L is. Compression is a property of the alignment rather than of an evaluator, so every backend consumes patterns and weights.

Equation 54 is Equation 2 run leaf to root with the root as the last variable: $L_u(i)$ is the product over u 's children of the factor-to-variable messages $m_{\psi_v \rightarrow u}(i)$, each of which sums the branch factor against the message arriving from below, and Equation 55 is the final factor-to-variable message from the prior. Replacing the sum by a maximum gives the most probable joint assignment of ancestral states; in the limit of short branches its cost counts substitutions, which is Fitch parsimony [25], the second criterion the search is scored against.

Both parsimony scores are the same post-order pass with no branch lengths. Fitch's recursion [26] carries at each node v the set S_v of states reachable at the minimum cost so far, and a count F of changes:

$$S_v = \begin{cases} S_a \cap S_b & S_a \cap S_b \neq \emptyset, \\ S_a \cup S_b & \text{and } F \leftarrow F + 1 \text{ otherwise,} \end{cases} \quad (57)$$

for the children a, b of v , folded pairwise at a trifurcation, a leaf's set being its observed state. Sankoff's recursion [64] replaces the set by a cost vector under a step matrix $W \in \mathbb{R}_{\geq 0}^{k \times k}$, W_{ij} the cost of a change from a parent in state i to a child in state j :

$$S_v(i) = \sum_{c \in \text{ch}(v)} \min_j [W_{ij} + S_c(j)], \quad S_{\text{leaf}}(i) = \begin{cases} 0 & i = x_{\text{leaf}}, \\ \infty & \text{otherwise,} \end{cases} \quad (58)$$

and a site's score is $\min_i S_{\text{root}}(i)$. Equation 54 in the min-plus semiring, returning Fitch's count exactly under the unit matrix $W = \mathbf{J} - \mathbf{I}$. The score is a rooted tree's: an asymmetric W scores each rooting of one unrooted topology differently, so a search over unrooted topologies is well defined only when W is a metric, where every rooting agrees.

The gradient in the branch lengths. Fitting τ needs $\partial \ell / \partial t_v$ for all $2n - 3$ branches. Differentiating Equation 54 branch by branch costs one recursion each; a single root-to-tip pass returns all of them together, the transpose of the post-order pass rather than a second algorithm [25]. Write $A_u^s(i) = \partial \ell / \partial L_u^s(i)$ for the adjoint of node u 's partial at site s , $m_v^s(i) = \sum_j \mathbf{P}_{ij}(t_v) L_v^s(j)$ for the message v sends its parent, and $\dot{\mathbf{P}}(t) = d\mathbf{P}/dt$, which is closed form under Equation 12 and $\mathbf{Q}\mathbf{P}(t)$ in general. Equation 55 starts the pass,

$$A_\rho^s(i) = \frac{\pi_i}{\sum_{j \in \mathcal{A}} \pi_j L_\rho^s(j)}, \quad (59)$$

and for a non-root node v with parent u , abbreviating $B_v^s(i) = A_u^s(i) \prod_{w \in \text{children}(u) \setminus \{v\}} m_w^s(i)$,

$$\frac{\partial \ell}{\partial t_v} = \sum_{i, j \in \mathcal{A}} \dot{\mathbf{P}}_{ij}(t_v) \sum_s B_v^s(i) L_v^s(j), \quad A_v^s(j) = \sum_{i \in \mathcal{A}} B_v^s(i) \mathbf{P}_{ij}(t_v), \quad (60)$$

which is Algorithm 2. Its cost is $O(nLk^2)$, the forward pass's, and independent of the number of branches differentiated.

Two properties of that pass are places an implementation goes wrong silently. First, the rescaling contributes nothing: ℓ is algebraically independent of the per-node scales, the divided partial and the accumulated logarithm moving against each other, so the derivative may be taken with the scales frozen at the forward pass's values and each adjoint divided by its own node's scale. Second, the sibling product $\prod_{w \neq v} m_w^s$ must be formed by a scan over the siblings and not as L_u^s/m_v^s : a message vanishes at any site whose observation forbids that state, where the quotient is $0/0$ and the derivative is finite.

Regime. Exact on every input, a tree having no cycle. The one failure mode is numerical—underflow of a product of n factors each below one—and the rescaling removes it rather than tolerating it.

Algorithm 2 Two-pass gradient of the pruning recursion in the branch lengths

- 1: run Equation 54 in post-order, keeping each node's partial L_v^s and the message m_v^s it sent its parent
 - 2: $A_\rho^s \leftarrow \pi / \sum_j \pi_j L_\rho^s(j)$ ▷ Equation 59
 - 3: **for** each non-root node v in pre-order, with parent u **do**
 - 4: $B_v^s(i) \leftarrow A_u^s(i) \prod_{w \in \text{children}(u) \setminus \{v\}} m_w^s(i)$
 - 5: $\partial \ell / \partial t_v \leftarrow \sum_{i,j} \mathbf{P}_{ij}(t_v) \sum_s B_v^s(i) L_v^s(j)$
 - 6: $A_v^s(j) \leftarrow \sum_i B_v^s(i) \mathbf{P}_{ij}(t_v)$
 - 7: **end for**
 - 8: **return** $(\partial \ell / \partial t_v)_{v \neq \rho}$
-

What pins it. Direct marginalization, Equation 13, at the sizes it allows, to a relative tolerance (Section 34.5). Two invariants past those sizes: the pulley principle, that a reversible model's likelihood does not depend on where the root is placed along a branch, and a gradient against central differences relative to its own norm—which refereed Algorithm 2 against the same likelihood differentiated by a tape, the two agreeing to a tighter tolerance than either agrees with the difference quotient. And the recovery test of Section 34: on simulated data the fitted branch lengths cover the truth at their nominal rate, and the search recovers the generating topology at a stated site count.

27.3 The discrete search

The number of unrooted binary topologies on n taxa is $(2n - 5)!!$ [62], so exhaustive enumeration is an oracle at small n and nothing more. Two neighbourhoods are standard. *Nearest-neighbour interchange* (NNI) swaps the subtrees adjacent to one internal edge, giving $|\mathcal{N}_{\text{NNI}}(\mathcal{T})| = 2(n - 3)$. *Subtree prune and regraft* (SPR) detaches a subtree and reattaches it at another edge, giving $2(n - 3)(2n - 7)$ and containing NNI. Neither is complete in one step; both are complete in the transitive closure, which for NNI is the connectivity of the associated graph [25] and is inherited by SPR because it dominates NNI.

Figure 21 runs both over the same start.

Hill climbing over either neighbourhood fits the continuous parameters of every candidate and moves to the best; its budget is counted in candidates scored, never in seconds, so a run reproduces from its seed. The same climb with Equation 57 or 58 in place of the fit is *large parsimony* [25]: nothing continuous is fitted, a candidate costs one pass, and the budget and the oracle are the same. A candidate sharing all but a few branches with its parent may start its fit from the parent's lengths, a branch identified by the leaf bipartition below it rather than by a node's name; and a neighbourhood may be *ranked* by one evaluation at those lengths before the few best are fitted, the accepted move always in full, so every reported score is an optimum. Whether the ranking finds the fitted best is measured per move set, not assumed.

Three further economies apply to the same climb, each separable from the others. The climb may begin at a most-parsimonious topology, since a parsimony pass costs no fit. The regraft may be bounded to edges within distance r of the vacated edge, giving $|\mathcal{N}_{\text{SPR}}^r(\mathcal{T})| = O(nr)$ in place of $O(n^2)$; $r = 1$ is exactly the NNI neighbourhood, so a bounded SPR inherits NNI's transitive completeness at every r , and r at least the tree's diameter is SPR itself. And a candidate may be fitted over only the branches whose bipartition the move changed — the path between the vacated and the graft edge, one edge under NNI — holding the rest at the parent's fitted lengths. That last optimizes over a subspace, so its value lower-bounds the candidate's maximum rather than equalling it: it decides which candidate to accept, and the accepted one is refitted over every branch, as a ranked neighbourhood is.

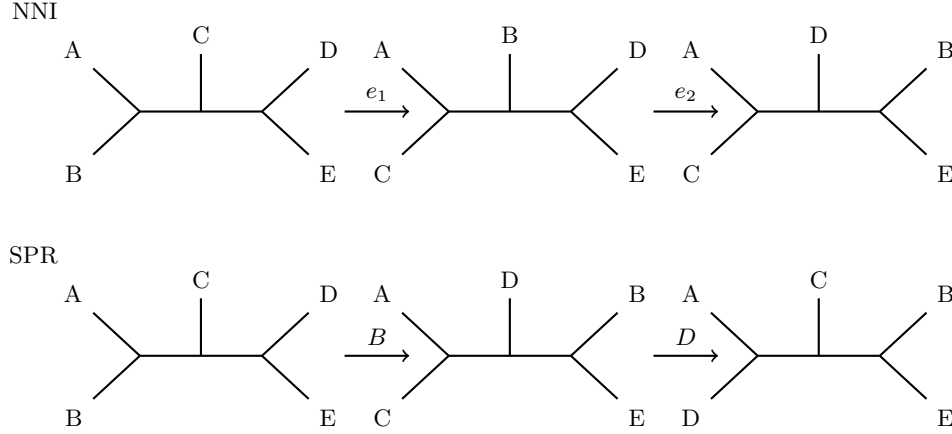


Figure 21: Three successive states of one five-taxon topology under each move set, from the same start. Above: two nearest-neighbour interchanges, each swapping the subtrees adjacent to the named internal edge, so exactly one bipartition changes per step. Below: two prune-and-regraft moves, each detaching the named taxon and reattaching it at another edge. The first regraft reaches in one move the topology the interchanges reach in two — it is the rightmost panel above — which is what the containment $\mathcal{N}_{\text{NNI}} \subset \mathcal{N}_{\text{SPR}}$ buys, at a neighbourhood of $2(n-3)(2n-7) = 12$ candidates against $2(n-3) = 4$.

What pins it. Enumeration of the $(2n-5)!!$ topologies referees the search below eight taxa: the climb must reach the enumerated maximum, and the neighbourhood counts above are asserted exactly over every topology. Each economy is held to that same enumeration alone and in combination, or the rate at which it loses the optimum is measured and stated. Two also have an exact reduction to something already refereed, which a count alone would not catch: the bounded regraft at $r = 1$ must equal the NNI neighbourhood, and at r past the diameter the unbounded one, candidate for candidate. A warm start is pinned by reaching the cold fit’s optimum; a cached partial likelihood by equalling the recomputed one bitwise, being the same arithmetic in the same order. Large parsimony is held to the same enumeration from every start, and on the Felsenstein-zone fixture its enumerated optimum is asserted to be the wrong tree while the likelihood optimum is the right one, since there the criterion’s failure is the theorem of Section 34 and not a defect of the search.

27.4 Spectral learning, and what it inverts

Three starts share one idea and are stated separately below, because what each assumes and what each guarantees are different. Spectral learning of a hidden Markov model [38] is a method of moments: the observable moments of short windows are inverted to the parameters, with a consistency guarantee and no likelihood surface to climb. Mossel and Roch [52] carry the argument to a phylogeny — the same inversion of pairwise moments learns a nonsingular tree — where it becomes a distance per pair of taxa (Section 27.5), a tree from those distances (Section 27.6), and, on two states, an exact conjugation inverting the whole pattern spectrum at once (Section 27.7). All three are a *start* and not an estimate: the fit and the search begin from them, and every reported number is the optimum they reach.

27.5 Distances, and the moment each inverts

The pair co-occurrence matrix F of two taxa, F_{ij} the fraction of sites at which the first is in state i and the second in j , is the moment; under Equation 12 its off-diagonal mass is $\frac{k-1}{k}(1 - e^{-kt/(k-1)})$ for t the path length between the two, which inverts in closed form on the fraction p of differing

sites,

$$\hat{d} = -\frac{k-1}{k} \log\left(1 - \frac{k}{k-1} p\right), \quad \text{Var}(\hat{d}) = \frac{p(1-p)}{L(1 - \frac{k}{k-1} p)^2}, \quad (61)$$

the variance by the delta method over the multinomial site counts. Under a general Markov process on the branches the log-det distance [72]

$$\hat{d} = -\frac{1}{k} \left[\log \det F - \frac{1}{2} (\log \det \Pi_x + \log \det \Pi_y) \right], \quad (62)$$

Π_x, Π_y the diagonal matrices of F 's row and column sums, is additive along the tree whatever the process; it equals t when $\mathbf{Q}$ has trace $-k$, which Equation 11 gives the Jukes–Cantor model, and $-\text{tr}(\mathbf{Q})t/k$ under a general reversible $\mathbf{Q}$, a constant that leaves the tree unchanged.

27.6 Neighbor joining, and the four-point condition

Neighbor joining [63] is the tree from the distances, Algorithm 3: $n-3$ agglomerations of the pair minimizing $(n-2)d_{ij} - r_i - r_j$, r the row sums, each placing the new node by the three-point formula and reducing the matrix by one. On an *additive* matrix — one whose entries are the path lengths of some tree — it returns that tree with every branch length exact.

Additivity is a property of four taxa at a time. A quartet has three resolutions and a tree realizes exactly one: its two interior nodes are joined by a path of length $m \geq 0$ that the path between differently paired taxa crosses and the path between paired taxa misses. Summing the path lengths in each of the three pairings therefore gives

$$d_{ij} + d_{kl} \leq d_{ik} + d_{jl} = d_{il} + d_{jk} = d_{ij} + d_{kl} + 2m, \quad (63)$$

for the resolution $ij \mid kl$: each of the two larger sums counts the interior path twice and they are therefore equal, and each exceeds the smallest by exactly $2m$ (Figure 22). Equation 63 holding on every quartet is the *four-point condition*, and it is necessary and sufficient for a dissimilarity to be the path metric of a tree [25]. Two consequences are used below. The *smallest* of the three sums names the quartet's resolution, which is what makes a topology an arg min of a linear function of the distances and is where Section 27.8 starts. And the gap between the smallest and the other two is twice the interior branch length, so a quartet whose interior edge is short is resolved by an estimated matrix only if the matrix's error is smaller than that edge.

That last is Atteson's radius in one quartet. On an estimated matrix Atteson [7] gives the guarantee in general: the topology is the true one whenever every entry's error is below half the shortest branch, and Equation 61's variance is what lets a run state whether it sits inside that radius.

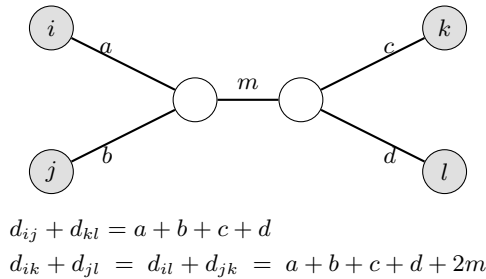


Figure 22: The four-point condition, Equation 63, on the quartet resolved as $ij \mid kl$. Each of the two larger pairing sums crosses the interior edge twice, so the two are equal and each exceeds the smallest by $2m$; the smallest sum names the resolution. A quartet with $m = 0$ is unresolved and all three sums agree, which is why an estimated matrix recovers a quartet only when its error is below the interior length — Atteson's radius, in one quartet.

Algorithm 3 Neighbor joining on a distance matrix D over n taxa

```
1: while more than three nodes remain do
2:    $r_i \leftarrow \sum_j D_{ij}$  for every remaining node  $i$ 
3:    $(i, j) \leftarrow \arg \min_{i \neq j} (n-2) D_{ij} - r_i - r_j$ 
4:   join  $i$  and  $j$  at a new node  $u$  with  $D_{iu} = \frac{1}{2} D_{ij} + \frac{r_i - r_j}{2(n-2)}$ ,  $D_{ju} = D_{ij} - D_{iu}$ 
5:    $D_{ku} \leftarrow \frac{1}{2}(D_{ik} + D_{jk} - D_{ij})$  for every other  $k$ ; remove  $i$  and  $j$ ;  $n \leftarrow n - 1$ 
6: end while
7: join the last three at the root by the three-point formula
```

27.7 The Hadamard spectrum

For the symmetric two-state model the inversion is exact rather than pairwise. Hendy and Penny [36] index the 2^{n-1} site patterns by the set of taxa differing from a reference taxon, and the same subsets index the splits of the leaf set; with s the pattern frequencies and q the branch length on each split, zero where the tree has no such branch, the two are conjugate under the Sylvester–Hadamard matrix H of that order,

$$s = H^{-1} \exp(Hq), \quad q = H^{-1} \log(Hs), \quad (64)$$

so the exact spectrum returns the tree’s own lengths on its splits and zero elsewhere. The *closest tree*, Algorithm 7, takes the $n - 3$ largest mutually compatible weights of an estimated q .

What the two-state transform applies to is a *lumping* of the alphabet, and which lumpings exist is a property of the substitution model rather than of the transform. A four-state process is two-state under a partition of $\mathcal{A}$ into two blocks whenever the total rate from a state into the opposite block is the same for both states of its own block; the lumped process is then Markov at that rate [25]. Under Jukes–Cantor every rate is equal, so all three pairings lump and each gives the same two-state chain, at $\frac{2}{3}$ of every branch length — the reduction the implementation uses. Under a model separating transitions from transversions the condition selects one pairing: the purine–pyrimidine partition $\{A, G\} \mid \{C, T\}$, where a state leaves its block at the transversion rate whichever state it is, so the lumped chain is the transversion process and the transition rate drops out of it. The other two pairings mix the two rates and are not Markov after lumping. The two-state solution is therefore a known method and not an approximation: it estimates the transversion process exactly and says nothing about the rate it lumps away, which is one reason it is a start. Refining that answer back to four states is a second fit rather than part of the transform; the four-state conjugation over the Klein four-group would do it in one step and is not implemented.

What pins it. Equation 61 against Equation 12 at the exact pair frequencies; Equation 62 against t under Jukes–Cantor and against $-\text{tr}(\mathbf{Q})t/k$ under a general reversible $\mathbf{Q}$; the stated variance by the coverage of its interval over seeds. Neighbor joining against the path lengths of every fixture and of random trees at 20 and 50 taxa, every branch to 10^{-12} ; on simulated alignments, recovery on every replicate inside Atteson’s radius, with no exception. The Hadamard conjugation against the pruning likelihood of Section 8 evaluated on every pattern, which shares nothing with the transform. And the starts against the objective’s own, at equal evaluations, through the budgeted comparison of Section 34.

27.8 The tropical relaxation

The four-point condition of Section 27.4 is more than a test for additivity: it is the defining relation of a variety. Speyer and Sturmfels [71] show that the tropical Grassmannian $\text{Gr}(2, n)$, the tropicalization of the Plücker relations for two-planes in n -space, is the space of trees on

n leaves — its points the vectors $d \in \mathbb{R}^{\binom{n}{2}}$ for which every quartet’s largest pairing sum is attained twice. Two consequences make it a relaxation of topology search. The coordinates are continuous, so a tree is a point of a Euclidean space; and a quartet’s resolution is the pairing attaining the *smallest* of its three sums, so the discrete structure is an arg min of a linear function of those coordinates.

Relaxing the arg min relaxes the topology. Write $s_Q(d) \in \mathbb{R}^3$ for the pairing sums of quartet Q and $\ell_Q \in \mathbb{R}^3$ for a score per resolution — the maximized log-likelihood of the four-taxon tree, say. The softmin at temperature τ gives

$$F_\tau(d) = \sum_Q \langle \text{softmax}(-s_Q(d)/\tau), \ell_Q \rangle, \quad (65)$$

differentiable in d throughout and linear in the softmin weights, so at a one-hot weight vector it equals the discrete objective $D(\mathcal{T}) = \sum_Q \ell_Q[r_{\mathcal{T}}(Q)]$ exactly: an extension of what it relaxes rather than a second model, the demand Section 34 also makes of the Gumbel-softmax relaxation of a discrete state space.

Three properties bound what may be claimed for it, and each is checkable. *The scale of d is a gauge*: $F_\tau(cd) = F_{\tau/c}(d)$ for $c > 0$, so an unconstrained ascent maximizes F by inflating the metric — sharpening the softmin — without moving a single resolution, and d must be normalized for τ to be the only sharpness. *The corner agreement is a derived tolerance, not a constant*: with g_Q the gap between a corner’s smallest and second-smallest pairing sum and R_Q the range of ℓ_Q , the softmin leaves at most $2 \exp(-g_Q/\tau)$ weight off the corner’s own resolution, so

$$|F_\tau(d) - D(\mathcal{T})| \leq \sum_Q 2 e^{-g_Q/\tau} R_Q, \quad (66)$$

which inverts for the τ any requested tolerance needs and shrinks with a fixture’s shortest internal branch. *The relaxation is outer*: F_τ is defined on all of $\mathbb{R}^{\binom{n}{2}}$ and not on $\text{Gr}(2, n)$, a general dissimilarity resolves every quartet, and the resolutions need not come from any tree. Ascent may leave the variety, and the topology is read off the point it stops at by neighbor joining, exact on the variety.

D is a quartet decomposition of the criterion and not the tree’s log-likelihood — a different surface, so substituting one for the other is licensed by measuring that the two agree at the arg max, never by a correlation.

What pins it. Equation 65 against D at the metric of every enumerated topology, relatively — the residual once Equation 66 is under an absolute bound is float64 rounding of a sum over $\binom{n}{4}$ terms, which does not transfer between problem sizes. The arg min reading of a quartet against the tree’s own leaf bipartitions, which share no arithmetic. Equation 64 on the exact two-state spectrum, whose edge weights sum over separating splits to a metric that must satisfy the four-point condition and resolve every quartet as the tree does — a route to a point of $\text{Gr}(2, n)$ that shares no algebra with a walk over the tree. The arg max of D against the arg max of the fitted likelihood, enumerated. And ascent against the enumerated maximum of D , or, where a fixture’s top two topologies lie within the convergence of the fits that scored them, against the generating topology instead, since a ranking that turns on less than the optimizer’s convergence is not a property of the model.

27.9 Choosing among these at the supported sizes

Four algorithms are assumed rather than derived here: the post-order marginalization (Appendix E.1), the two-pass adjoint that differentiates it (Algorithm 2), the agglomeration building a tree from a distance matrix (Algorithm 3), and the climb over a neighbourhood (Algorithm 14).

Taxa	What referees	What to run, and why not the alternative
5–6	Enumeration, 15 and 105 topologies	The climb is run to be <i>scored</i> , not to find. Ranking a neighbourhood by a surrogate buys nothing: the neighbourhood is smaller than the number of fits the ranking saves.
7–8	Enumeration, 945 and 10,395	Enumeration is still the referee but now costs one continuous fit per candidate, so the climb is what is run and the enumeration what scores it. Parsimony reaches the same neighbourhood for one pass per candidate instead of one optimization.
9–12	Nothing exhaustive; the two-state spectrum, 2^{n-1} entries	The exact conjugation is the only start with an argument behind it rather than a measurement, and it is the last size at which one exists. A likelihood optimum here is reported and not certified.
13–200	Additivity, and the radius an estimated matrix must sit inside	Distances and the agglomeration are the start; the climb reports the optimum it reached. Whether that is the optimum is not established, and the one surviving statement is the radius — which the variance of Equation 61 says whether a run is inside.
to 1000	Recovery of the generating tree, or nothing	Only the evaluation is affordable, at $O(nLk^2)$ a pass. The search is unrefereed, and closing it needs a bound on the optimum rather than a faster climb — the cell Section 16 marks.

Table 10: What referees a phylogenetic claim at each taxon count, and what that leaves worth running. The taxon count and not the alignment length is the axis, for the reason the paragraph below gives.

Each is standard and stated in the appendix. Citing them does not settle which to spend a budget on at a given taxon count; Table 10 answers that.

The alignment length is a second axis and behaves oppositely. Pattern compression saturates at $P \leq k^n$ (Equation 56), so past a few thousand sites an evaluation stops growing in cost while the estimate keeps sharpening: length is the cheapest resolution on offer, and the reason the declared range reaches eleven thousand sites at a taxon count whose search is unrefereed. Below a few hundred sites the same variance decides whether a distance matrix sits inside the recovery radius at all.

What each method needs of the model. The recursion applies to any tree and any rate matrix, reversible or not. The closed form Equation 12 is Jukes–Cantor’s alone; the log-det distance, Equation 62, is additive under any Markov process on the branches and is the one distance surviving a model the fit does not assume. The conjugation of Equation 64 is the symmetric two-state model’s, reaching four states only through the exact pairing reduction. The tropical relaxation applies wherever a quartet carries a score, but is *outer*: ascent may leave the space of trees, so it is a way of moving and never of reporting, and the topology is read back by an algorithm exact on the space.

What the validation assumes. Three application-specific properties, and every referee in Section 8.3 rests on one. Sites are independent and identically distributed given the tree, which makes direct marginalization a per-column sum and pattern compression an identity. The model is time-reversible, which is the whole content of the pulley principle: the invariant is a statement about $\mathbf{Q} = \mathbf{S} \text{diag}(\boldsymbol{\pi})$ and is silent about a model that is not reversible. And recovery assumes the data came from the model, so it calibrates the pipeline on its own samples and transfers to nothing else — the referee of last resort.

28 Algorithms: Large Parsimony

28.1 Hill climbing on the score

The climb of Section 8, Algorithm 14, with the fit replaced by one post-order pass: a random start by stepwise insertion, the best neighbour under nearest-neighbour interchange or subtree prune-and-regraft taken while one improves, each topology scored at most once, and the budget counted in candidates scored. A candidate costs one pass rather than one optimization, so the same budget reaches a neighbourhood an order of magnitude larger, and the trade is that the criterion it climbs is the wrong one in a region that can be named.

Regime. The step matrix must be a metric, since an unrooted topology has one score only when every rooting scores the same; an asymmetric matrix is refused rather than scored at an arbitrary root. And the criterion is *statistically inconsistent* in the Felsenstein zone (Figure 10): with two long branches that are not a cherry, convergent change on them is cheaper to explain by grouping them than by the true tree, so the error rate goes to one as the alignment grows [25]. Moving the same branches to be a cherry gives the Farris zone, where the criterion is consistent and fast; the pair is what makes a measurement interpretable, because an implementation that is merely broken fails both.

What pins it. The score against a brute force over every labelling of the internal nodes, which shares no traversal with either recursion, at exact equality; Sankoff under the unit matrix against Fitch on every topology of a fixture; and the search against the enumeration of Equation 14's domain from every start where it reaches, with the Felsenstein-zone fixture asserted to return the wrong tree while the likelihood optimum on the same alignment is the right one.

28.2 Choosing among these at the supported sizes

Nothing is assumed here that Section 8 does not already assume: the same neighbourhoods, the same budget accounting and the same climb (Algorithm 14), with one post-order pass in place of one continuous optimization. The whole of the trade is that ratio, and the whole of the objection is that the criterion it climbs is not always the right one.

The trade, stated once. At an equal budget of candidates scored, a criterion costing one pass reaches a neighbourhood larger than one costing a fit by the ratio of their costs, which at the taxon counts stated above is an order of magnitude. Where the criterion is right that ratio is free resolution, and the sensible arrangement is to *search* on the score and *report* the likelihood — which is what Equation 42 makes rigorous, the score entering the likelihood problem as an upper bound a neighbourhood can be ranked by rather than as a rival answer. Where the criterion is wrong the ratio buys nothing at all: the Felsenstein zone is not a hard instance but a different objective, and no budget corrects an objective. That is the reason the criterion is carried *beside* the likelihood and never instead of it.

What it needs of the model, and what it does not. It needs a step matrix that is a metric, or an unrooted topology has no single score and the search is not well posed — an asymmetric matrix is refused rather than scored at an arbitrary root. It needs nothing else: no branch lengths, no rate matrix, no stationary distribution, which is what makes it applicable where a likelihood is not yet parameterized and why it is the natural start for a climb that will later fit. One consequence is easy to miss: the score is an integer, so ties are exact and frequent, where a likelihood's ties are numerical and mean something different. A search that breaks integer ties by the order candidates were generated is reading its input rather than the alignment, and reports a topology that a re-ordering would change.

What the validation assumes. The brute force over every labelling of the internal nodes referees the score by sharing no traversal with either recursion, and it assumes only that both are scoring the same rooted tree. The load-bearing assumption is elsewhere: that the Felsenstein zone’s inconsistency is a property of the criterion and not of an implementation — the strongest assumption in this section, and the reason the zone is carried as a *pair* with its consistent counterpart: only the pair separates a defect from a theorem the section asserts the code reproduces.

29 Algorithms: Potts Models in an External Field

29.1 The belief propagation algorithm

Exact marginals come from enumeration at small sizes and from belief propagation on a tree, where it is exact; on a loopy graph it is an approximation whose accuracy is a property of the graph rather than of the implementation [41, 28]. It is Equation 2 with every factor pairwise or unary: composing the variable-to-factor message with the unary factor gives the node-to-node form, in which the message from i to a neighbour j is

$$m_{i \rightarrow j}(\sigma_j) \propto \sum_{\sigma_i} \exp(J_{ij} \delta(\sigma_i, \sigma_j) + h_i(\sigma_i)) \prod_{l \in \partial i \setminus j} m_{l \rightarrow i}(\sigma_i), \quad (67)$$

iterated to a fixed point, with beliefs $b_i(\sigma_i) \propto e^{h_i(\sigma_i)} \prod_{l \in \partial i} m_{l \rightarrow i}(\sigma_i)$ at a node and likewise on an edge. The messages are carried in the logarithm and normalized every sweep: a coupling of a few units on a small lattice already puts a product of exponentials past what a linear recursion holds. The Bethe free energy of the fixed point,

$$F_{\text{Bethe}} = \sum_{\langle i, j \rangle} \sum_{\sigma_i, \sigma_j} b_{ij} (E_{ij} + \log b_{ij}) + \sum_i \sum_{\sigma_i} b_i E_i - \sum_i (d_i - 1) \sum_{\sigma_i} b_i \log b_i, \quad (68)$$

with $E_{ij} = -J_{ij} \delta(\sigma_i, \sigma_j)$, $E_i = -h_i(\sigma_i)$ and d_i the degree of i , equals $-\log Z$ exactly on a tree and approximates it on a loopy graph [83, 50]; Appendix E.6 states why, and Appendix E.3 places the tree case. A fixed point the iteration never reached estimates nothing, so non-convergence is a refusal rather than a number.

What a region sees that an edge cannot. Equation 68 is exact when the graph factorizes over its edges, and a lattice’s shortest cycle has four of them: no edge term can hold the correlation that runs around a unit cell. Taking the cell itself as the unit of the approximation is the Kikuchi free energy, Appendix E.4, whose stationary points parent-to-child message passing finds (Appendix E.7). It reduces to Equation 68 when the regions are the edges and the sites, so it is one expression and not a second approximation, and on the 3×3 lattice it is three to four orders of magnitude nearer the enumerated $\log Z$ for $3.4\times$ the wall clock. Two statements bound what that buys. The advantage *decays with coupling* — $23,378\times$ at $J = 0.3$ against $130\times$ at $J = 1.2$ — since deep in the ordered phase both approximations concentrate on the same configuration; and at the 6×4 strip the plaquette messages settle at $J = 0.25$ and *do not settle at all* at $J = 0.5$, at any damping measured, so the binding constraint at size is convergence rather than accuracy. Neither is a bound: like Bethe, the region-based free energy is a stationary point of a non-convex functional and may fall either side of $\log Z$.

Regime. Exact on a tree, in two passes. On a loopy graph the flooding schedule is iterated with damping to a tolerance and returns the Bethe approximation; the tree schedule is refused rather than run, a number that looks exact and is not being the worst output an evaluator can produce.

What pins it. On a tree, $\log Z$ and every marginal against enumeration to a stated relative tolerance. On a loopy graph nothing asserts agreement with the truth—that would assert something false—and what is asserted is the structure the physics requires: exact at zero coupling, with a deviation from the exact transfer matrix that grows away from it and is reported as a measurement.

29.2 The transfer matrix

The referee the ladder of Section 3 puts above enumeration is the transfer matrix, and it is one identity. Cut an $W \times L$ lattice into L columns of W sites and write a column's configuration as a single variable $c_\ell \in \mathcal{A}^W$, of which there are q^W . Every interaction is then either inside a column or between adjacent ones, so

$$Z = \sum_{c_1, \dots, c_L} \prod_{\ell} \nu(c_\ell) \prod_{\ell < L} \mathsf{T}(c_\ell, c_{\ell+1}), \quad \mathsf{T}(c, c') = \exp \left[\sum_w J \delta(c_w, c'_w) \right], \quad (69)$$

with ν carrying a column's own field and vertical couplings. The sum is a matrix product, $Z = \nu_1^\top \mathsf{T}^{L-1} \nu_L$ under open boundaries and $\text{tr}(\mathsf{T}^L)$ under periodic ones, where the translation invariance folds ν into T . Either way it costs $O(Lq^{2W})$: exponential in the width and *linear* in the length. That asymmetry is the whole reason the method earns a rung. Enumeration over q^{WL} configurations dies in both directions at once, while the transfer matrix dies only in W , so it supplies an exact normalizer on a strip of any length at a width that fits — which is what refereeing belief propagation on a loopy graph requires, since the quantity to compare against has to be exact and cannot be enumerated.

The chain is the case $W = 1$, and there Equation 69 is the forward recursion of Equation 51 with the emission folded into ν : a hidden Markov model's likelihood and a one-dimensional Potts normalizer are one matrix product under two names (Section 26). Replacing the sum by a maximum gives the same product in the max-plus semiring, which is Viterbi on the chain and the ground state of the strip. Two properties follow and both are asserted: the largest eigenvalue of T gives the free energy per column in the limit of many columns, $-\lim_L (\log Z)/L = -\log \lambda_{\max}$, and T is symmetric with strictly positive entries, so Perron–Frobenius makes $\lambda_{\max}$ simple and its eigenvector positive.

What pins it. Against enumeration wherever both run — q^{WL} against $O(Lq^{2W})$, so that is a small lattice — and, past it, against itself contracted the other way: a lattice's normalizer is the same number whether the columns or the rows are the transfer direction, and the two contractions share no loop order. The eigenvalue statement is checked against $(\log Z)/L$ at growing L .

29.3 Ground states as cuts

With two states write $\sigma_i \in \{-1, +1\}$. The pairwise term of Equation 15 is constant on agreeing edges and pays J_{ij} on disagreeing ones, so minimizing $\mathcal{H}$ minimizes the weight of the *cut*—the set of edges whose endpoints differ—plus the field:

$$\mathcal{H}(\sigma) = \sum_{\langle i, j \rangle: \sigma_i \neq \sigma_j} J_{ij} - \sum_i h_i(\sigma_i) + \text{const.} \quad (70)$$

When every $J_{ij} \geq 0$ the pairwise term is submodular, the field becomes a terminal edge to a source or a sink, and the ground state is a minimum s – t cut, exact by max-flow in polynomial time [33, 19]. When couplings are negative the problem is maximum cut, which is NP-hard; the semidefinite relaxation rounded by a random hyperplane gives a cut whose expected weight is at

least $\alpha_{\text{GW}} = 0.87856$ of the optimum,

$$\mathbb{E}[w(\text{cut})] \geq \alpha_{\text{GW}} \max_{\sigma} w(\sigma), \quad \alpha_{\text{GW}} = \min_{0 < \vartheta \leq \pi} \frac{2}{\pi} \frac{\vartheta}{1 - \cos \vartheta}, \quad (71)$$

and the relaxation's own value bounds the optimum from above [32]. The bound is the claim holding at every size; the realized ratio on an instance is measured beside it and never quoted as the guarantee.

Regime. Exact where the couplings are submodular and refused rather than approximated where they are not (Section 34): a wrong ground state on a structured instance is indistinguishable from a right one at any size worth solving. The relaxation runs on any instance and certifies a ratio.

What pins it. Enumeration over $2^{|V|}$ configurations where it reaches; where it does not, a *reduction*: on a bipartite graph the maximum cut is every edge and on a uniform ferromagnet the ground state is constant. A construction error in the cut graph yields a merely worse labelling, which only a regime forcing the general construction to reproduce a simpler validated one catches. A fixture whose answer is trivial measures nothing, so the instances carry the odd cycles of Section 10.

29.4 Alpha expansion

With $q > 2$ states the ground state is NP-hard in general, and alpha expansion approximates it by a sequence of exact two-state problems [18]. An *expansion* on label α lets every site keep its label or switch to α ; the best such move is the minimum cut of a two-state graph built from the current labelling, exact when the pairwise term is a metric on labels, which $\delta(\sigma_i, \sigma_j)$ is. Cycling over α until no expansion lowers the energy, Algorithm 13, reaches a local minimum within a constant of the global one,

$$\mathcal{H}(\hat{\sigma}) \leq 2c \mathcal{H}(\sigma^*), \quad c = \max_{\alpha \neq \beta'} d(\alpha, \beta') / \min_{\alpha \neq \beta'} d(\alpha, \beta'), \quad (72)$$

so a factor of two for the Potts metric, where $c = 1$. The guarantee assumes each expansion is solved to optimality; where it is solved approximately the certificate is optimistic, and only the symptom can be asserted—a value an exact solve could not have produced.

What pins it. The two-label case must reproduce the exact minimum cut of Section 29.3 exactly; enumeration referees the multi-label case where it reaches; and the realized ratio against the enumerated optimum is measured, since the bound is a theorem and the ratio is a number.

29.5 The heat bath, Gibbs sampling and cluster moves

The single-site heat bath draws each spin from its conditional,

$$p(\sigma_i \mid \sigma_{\partial i}) \propto \exp\left(\beta \left[\sum_{j \in \partial i} J_{ij} \delta(\sigma_i, \sigma_j) + h_i(\sigma_i) \right]\right), \quad (73)$$

one site at a time; a *sweep* is one update of every site, Algorithm 9. Near a critical coupling correlated regions span the lattice and single-site moves decorrelate slowly. On the square lattice that coupling is the self-dual point $J_c = \ln(1 + \sqrt{q})$, and on the triangular lattice it is $\ln(1 + v)$ for the positive root of $v^3 + 3v^2 = q$ [12]; both are closed forms, so an instance meant to sit at one is placed there rather than near it. Cluster moves flip such regions together [76, 81, 56]: in the Edwards–Sokal representation [23] a bond is placed between equal neighbours with probability $1 - e^{-\beta J_{ij}}$, and each bond-connected cluster is recoloured uniformly—every cluster at once in

Swendsen–Wang, one cluster grown from a random seed in Wolff. The bond construction is exact at zero field only. In a field the recolouring is accepted with the Metropolis probability on the field energy alone,

$$a = \min \left\{ 1, \exp \left(\beta \sum_{i \in \mathcal{C}} [h_i(\nu) - h_i(\mu)] \right) \right\}, \quad (74)$$

for a cluster $\mathcal{C}$ recoloured from μ to ν , which restores detailed balance (Appendix E.8). Algorithm 10 is the construction and Algorithm 11 the replica exchange of Section B over it.

Regime. Every move preserves the Boltzmann distribution at β , so a sweep may not stop on a state-dependent condition: composing a *number* of steps chosen from the outcome biases toward the states that triggered the stop. The bond probability is a probability only for $J_{ij} \geq 0$, and an antiferromagnet has no like-spin clusters to flip, so the cluster moves refuse a negative coupling rather than run badly.

What pins it. A sampler is validated by the distribution it converges to and never by inspection: at an enumerable size the exact Boltzmann distribution is available, and a chain is held to it by a goodness-of-fit test at a declared significance, thinned by several autocorrelation times since successive sweeps are not independent draws. The test’s power is itself pinned—a move set with the field accept step removed runs, mixes, and is rejected.

29.6 Choosing among these at the supported sizes

Five algorithms are assumed and stated elsewhere: the message iteration and the free energy of its fixed point (Appendices E.3 and E.6), the minimum cut and the expansion cycle built on it (Algorithm 13), the single-site and cluster updates (Algorithms 9 and 10), the replica exchange over them (Algorithm 11), and the semidefinite relaxation with its rounding. They are not alternatives in the ordinary sense: a sampler estimates a distribution, a cut returns one configuration, and message passing returns marginals and a free energy, so the question being asked is what settles first. Scoring one against another’s referee is the standing error here.

Temperature is the axis the table does not carry, and it moves the answer twice. Near a critical coupling the correlated regions span the lattice, so a single-site sweep decorrelates slowly and the cluster move is what makes a sampled estimate affordable — but only at non-negative couplings, exactly where the exact cut is also available for the ground state. Away from criticality the cluster move’s advantage narrows and its refusals remain. The two closed forms of Section 29.5 let an instance be placed *at* a critical point rather than near one, so the comparison runs where the difference is largest.

What the validation assumes. The referees in Section 10.4 rest on three properties of the model, all easy to lose in a move set. The chain must reach every configuration the enumeration sums over: a move set that cannot leave a subspace mixes, passes a convergence diagnostic, and is scored against a distribution over a space it never visits. The number of moves composing a sweep must not depend on the state, since a count chosen from the outcome biases toward the states that chose it — which is why a sweep is defined before it is run. And a temperature must act as a scaling of J and h and in no other way, which is what lets the *unscaled* model’s enumeration referee every replica of a tempered ensemble at once.

30 Algorithms: Frustrated Lattices

30.1 The algorithms that apply

Enumeration over 2^N configurations and the maximum cut of Section 29.3, where they reach; the semidefinite relaxation of Equation 71, which certifies a ratio at any size; single-site descent,

Instance	What is exact	What is worth running
Tree-shaped graph, any size	Message passing, in two passes	Nothing else. Enumeration is affordable only where it is also unnecessary.
Loopy, $q^{ V }$ inside the enumeration ceiling	Enumeration: $\log Z$, every marginal, the ground state	Every method, and none of them for its answer — this is where a method is scored rather than used.
A strip at a width that fits, any length	The transfer matrix, for the normalizer	Message passing, whose deviation from it is the measurement that says what the loopy fixed point costs.
Two states, every coupling non-negative	The minimum cut, in polynomial time at any size	The cut. A sampler run for a ground state here spends a budget on a solved problem.
More than two states, couplings non-negative	Nothing	Expansion, whose factor-of-two bound (Equation 72) is a certificate where an optimum is unavailable; the realized ratio is measured beside it.
Any negative coupling	Nothing	Single-site sampling, annealing and tempering — the cut and the cluster moves refuse. Which of the three wins is a property of the instance, measured on Section 11's.
Marginals past the enumeration ceiling	Nothing	Message passing alone, returning the Bethe approximation, which is reported and asserted against nothing.

Table 11: What is exact on each kind of Potts instance and what is therefore worth a budget. The rows are instance properties and not sizes: above two states, or below zero coupling, a method is refused whatever the lattice extent.

which is Algorithm 9 at zero temperature, and the annealing and parallel tempering of Section B, Algorithm 11, which are the searches these instances exist to separate. The exact cut refuses a negative coupling, since the energy is not submodular, and the cluster moves refuse it for the reason above, so the instances draw the boundary between what is solved exactly and what is only bounded or sampled.

30.2 The initializer

A uniformly random configuration, and restarts of it: on a landscape whose optimum is degenerate, which start is drawn decides which ground state is reached and the count of restarts is part of the budget the comparison charges.

What pins it. Equation 16 at every size, and enumeration attaining it at the sizes enumeration reaches, with the degeneracy asserted and the entropy reported; Equation 17 as an inequality on every search's result, with descent landing on the planted energy at low f and below it at high f measured rather than assumed. Whether annealing or tempering beats restarts is a property of the instance, stated as a measurement at equal sweeps (Section B).

30.3 Choosing among these at the supported sizes

These instances add no algorithm. Descent, annealing and replica exchange are Section 10's unchanged (Algorithms 9 and 11); what the section supplies is an answer known from outside at every size, so a search is scored past the reach of every other oracle here.

What each instance can and cannot referee. The difference decides what a measurement on either may say. The triangular antiferromagnet states an *energy* and not a configuration: the optimum is massively degenerate, so attaining Equation 16 proves a search succeeded and says nothing about which ground state it found. The planted glass states an *upper bound* and not

an energy: a search returning more than Equation 17 has failed, and one reaching it has not thereby succeeded, since at a large inverted fraction the true optimum lies below the planted state. One instance can acquit and the other can only convict.

The axis is the disorder, not the size. On the glass the useful parameter is the inverted fraction and not the node count: at zero inversion the instance is a gauge transform of the ferromagnet, a random start finds the answer and the comparison measures nothing, so the range worth running begins where the planted state stops being near-optimal. A success rate reported without its fraction describes no instance. On the triangular lattice the analogous parameter is the extent, and it moves the degeneracy rather than the energy: the closed form holds at every extent while the number of configurations attaining it does not, which is why the extents are declared rather than swept.

What is unavailable here, by construction. This is the one section whose applicability question is answered entirely by refusals. The exact cut refuses, the energy not being submodular; the cluster moves refuse, $1 - e^{-\beta J}$ not being a probability at a negative coupling. The exact and near-exact methods are gone before the comparison starts, and what is compared is three searches with no guarantee between any two, at an equal budget of sweeps over shared seeds. Those refusals are checked as results rather than described in prose.

What the validation assumes. The counting argument closes on a geometry: periodic boundaries, $3N$ bonds, $2N$ triangles, every bond in exactly two of them. An open boundary or a different lattice leaves the same code running and Equation 16 false, and no enumeration past the $2^{|V|}$ ceiling would notice. The residual entropy is the standing example of a quantity reported and not asserted: it is a thermodynamic limit, and a finite lattice is not the limit.

31 Algorithms: The Coupled Spatio-Sequential Model

31.1 Block-coordinate ascent

The blocks are the emission parameters θ and the labels ℓ , with the chain states marginalized. For θ given ℓ , the gradient of the marginal likelihood is the posterior-weighted gradient of the emission terms,

$$\nabla_{\theta} \log p(x \mid \ell, \theta) = \sum_{n,m} \mathbf{1}[\ell_n = m] \sum_{s,k_{s,m}} \mathbb{Q}(k_{s,m} \mid \theta, \ell; x) \nabla_{\theta} \log p(x_{sn} \mid k_{s,m}, \theta_m), \quad (75)$$

where $\mathbb{Q}$ is the per-class posterior of Equation 52: this is expectation-maximization with the E step forward-backward on each class's chain over the nodes assigned to it, and t and Π_m re-estimate in closed form. For ℓ given θ , the same posterior defines a per-node, per-class external field,

$$h_{nm}(\theta, \theta'; \ell) \equiv - \sum_{s,k_{s,m}} \mathbb{Q}(k_{s,m} \mid \theta', \ell; x) \log p(x_{sn} \mid k_{s,m}, \theta), \quad (76)$$

and the label update is the ground state of a Potts model in that field,

$$\ell^* = \arg \min_{\ell} \beta \left[\sum_{\langle n, n' \rangle} J_{nn'} \mathbf{1}[\ell_n \neq \ell_{n'}] + \sum_{n,m} \mathbf{1}[\ell_n = m] h_{nm} \right], \quad (77)$$

which is the problem of Section 29.4, solved by alpha expansion with its bound, descended by iterated conditional modes, or sampled by an annealed cluster move with a field (Section 29.5). Each block does not decrease the joint likelihood, which is the monotonicity every expectation-maximization here is held to.

Why a cluster move. Under an effective spatial prior each class spans a contiguous region, and a single-site move must cross it one node at a time against the prior’s penalty; iterated conditional modes freezes into spurious disjoint clusters, and single-site annealing slows critically for the same reason. The Wolff move of Section 29.5 recolours a whole cluster at once, and Algorithm 4 is that move with the field of Equation 76 in its accept step. At $\beta \rightarrow 0$ it is a single-site Gibbs update, at $\beta \gg 1$ the greedy entrapment of iterated conditional modes; annealing in β moves between them.

Algorithm 4 One Wolff update on the labels, in the external field

```

1: draw a root  $\rho$  uniformly from the nodes;  $\mu \leftarrow \ell_\rho$ ;  $\mathcal{C} \leftarrow \{\rho\}$ ; queue  $Q \leftarrow \{\rho\}$ 
2: while  $Q \neq \emptyset$  do
3:   pop  $n$  from  $Q$ 
4:   for each neighbour  $n'$  of  $n$  with  $\ell_{n'} = \mu$  and  $n' \notin \mathcal{C}$  do
5:     with probability  $1 - e^{-\beta J_{nn'}}$ : add  $n'$  to  $\mathcal{C}$  and to  $Q$ 
6:   end for
7: end while
8: draw a new label  $\nu$  uniformly from  $\{1, \dots, M\}$ 
9:  $\Delta H \leftarrow \sum_{n \in \mathcal{C}} (H_{n\nu} - H_{n\mu})$ 
10: with probability  $\min\{1, e^{-\beta \Delta H}\}$ :  $\ell_n \leftarrow \nu$  for every  $n \in \mathcal{C}$ 

```

31.2 The data-sampled burn-in

A start ignoring the graph and the emission family—*k-means++* on the raw observations—leaves degenerate classes and false positives the ascent then polishes. Algorithm 5 anneals in β from zero, so early on the labels are nearly free and the chains and emission parameters are fitted to what the data alone supports, the spatial prior tightening as the classes separate. Its seeding step, EMISSIONMIXTURE++, is *k-means++* with the emission family’s own Bregman divergence in place of squared Euclidean distance [6, 11]: candidates are drawn proportionally to how badly the current parameters explain them *relative to the best the family does at each*, constrained to the observations the current labels and states assign to the class being seeded. The $8(\ln K + 2)$ expected-cost bound of *k-means++* is a statement about squared distance and is not inherited; what is measured instead is recovery against a uniform start at an equal budget, over seeds, reported whether it wins or not.

Algorithm 5 Data-sampled burn-in on the coupled graph

```

1:  $\beta \leftarrow 0$ ;  $H \leftarrow 0$ ;  $\ell_n \sim \text{Uniform}\{1, \dots, M\}$  for every node
2:  $\theta \leftarrow \text{EMISSIONMIXTURE++}(\beta, K, \theta, x)$ 
3: while  $\beta < \beta_0$  do
4:   WOLFFUPDATE( $\beta, \ell, J, H$ ) ▷ Algorithm 4
5:   for each class  $m$  do draw the chain  $k_{\cdot, m}$  by block Gibbs given  $\ell, \theta$ 
6:   end for
7:   for each state  $k$  do  $\theta_k \leftarrow \text{EMISSIONMIXTURE++}(\beta, K, \theta, x_k, \ell)$  ▷ state-constrained
8:   end for
9:    $\mathbb{Q} \leftarrow$  forward-backward per class (Equation 52); M step for  $\theta, t, \Pi$ 
10:   $h_{nm} \leftarrow$  Equation 76
11:   $\beta \leftarrow \beta + \Delta\beta$ 
12: end while
13: return  $\ell, \theta$ 

```

What pins it. The graph of Equation 18 is constructed from the three adapters and its log-density equals the joint written out on every assignment of an enumerable instance; the simulator’s label and state marginals are held to enumeration; the gradient identity Equation 75 to central differences; the joint is non-decreasing across every block for every label solver; and label accuracy up to permutation against the planted labels is measured at an equal budget of sweeps for each of the three solvers, so whether a cluster move escapes what iterated conditional modes freezes into is a measurement and not a premise—and on the planted lattice the start, not the move, decides it.

31.3 Choosing among these at the supported sizes

The estimator is dictated rather than chosen: neither marginal of Equation 18 is tractable, so the blocks are what remain, and what is chosen is the label solver and the start. Three algorithms are assumed — the per-class chain marginalization of Section 7, one of the three label solvers of Section 10, and the annealed start of Algorithm 5, whose cluster move is Algorithm 4.

What each size decides. The enumerable instance scores the constructed graph’s log-density and the per-class posteriors against a sum, and decides nothing about the label solvers, which are separated past it. There the planted labels are the referee and the choice between solvers is a measurement at an equal budget of sweeps, whose standing result is that on the planted lattice the start decides the labelling and the move does not.

The two axes past enumeration cost differently. The sequential extent enters the chain pass linearly and identifies the emission parameters, so more positions buy sharper components. The node count enters the label solve, which is not linear in it, and buys spatial resolution alone. An instance at five thousand nodes and twenty thousand positions is two hard problems bolted together, refereed by the planted labelling for the first and a reference posterior for the second.

Which label solver, and what each needs. Descent needs nothing of the model and is what remains when the budget is too small for either alternative; it freezes into disjoint clusters, a property of a single-site move against a prior penalizing disagreement rather than a defect. Expansion needs the pairwise term to be a metric on labels, which δ is, and returns the factor-of-two certificate of Equation 72. The cluster move needs non-negative couplings — which Equation 19 has by construction, so Section 11’s refusal does not bind here — and moves a whole region at once, at the price of being a sampler with a schedule rather than a solver.

What the validation assumes. One structural property carries most of Section 12.3: the gated factor has degree two, joining one label to one chain state, so conditioning on the labels leaves a set of independent chains and the per-class two-pass recursion *is* the expectation step rather than an approximation of it. A gate over more than one chain state would leave the same code running and the same monotonicity holding, with the expectation step no longer the posterior. Two weaker assumptions follow: classes are identified only up to permutation, and the planted labelling is what a correct estimator recovers — strictly weaker than its being the maximum of Equation 77.

32 Algorithms: LDPC Decoding

32.1 Sum-product in the log domain

A binary variable’s message is one number, its log-odds, and Equation 2 on a parity factor has a closed form [29, 47, 61]; the decoder is Algorithm 16. From bit i to check a , and from check a to

bit i ,

$$m_{i \rightarrow a} = L_i + \sum_{b \in \partial i \setminus a} m_{b \rightarrow i}, \quad m_{a \rightarrow i} = 2 \operatorname{artanh} \prod_{j \in \partial a \setminus i} \tanh \frac{m_{j \rightarrow a}}{2}, \quad \tilde{L}_i = L_i + \sum_{b \in \partial i} m_{b \rightarrow i}, \quad (78)$$

where the check update follows from $\tanh(L/2) = p(0) - p(1)$ and the fact that the parity of independent bits has a difference of probabilities equal to the product of theirs. The hard decision $\hat{c}_i = \mathbf{1}[\tilde{L}_i < 0]$ is the bitwise MAP under the Bethe beliefs. Replacing the sum by a maximum in Equation 2 gives max-product, whose log-domain check update is

$$m_{a \rightarrow i} = \left(\prod_{j \in \partial a \setminus i} \operatorname{sgn} m_{j \rightarrow a} \right) \min_{j \in \partial a \setminus i} |m_{j \rightarrow a}|, \quad (79)$$

the min-sum decoder, whose decision is the blockwise MAP and is exact on a cycle-free code with a unique maximum; on a loopy code it is the cheaper and the weaker of the two, and the gap is measured rather than assumed.

Three details are load-bearing. The schedule is flooding: every check reads the bit messages of the previous half-iteration and every bit reads the check messages just written, one Jacobi sweep per direction over all edges at once, the schedule the general implementation runs and so the one it is compared against. The loop stops at the first iteration whose hard decision has every bit decided and satisfies $\mathbf{H}\hat{\mathbf{c}} = 0$, the syndrome check, and otherwise at an iteration cap, which is a decoding failure and the event a block error rate counts. And messages are clipped at $\pm L_{\max}$ with $L_{\max} = 30$ before the check update, because a certain bit's ratio is infinite on the erasure channel and the leave-one-out sums in Equation 78 are formed by subtraction: $\tanh(L_{\max}/2)$ is below one by 1.9×10^{-13} in double precision, and a belief formed under the cap differs from the exact one by less than $e^{-L_{\max}}$ in probability, which the oracle tests absorb.

32.2 The all-zero codeword

Every simulation past the reach of an encoder sends the zero word, and the argument that this loses nothing is standard [61, Sec. 4.1]. Each channel of Equation 21 is output symmetric: sending $c_i = 1$ instead of 0 negates L_i in distribution, and on the symmetric and erasure channels it negates it on every realization, since a flip or an erasure is applied to the bit whatever its value. Both check updates are odd in each argument and the bit update is linear, so negating the ratios at the bits where $c_i = 1$ negates every message and every posterior at exactly those bits, and the hard decision under c is the hard decision under 0 with c added: the error pattern $c \oplus \hat{\mathbf{c}}$ has the same distribution whatever codeword was sent. Bit and block error rates measured on the zero word are therefore the rates on any codeword. Where an encoder exists—Gauss–Jordan elimination over $\text{GF}(2)$ to a systematic form, at $n \leq 512$ —the identity is checked per realization on a non-trivial codeword, so the shortcut is itself pinned.

What pins it. Four oracles, none sharing code with the decoder. On a cycle-free code the tree schedule of the general implementation and the enumeration over all 2^k codewords are exact, and the decoder equals both; min-sum equals the general max-product there and returns the enumerated maximum-likelihood codeword, with the margin over the runner-up pinned so that no tie is being broken. On loopy codes the decoder and the general damped flooding are the same Bethe approximation and reach the same fixed point, which is agreement and not truth. The encoder is pinned by $\mathbf{H}\mathbf{c} = 0$ and the offsets layout by the dense product it stands in for. And at size, where no enumeration reaches, the closed form is density evolution (Appendix E.10): on the erasure channel the $(3, 6)$ ensemble has a threshold $\epsilon^* = 0.4294$ below which the infinite-length decoder resolves every erasure and above which it does not, and the 19,998-bit code is held to bracketing it.

32.3 Choosing among these at the supported sizes

Four things are assumed and stated elsewhere: the specialized message updates (Algorithm 16), the ensemble construction, an elimination-based encoder, and the fixed-point recursion of Appendix E.10. The choice inside the section is between the two updates, and the choice across sizes is between three kinds of claim that do not overlap.

Which update, and they answer different questions. The tanh rule of Equation 78 carries the whole distribution of a check’s incoming messages; the min-sum rule of Equation 79 keeps a sign and a magnitude and discards the rest. On a cycle-free code the two are the bitwise posterior and the most probable codeword, both exact and both meaningful — the same pair Section 7 draws on a chain. On a loopy code neither is exact, and min-sum is the cheaper and the weaker; the gap between them is measured rather than argued, and it is the reason the weaker one is kept in the document at all.

Three regimes, three kinds of claim. Enumerating 2^k codewords reaches about thirty-four bits at rate one-half, and inside that a decode is compared with an answer. To five hundred and twelve bits an encoder exists, so a non-trivial codeword can be transmitted and the symmetry argument that licenses everything past it is *checked* rather than assumed. Past both, the only statement is about the ensemble rather than the code: the threshold of Appendix E.10 exists on the erasure channel and on neither of the other two, so at the length the roadmap names the symmetric and Gaussian channels carry a measured curve with no oracle behind it. That is a marked cell in Section 16 and not an oversight, and what would fill it is the same recursion extended past the erasure channel.

What the decoder needs of the instance. Sparsity, and short cycles are what cost it accuracy — so a dense parity-check matrix is not a small instance of this problem but a different one, and the accuracy of a decode is a property of the graph drawn from the ensemble rather than of the update applied to it. The message clip is the other precondition and is a numerical one with a stated consequence: a certain bit’s ratio is infinite on the erasure channel and the leave-one-out sums are formed by subtraction, so the clip is what makes the arithmetic finite, and its effect on a belief is bounded in probability rather than removed.

What the validation assumes. Output symmetry of the channel, together with the check update being odd in each argument and the bit update linear. Those three are what make the error pattern’s distribution independent of the codeword sent, and therefore what license every rate measured on the all-zero word past the reach of an encoder — which is most of the measurements in this section. The second assumption is a stopping rule: an iteration that hits the cap is a decoding failure and is counted as one, so a rate measured without that convention counts a different event and is not comparable with anything here.

33 Algorithms: Turbo Decoding

33.1 BCJR and the extrinsic exchange

The forward and backward recursions on Equation 35 are the BCJR algorithm [10], which is Equation 51 with the observation moved from the state to the edge:

$$\alpha_{t+1}(s') = \sum_s \alpha_t(s) \psi_t(s, s'), \quad \beta_t(s) = \sum_{s'} \psi_t(s, s') \beta_{t+1}(s'), \quad (80)$$

and the bitwise ratio the decoder reports is their product over the edges each input bit labels,

$$\tilde{L}_t = \log \frac{\sum_{(s,s'): u=0} \alpha_t \psi_t \beta_{t+1}}{\sum_{(s,s'): u=1} \alpha_t \psi_t \beta_{t+1}}, \quad (81)$$

with α_0 and β_{K+m} concentrated on the zero state because both ends are terminated. Replacing the sum by a maximum gives Viterbi, whose decision is the blockwise MAP against Equation 81's bitwise one — two decodings of one model and different answers, as Section 7 says of the chain, and the received word that separates them is a test.

Turbo decoding is Algorithm 17: two BCJR passes exchanging the *extrinsic* part of the posterior,

$$L_t^e = \tilde{L}_t - L_t^s - L_t^a, \quad (82)$$

what one chain learned about u_t from every step but t . The subtraction is not a refinement: a decoder handed the other's full posterior would be handed its own previous output inside it, and the two would agree on a shared error whatever the channel said. Equation 82 is the sum-product message from the message-bit variable to the other chain, and the serial order — chain one, interleave, chain two, deinterleave — is the schedule, not a consequence of it.

33.2 What pins it

Four referees, and they answer different questions. *Enumeration over the 2^K messages* is exact at $K = 12$ and referees three things: the BCJR posterior of one constituent code, which it matches to 3×10^{-14} because a chain is a tree and the recursion is exact there; the Viterbi decision, with the margin over the runner-up pinned so that no tie is being broken; and the turbo iteration, which it does *not* match, since the joint graph has cycles — what is asserted there is that the iteration's bit error rate falls from its first iteration and stays within a stated factor of the exact MAP's. *The general sum-product*, on the same chain built as a factor graph, referees the specialization: the tree schedule gives the same posteriors and the same $\log Z$ to 3×10^{-15} , and max-product returns the Viterbi path, which is the pairing Section 13 established. *The nullspace parity-check matrix* referees the encoder: the 2^K words the shift register produces are the 2^K words Section 13's enumeration finds in the null space of a matrix built without it. And *the uncoded closed form $Q(\sqrt{2E_b/N_0})$* referees the waterfall at the lengths no enumeration reaches.

33.3 Successive cancellation, and the list

Equation 36's butterfly is decoded one input at a time. The recursion halves the block at each step: the left half sees the soft combination f of the two halves' ratios, and the right half sees the second half corrected by what the left half decided, g (Equation 39). A frozen input is not decided — it is known — but its evidence still enters the path metric, which is what keeps an unlikely prefix from ranking beside a likely one. Algorithm 18 is one candidate and Algorithm 19 is L of them [5, 77].

33.4 Choosing among these at the supported sizes

The trellis recursions of Equation 81, their max-plus twin, and the extrinsic exchange of Algorithm 17 are what this section assumes; the channel and the ratio convention are Section 13's unchanged. What is worth discussing is why the constituent decoder and the pair sit on opposite sides of every claim in this document.

One code is a chain; two are not. A terminated trellis is a chain with both ends pinned, so its factor graph is a tree and its decoder is exact at any length — the only decoder in the two coding sections with that property, and the reason a constituent posterior can be pinned to an enumeration to the last few digits. The pair is loopy, and the cycles are not an artefact of a short interleaver: the permutation puts one around every pair of positions it reorders. So the length that makes the iteration behave is the length that makes those cycles long, and a *short* interleaver is the hard case while a long one is the easy one. That inverts the size axis of every other problem statement here, where a larger instance is a harder one, and it is why the declared message lengths span two orders of magnitude rather than clustering at the enumerable end.

Three regimes. Inside the message enumeration the exact bitwise answer is available, so the iteration’s gap is a measured factor rather than a hope. To five hundred and twelve bits a dense nullspace turns the code into a parity-check matrix, and the referees of Section 13 are inherited whole. Past both, one closed form bounds the curve from a single side — the uncoded error rate, which every coded curve must fall below — and the resolution of a measurement is set by the frame count it rests on, so a rate is reported with that count or it is not reported.

What may not be claimed, and why it is stated as a refusal. The natural claim about an iterative decoder is that each iteration improves on the last. It is not made here, because the measurement refuses it: the ensemble error rate rises between consecutive iterations at several declared points, every rise inside one binomial interval of the sample the point rests on. The weaker pair the sample supports is asserted instead. This is the standing example in this document of a property that sounds like a correctness condition and is not one — non-monotonicity is a property of loopy message passing on a serial schedule, so a decoder asserted to be monotone would be asserting a defect of its referee.

What the validation assumes. Both trellises terminated, which is what pins the recursion’s boundary at each end and makes the constituent graph a chain rather than a chain with a free end. Output symmetry carried through a *recursive* encoder and through the message map, re-checked here rather than inherited from Section 13, because the tail of a recursive encoder is a function of the register state and not a string of zeros. And that the interleaver is a permutation drawn from a recorded seed, so the code is reproduced by drawing it again — a written-out array would be an artefact no reader could check and no rebuild could verify.

Part III

Validation

34 The Properties That Referee Each Method

An algorithm is accepted here against a property that is true independently of this implementation. The properties are not interchangeable, and which one applies is a statement about the method rather than a matter of convenience. Each *What pins it* paragraph above names one of the five kinds below, and the regression suite marks every check with the kind it is: an oracle, a simulated truth, a mathematical invariant, an edge case, or a structural property of the code. A check that fits none of them is either a missing category or a test with nothing to assert.

34.1 Exact answers, where an oracle exists

Where a quantity can be computed a second way that shares no recursion with the first, equality is asserted to a stated tolerance. Direct marginalization over k^{n-2} internal assignments referees

pruning; a sum over k^T paths referees the forward recursion; exhaustive enumeration over $q^{|V|}$ configurations referees a Potts marginal; enumeration of $(2n - 5)!!$ topologies referees a search. Every one is exponential, and that is what makes it an oracle rather than a second opinion. **Two implementations agreeing prove nothing if both are wrong**, so an accelerated method is pinned against the reference and never against another acceleration.

34.2 Structural invariants, where no oracle reaches

Past the sizes enumeration allows, what remains are properties the model must satisfy whatever the answer is:

- rows of a transition matrix sum to one, and $\mathbf{P}(0) = \mathbf{I}$;
- a reversible model satisfies detailed balance, $\pi_i \mathbf{Q}_{ij} = \pi_j \mathbf{Q}_{ji}$;
- the *pulley principle*: on a reversible model the likelihood is invariant to where the root is placed along a branch, so a rooted computation and its re-rooting must agree exactly;
- a gradient matches central differences, relative to the gradient's own norm—entrywise fails wherever an entry is zero, and an absolute bound does not transfer across data sizes;
- an expectation-maximization iterate increases the likelihood monotonically.

None of these establishes that a method is right. Each establishes that a specific way of being wrong did not happen, which is a weaker and more defensible claim.

34.3 Recovery, which is the acceptance test for a fit

A likelihood that increases proves the optimizer runs, not that the model is right. The acceptance test is to fit data simulated from known parameters and require the intervals to cover the truth at their nominal rate over independent replicates. Two qualifications are load-bearing. Where a model has an exact symmetry—permuting the hidden states of an HMM or the components of a mixture leaves the likelihood unchanged—recovery is stated *up to that symmetry*, and the alignment is part of the test. And coverage is measured as a function of how identifiable the instance is, because a fixture chosen where everything is well separated is a test of the fixture.

34.4 Where the likelihood has no maximum, or no curvature

Two failures are properties of models rather than defects, and both must be handled before they are discovered.

An *unbounded* likelihood has no maximum to find: put a Gaussian component's mean on a single observation and let its variance go to zero and the density there diverges. A fit that converges there was stopped by its starting point or by a floor, and which one must be knowable—so the bound is explicit, derived from the data rather than chosen, and reaching it is a refusal rather than a clamp.

A *flat* likelihood has a maximum that runs to the boundary: an overdispersed count model approaches its Poisson limit as the dispersion grows, and past the point where the overdispersion is smaller than the sampling noise on measuring it, the data cannot tell one value from another. The two failures are mirror images, and they announce themselves differently—the first as a divergence, the second as a singular information matrix—which is why each is measured rather than inferred from the other.

34.5 Agreement across implementations is a tolerance

An accelerated implementation is pinned against the reference within a declared *relative* tolerance, never bitwise. Relative, because a log-likelihood is a sum over sites and an absolute bound fixed at one size does not transfer to another. Keyed on the *lowest* precision in the comparison, because a single-precision device cannot be held to a double-precision bound, and one bound loose enough for single precision would let a broken double-precision backend pass. A discrepancy inside the tolerance is not a defect and is not corrected; one outside it is not a tolerance to loosen. The values themselves live beside the measurements they are derived from, not here.

34.6 Published constants, and asymptotic results

Some claims come from outside entirely: the $(2n - 5)!!$ count, Goemans–Williamson’s 0.87856 for maximum cut, alpha expansion’s factor of two, the $8(\ln k + 2)$ bound on k -means++ seeding. These referee an implementation at any size. **No asymptotic result is testable at the sizes an exact oracle allows**, however: a threshold or a limit holds as the problem grows and says nothing where enumeration can check it. Such a result is reported for orientation and the per-instance property is what is asserted.

35 Relevance to the Roadmap

The problem classes are not applications. They are chosen so that each abstraction is tested against something that is not the model it was extracted from: a likelihood engine justified only by trees would be a phylogenetics library, and an optimizer justified only by an HMM would encode a chain.

The order the milestones take follows the dependency between the properties above. Simulation and its ground truth come first, because every later claim is checked against data whose generating parameters are known. The exact evaluators come next, because they are the oracles the accelerated ones are pinned to. Continuous optimization follows, since recovery is the acceptance test and needs both. Discrete search and learned proposals come last, because their acceptance criterion—reaching the optimum enumeration identifies—only exists once enumeration does.

35.1 Beyond the Roadmap

Each problem statement above ends with what it would take to go further and what it deliberately does not attempt. Three directions recur across them, and they are worth stating once as a whole rather than eleven times in pieces.

A bound where there is no oracle. The recurring gap is not speed: it is that past a size, a search returns an answer nothing can score. Every section that names a missing referee names the same kind of object — a branch and bound over topologies, a boundary contraction for a lattice, a density evolution past the erasure channel, an enumerated maximum-posterior labelling on one coupled instance. Each converts a reported optimum into a bracket, and a bracket is the difference between a result and a measurement.

Learning as a proposal, never as an answer. A learned policy, a learned surrogate, a learned ground state and a learned decoder all enter this document in the same position: they choose what to evaluate, and the thing evaluated is still the objective the section defines. That is what keeps them refereeable — a proposal that fails is slower, where an answer that fails is wrong — and it is the reading the field’s own reviews take of the combinatorial case [13, 49]. The architectures the proposals would use are settled elsewhere and are cited rather than restated: a graph network over the structure [39], an attention model over its parts [43]. The standing caution is the one Section 10.5 records — such a method is measured against a competently

tuned classical baseline at a matched budget, or it has not been measured [3] — and the standing requirement is that a rate is reported with the interval, the seeds and the budget behind it [35, 1].

Structure that is differentiable. Section 19 and Section 27.8 approach one question from two sides: whether a discrete structure can be moved by a gradient without becoming a different problem. Both answer it the same way — the relaxation must agree with what it relaxes at every corner — and both are limited by the same thing, which is that the corner check needs an enumeration. Extending it to a space no enumeration reaches is where the structured relaxations [59] and the parameterized topology spaces [85] would have to meet.

References

- [1] Rishabh Agarwal, Max Schwarzer, Pablo Samuel Castro, Aaron Courville, and Marc G. Bellemare. Deep reinforcement learning at the edge of the statistical precipice. In *Advances in Neural Information Processing Systems*, volume 34, 2021.
- [2] Gautam Altekar, Sandhya Dwarkadas, John P. Huelsenbeck, and Fredrik Ronquist. Parallel Metropolis coupled Markov chain Monte Carlo for Bayesian phylogenetic inference. *Bioinformatics*, 20(3):407–415, 2004.
- [3] Maria Chiara Angelini and Federico Ricci-Tersenghi. Modern graph neural networks do worse than classical greedy algorithms in solving combinatorial optimization problems like maximum independent set. *Nature Machine Intelligence*, 5:29–31, 2023.
- [4] Thomas Anthony, Zheng Tian, and David Barber. Thinking fast and slow with deep learning and tree search. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [5] Erdal Arıkan. Channel polarization: A method for constructing capacity-achieving codes for symmetric binary-input memoryless channels. *IEEE Transactions on Information Theory*, 55(7):3051–3073, 2009.
- [6] David Arthur and Sergei Vassilvitskii. k-means++: The advantages of careful seeding. In *Proceedings of the Eighteenth Annual ACM-SIAM Symposium on Discrete Algorithms*, pages 1027–1035, 2007.
- [7] Kevin Atteson. The performance of neighbor-joining methods of phylogenetic reconstruction. *Algorithmica*, 25(2–3):251–278, 1999.
- [8] Dana Azouri, Shiran Abadi, Yishay Mansour, Itay Mayrose, and Tal Pupko. Harnessing machine learning to guide phylogenetic-tree search algorithms. *Nature Communications*, 12: 1983, 2021.
- [9] Pierre-Luc Bacon, Jean Harb, and Doina Precup. The option-critic architecture. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 31, 2017.
- [10] Lalit Bahl, John Cocke, Frederick Jelinek, and Josef Raviv. Optimal decoding of linear codes for minimizing symbol error rate. *IEEE Transactions on Information Theory*, 20(2): 284–287, 1974. doi: 10.1109/TIT.1974.1055186.
- [11] Arindam Banerjee, Srujana Merugu, Inderjit S. Dhillon, and Joydeep Ghosh. Clustering with bregman divergences. *Journal of Machine Learning Research*, 6:1705–1749, 2005.
- [12] Rodney J. Baxter. *Exactly Solved Models in Statistical Mechanics*. Academic Press, London, 1982.

- [13] Yoshua Bengio, Andrea Lodi, and Antoine Prouvost. Machine learning for combinatorial optimization: A methodological tour d’horizon. *European Journal of Operational Research*, 290(2):405–421, 2021.
- [14] Claude Berrou, Alain Glavieux, and Punya Thitimajshima. Near Shannon limit error-correcting coding and decoding: Turbo-codes. In *Proceedings of ICC ’93 — IEEE International Conference on Communications*, volume 2, pages 1064–1070, 1993. doi: 10.1109/ICC.1993.397441.
- [15] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, New York, 2006.
- [16] Richard E. Blahut. *Algebraic Codes for Data Transmission*. Cambridge University Press, Cambridge, 2003.
- [17] Stephen Boyd and Lieven Vandenberghe. *Convex Optimization*. Cambridge University Press, 2004.
- [18] Yuri Boykov, Olga Veksler, and Ramin Zabih. Fast approximate energy minimization via graph cuts. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 23(11):1222–1239, 2001.
- [19] Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein. *Introduction to Algorithms*. MIT Press, Cambridge, Massachusetts, 4th edition, 2022.
- [20] Thomas M. Cover and Joy A. Thomas. *Elements of Information Theory*. Wiley, 2 edition, 2006.
- [21] Richard Durbin, Sean R. Eddy, Anders Krogh, and Graeme Mitchison. *Biological Sequence Analysis: Probabilistic Models of Proteins and Nucleic Acids*. Cambridge University Press, Cambridge, 1998.
- [22] David J. Earl and Michael W. Deem. Parallel tempering: Theory, applications, and new perspectives. *Physical Chemistry Chemical Physics*, 7:3910–3916, 2005.
- [23] Robert G. Edwards and Alan D. Sokal. Generalization of the Fortuin–Kasteleyn–Swendsen–Wang representation and Monte Carlo algorithm. *Physical Review D*, 38(6):2009–2012, 1988.
- [24] Joseph Felsenstein. Evolutionary trees from DNA sequences: A maximum likelihood approach. *Journal of Molecular Evolution*, 17(6):368–376, 1981.
- [25] Joseph Felsenstein. *Inferring Phylogenies*. Sinauer Associates, Sunderland, Massachusetts, 2004.
- [26] Walter M. Fitch. Toward defining the course of evolution: Minimum change for a specific tree topology. *Systematic Zoology*, 20(4):406–416, 1971.
- [27] G. David Forney. Codes on graphs: Normal realizations. *IEEE Transactions on Information Theory*, 47(2):520–548, 2001.
- [28] Brendan J. Frey. *Graphical Models for Machine Learning and Digital Communication*. MIT Press, Cambridge, Massachusetts, 1998.
- [29] Robert G. Gallager. Low-density parity-check codes. *IRE Transactions on Information Theory*, 8(1):21–28, 1962.

- [30] Charles J. Geyer. Markov chain Monte Carlo maximum likelihood. In *Computing Science and Statistics: Proceedings of the 23rd Symposium on the Interface*, pages 156–163, 1991.
- [31] Charles J. Geyer. Practical Markov chain Monte Carlo. *Statistical Science*, 7(4):473–483, 1992.
- [32] Michel X. Goemans and David P. Williamson. Improved approximation algorithms for maximum cut and satisfiability problems using semidefinite programming. *Journal of the ACM*, 42(6):1115–1145, 1995.
- [33] D. M. Greig, B. T. Porteous, and A. H. Seheult. Exact maximum a posteriori estimation for binary images. *Journal of the Royal Statistical Society: Series B*, 51(2):271–279, 1989.
- [34] Nikolaus Hansen and Andreas Ostermeier. Completely derandomized self-adaptation in evolution strategies. *Evolutionary Computation*, 9(2):159–195, 2001.
- [35] Peter Henderson, Riashat Islam, Philip Bachman, Joelle Pineau, Doina Precup, and David Meger. Deep reinforcement learning that matters. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 32, 2018.
- [36] Michael D. Hendy and David Penny. Spectral analysis of phylogenetic data. *Journal of Classification*, 10(1):5–24, 1993.
- [37] Matthew D. Hoffman and Andrew Gelman. The No-U-Turn sampler: Adaptively setting path lengths in Hamiltonian Monte Carlo. *Journal of Machine Learning Research*, 15:1593–1623, 2014.
- [38] Daniel Hsu, Sham M. Kakade, and Tong Zhang. A spectral algorithm for learning hidden Markov models. *Journal of Computer and System Sciences*, 78(5):1460–1480, 2012.
- [39] Elias Khalil, Hanjun Dai, Yuyu Zhang, Bistra Dilikina, and Le Song. Learning combinatorial optimization algorithms over graphs. In *Advances in Neural Information Processing Systems*, volume 30, 2017.
- [40] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi. Optimization by simulated annealing. *Science*, 220(4598):671–680, 1983.
- [41] Daphne Koller and Nir Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press, Cambridge, Massachusetts, 2009.
- [42] Vladimir Kolmogorov and Ramin Zabih. What energy functions can be minimized via graph cuts? *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 26(2):147–159, 2004.
- [43] Wouter Kool, Herke van Hoof, and Max Welling. Attention, learn to solve routing problems! In *International Conference on Learning Representations*, 2019.
- [44] Alexey M. Kozlov, Diego Darriba, Tomáš Flouri, Benoit Morel, and Alexandros Stamatakis. RAXML-NG: A fast, scalable and user-friendly tool for maximum likelihood phylogenetic inference. *Bioinformatics*, 35(21):4453–4455, 2019.
- [45] Frank R. Kschischang, Brendan J. Frey, and Hans-Andrea Loeliger. Factor graphs and the sum-product algorithm. *IEEE Transactions on Information Theory*, 47(2):498–519, 2001.
- [46] Jonathan Machta. Population annealing with weighted averages: A Monte Carlo method for rough free-energy landscapes. *Physical Review E*, 82:026704, 2010.

- [47] David J. C. MacKay. *Information Theory, Inference, and Learning Algorithms*. Cambridge University Press, Cambridge, 2003.
- [48] David J. C. MacKay, Graeme Mitchison, and Paul L. McFadden. Sparse-graph codes for quantum error correction. *IEEE Transactions on Information Theory*, 50(10):2315–2330, 2004.
- [49] Nina Mazyavkina, Sergey Sviridov, Sergei Ivanov, and Evgeny Burnaev. Reinforcement learning for combinatorial optimization: A survey. *Computers & Operations Research*, 134: 105400, 2021.
- [50] Marc Mézard and Andrea Montanari. *Information, Physics, and Computation*. Oxford University Press, Oxford, 2009.
- [51] Bui Quang Minh, Heiko A. Schmidt, Olga Chernomor, Dominik Schrempf, Michael D. Woodhams, Arndt von Haeseler, and Robert Lanfear. IQ-TREE 2: New models and efficient methods for phylogenetic inference in the genomic era. *Molecular Biology and Evolution*, 37(5):1530–1534, 2020.
- [52] Elchanan Mossel and Sébastien Roch. Learning nonsingular phylogenies and hidden Markov models. *The Annals of Applied Probability*, 16(2):583–614, 2006.
- [53] Kevin P. Murphy. *Probabilistic Machine Learning: Advanced Topics*. MIT Press, 2023.
- [54] Eliya Nachmani, Elad Marciano, Loren Lugosch, Warren J. Gross, David Burshtein, and Yair Be’ery. Deep learning methods for improved decoding of linear codes. *IEEE Journal of Selected Topics in Signal Processing*, 12(1):119–131, 2018.
- [55] Radford M. Neal. MCMC using Hamiltonian dynamics. In Steve Brooks, Andrew Gelman, Galin L. Jones, and Xiao-Li Meng, editors, *Handbook of Markov Chain Monte Carlo*, pages 113–162. Chapman and Hall/CRC, 2011.
- [56] M. E. J. Newman and G. T. Barkema. *Monte Carlo Methods in Statistical Physics*. Oxford University Press, Oxford, 1999.
- [57] Jorge Nocedal and Stephen J. Wright. *Numerical Optimization*. Springer, 2nd edition, 2006.
- [58] Christos H. Papadimitriou and Kenneth Steiglitz. *Combinatorial Optimization: Algorithms and Complexity*. Dover, 1998.
- [59] Max B. Paulus, Dami Choi, Daniel Tarlow, Andreas Krause, and Chris J. Maddison. Gradient estimation with stochastic softmax tricks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.
- [60] Judea Pearl. *Probabilistic Reasoning in Intelligent Systems: Networks of Plausible Inference*. Morgan Kaufmann, San Mateo, California, 1988.
- [61] Tom Richardson and Rüdiger Urbanke. *Modern Coding Theory*. Cambridge University Press, Cambridge, 2008.
- [62] Kenneth H. Rosen. *Discrete Mathematics and Its Applications*. McGraw-Hill, 8th edition, 2019.
- [63] Naruya Saitou and Masatoshi Nei. The neighbor-joining method: A new method for reconstructing phylogenetic trees. *Molecular Biology and Evolution*, 4(4):406–425, 1987.
- [64] David Sankoff. Minimal mutation trees of sequences. *SIAM Journal on Applied Mathematics*, 28(1):35–42, 1975.

- [65] Ulrich Schollwöck. The density-matrix renormalization group in the age of matrix product states. *Annals of Physics*, 326(1):96–192, 2011.
- [66] Julian Schrittwieser, Ioannis Antonoglou, Thomas Hubert, Karen Simonyan, Laurent Sifre, Simon Schmitt, Arthur Guez, Edward Lockhart, Demis Hassabis, Thore Graepel, Timothy Lillicrap, and David Silver. Mastering Atari, Go, chess and shogi by planning with a learned model. *Nature*, 588:604–609, 2020.
- [67] Martin J. A. Schuetz, J. Kyle Brubaker, and Helmut G. Katzgraber. Combinatorial optimization with physics-inspired graph neural networks. *Nature Machine Intelligence*, 4: 367–377, 2022.
- [68] John Schulman, Philipp Moritz, Sergey Levine, Michael I. Jordan, and Pieter Abbeel. High-dimensional continuous control using generalized advantage estimation. In *International Conference on Learning Representations*, 2016.
- [69] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.
- [70] David Silver, Julian Schrittwieser, Karen Simonyan, Ioannis Antonoglou, Aja Huang, Arthur Guez, Thomas Hubert, Lucas Baker, Matthew Lai, Adrian Bolton, Yutian Chen, Timothy Lillicrap, Fan Hui, Laurent Sifre, George van den Driessche, Thore Graepel, and Demis Hassabis. Mastering the game of go without human knowledge. *Nature*, 550:354–359, 2017.
- [71] David Speyer and Bernd Sturmfels. The tropical Grassmannian. *Advances in Geometry*, 4 (3):389–411, 2004.
- [72] Mike Steel. Recovering a tree from the leaf colourations it generates under a Markov model. *Applied Mathematics Letters*, 7(2):19–23, 1994.
- [73] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction*. MIT Press, Cambridge, Massachusetts, 2nd edition, 2018.
- [74] Richard S. Sutton, Doina Precup, and Satinder Singh. Between MDPs and semi-MDPs: A framework for temporal abstraction in reinforcement learning. *Artificial Intelligence*, 112 (1–2):181–211, 1999.
- [75] Robert H. Swendsen and Jian-Sheng Wang. Replica Monte Carlo simulation of spin-glasses. *Physical Review Letters*, 57(21):2607–2609, 1986.
- [76] Robert H. Swendsen and Jian-Sheng Wang. Nonuniversal critical dynamics in monte carlo simulations. *Physical Review Letters*, 58(2):86–88, 1987.
- [77] Ido Tal and Alexander Vardy. List decoding of polar codes. *IEEE Transactions on Information Theory*, 61(5):2213–2226, 2015.
- [78] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. A new class of upper bounds on the log partition function. *IEEE Transactions on Information Theory*, 51(7): 2313–2335, 2005.
- [79] Martin J. Wainwright, Tommi S. Jaakkola, and Alan S. Willsky. MAP estimation via agreement on trees: Message-passing and linear programming. *IEEE Transactions on Information Theory*, 51(11):3697–3717, 2005.
- [80] Chris Whidden and Frederick A. Matsen. Quantifying MCMC exploration of phylogenetic tree space. *Systematic Biology*, 64(3):472–491, 2015.

- [81] Ulli Wolff. Collective monte carlo updating for spin systems. *Physical Review Letters*, 62(4):361–364, 1989.
- [82] Ziheng Yang. *Molecular Evolution: A Statistical Approach*. Oxford University Press, 2014.
- [83] Jonathan S. Yedidia, William T. Freeman, and Yair Weiss. Constructing free-energy approximations and generalized belief propagation algorithms. *IEEE Transactions on Information Theory*, 51(7):2282–2312, 2005.
- [84] Cheng Zhang and Frederick A. Matsen. Generalizing tree probability estimation via Bayesian networks. In *Advances in Neural Information Processing Systems*, volume 31, 2018.
- [85] Cheng Zhang and Frederick A. Matsen. Variational Bayesian phylogenetic inference. In *International Conference on Learning Representations*, 2019.

A Continuous Sampling: Hamiltonian Monte Carlo

The sampler every interval that is not a delta-method interval comes from, and the one Appendix B runs replicas of. Stated here in full — the integrator, the acceptance, the warm-up that sets both, and what each is held to — because a section of the main text cites it rather than deriving it.

A.1 The sampler

Where the delta-method interval is an approximation, the posterior can be sampled instead and an interval read as a quantile. Hamiltonian Monte Carlo [55] reads $\mathcal{L}$ as a negative log density, adds a momentum $p \sim \mathcal{N}(0, M)$, and proposes by integrating the Hamiltonian $H(\theta, p) = \mathcal{L}(\theta) + \frac{1}{2}p^\top M^{-1}p$ with a symplectic integrator for L steps of size ε , accepting with probability $\alpha = \min\{1, \exp(-\Delta H)\}$. The integrator is reversible and volume-preserving, so ΔH is the whole of the acceptance ratio, and exact on the flow it conserves, so ΔH is bounded rather than growing with L . A temperature enters as the momentum’s variance, and a diagonal mass $M = \text{diag}(1/\sigma_i^2)$ as a change of coordinates, $\phi_i = \theta_i/\sigma_i$: the unit-mass dynamics on $\mathcal{L}(\sigma \odot \phi)$ are the mass- M dynamics on θ term for term, so one integrator serves every metric and every temperature.

A.2 The integrator: leapfrog against Yoshida

Every integrator here is a composition of the second-order kick–drift–kick step, differing only in the sub-step lengths: a momentum half-kick, a position drift, a second half-kick, with the adjacent half-kicks of consecutive sub-steps merging into one. The leapfrog method is the trivial composition, one sub-step of full length, and costs one gradient per step. Yoshida’s fourth-order method is the palindromic triple jump (w_1, w_0, w_1) with

$$w_1 = \frac{1}{2 - 2^{1/3}}, \quad w_0 = -\frac{2^{1/3}}{2 - 2^{1/3}}, \quad 2w_1 + w_0 = 1, \quad (83)$$

and costs three. Both are reversible — the weights are a palindrome — and both are volume-preserving, which is what makes $\Delta\mathcal{H}$ the whole of the acceptance ratio for either; the order of accuracy changes the cost of a draw and never the distribution drawn from. That $w_0 < 0$ is not a sign error and is not avoidable: no symmetric composition of a second-order method reaches fourth order with positive coefficients, so the middle sub-step runs backwards in time and a plotted trajectory is non-monotone.

Whether the higher order pays is a measurement at equal *gradient evaluations*, and on these targets it does not. A fourth-order method pays where the step is limited by accuracy; here it is limited by *stability*. Yoshida’s largest sub-step is $|w_0| = 1.70$ times the nominal step, so its stability limit in ε is a factor of about 0.6 of leapfrog’s, and three gradients per step multiply that. On the Potts posterior the regression suite pins the consequence: leapfrog reaches an acceptance of 0.7 within 21 gradients per trajectory, where Yoshida needs at least 60 and at 61 accepts nothing. Leapfrog is therefore the default, and the composition is carried as an object with declared weights, order and cost so that the comparison is at equal evaluations rather than at equal steps.

A.3 Adaptation, as a warm-up

The step ε right for one posterior is wrong for the next, and the coordinates’ scales σ_i are the posterior’s. Both are set in a warm-up whose draws are discarded, after which the chain runs at fixed values and has the target as its stationary distribution; a chain that kept adapting would not. The scales are the sample standard deviations of a warm-up window. The step is driven to a target acceptance δ by dual averaging [37, Sec. 3.2]: after proposal m with acceptance probability α_m ,

$$\begin{aligned}\bar{h}_m &= \left(1 - \frac{1}{m + t_0}\right) \bar{h}_{m-1} + \frac{\delta - \alpha_m}{m + t_0}, \\ \ln \varepsilon_m &= \mu - \frac{\sqrt{m}}{\gamma} \bar{h}_m, \\ \ln \bar{\varepsilon}_m &= m^{-\kappa} \ln \varepsilon_m + (1 - m^{-\kappa}) \ln \bar{\varepsilon}_{m-1},\end{aligned}\tag{84}$$

with $\mu = \log(10\varepsilon_0)$ and the constants γ, t_0, κ setting the rate; the iterates ε_m do not converge, the averaged $\bar{\varepsilon}_m$ does, and the chain runs at the last of those. The iteration runs twice, once at unit mass to estimate the scales and once on the metric they define, since rescaling the coordinates changes the step right for them.

A.4 The regime the adaptation sits in

On a target that is locally quadratic the leapfrog energy error is bounded and oscillatory, so $\alpha(\varepsilon)$ stays near one up to the stability limit $\varepsilon \omega_{\max} < 2$ on the stiffest direction and falls to zero past it: a target of $\delta = 0.65$ lies *on* that cliff, and an iteration driven by a single Metropolis probability per proposal oscillates across it and averages to a step whose acceptance is not δ . Two remedies, both measured rather than assumed: each proposal’s step is drawn uniformly from $\bar{\varepsilon}(1 \pm j)$ [55, Sec. 5.4.2.2], which averages α over the band and turns the cliff into a slope with one root; and γ is set larger than the value published for a trajectory-averaged statistic, since the iterates’ spread in $\log \varepsilon$ scales as $1/(\gamma\sqrt{m})$. The chain after warm-up keeps the jitter, the step its acceptance was adapted for, and every draw is still from a kernel with the right stationary distribution.

A.5 What a warm-up buys

The warm-up is not free — its draws are discarded — so what it buys is measured on a target whose answer is known. Figure 23 runs one adapted chain and one at fixed step and unit mass on the same correlated Gaussian, from the same start and at the same starting step. That Gaussian’s two directions differ in width by a factor of 534, so at unit mass one step size must serve both and no value serves either well: above the narrow direction’s stability limit the chain diverges, and below it the wide direction is crossed by a random walk. The adapted chain’s running mean reaches the last tenth of a standard deviation at draw 211 and the fixed chain’s at 323, and the separation that matters is the cost of a draw — 2.08×10^{-2} effective draws

per gradient against 5.85×10^{-3} , a factor of 3.6, with the warm-up’s own gradients charged to the chain that spent them. Both chains are correct: each is a kernel with the right stationary distribution, and each recovers the exact mean inside four standard errors at its own effective sample size. So what separates them is cost per effective draw and nothing else, which is the only claim a warm-up can make.

A.6 What pins it

Equation 84 is arithmetic and is stepped by hand against the implementation; the mass matrix is a change of coordinates and is held to a mass-matrix leapfrog written out, to round-off. What a draw costs is the effective sample size per gradient evaluation, with the size estimated by the initial positive sequence [31] and that estimator held to an autoregressive chain whose integrated autocorrelation time is a closed form. Three statistics follow: the adapted chain’s acceptance lands within a stated distance of δ over independent seeds; its posterior means and spreads agree with a fixed-parameter chain’s within Monte Carlo error, on the analytic Gaussian and on a four-taxon tree; and the adapted ladder’s exchanges lie inside the band on the frustrated lattice, with its ground-state hit rate at equal sweeps reported beside the hand ladder’s. The fixed-parameter samplers are unchanged and remain the oracles.

B Tempering

A temperature is a knob on the same model, and three methods turn it: a schedule that lowers it, an ensemble that holds several at once, and a limit that removes it. They are collected here because every problem with a discrete search cites one of them, and because the comparison that decides between them is one comparison, stated at the end.

B.1 Schedules, annealing and replica exchange

A temperature T scales the model, $\mathcal{H}/T$, so a chain at T targets $\exp(-\mathcal{H}/T)$; a *schedule* $T(t)$ over a budget of t steps is one object, shared by every consumer of it, with both endpoints reached exactly at the declared steps and a step past the end refused. Simulated annealing is the sampler on a decreasing schedule, returning the lowest-energy state seen [40]; at $T \rightarrow 0$ the heat bath is the argmin over each site’s conditional, which is iterated conditional modes, so annealing and greedy descent are one search separated by the schedule alone.

Parallel tempering runs R replicas at fixed temperatures $T_1 < \dots < T_R$ and, after each sweep, proposes to exchange the configurations of adjacent replicas, accepting with

$$a_{ij} = \min \{1, \exp[(\beta_i - \beta_j)(E_i - E_j)]\}, \quad (85)$$

the ratio of the joint target, a product of tempered marginals [75, 30, 22]; Appendix E.9 derives it. The hot replicas cross barriers the cold one cannot, and an exchange carries what they find down the ladder. Each replica draws from its own generator, spawned from one parent that draws only the exchange uniforms, so no two replicas share a stream and one seed reproduces the run.

The replica at $T_1 = 1$ targets the unscaled model, so the fraction of its N recorded sweeps spent at a structure x estimates that structure’s weight over the whole space,

$$\hat{w}(x) = \frac{1}{N} \sum_{n=1}^N [\sigma_1^{(n)} = x] \rightarrow \frac{\exp(-\mathcal{H}(x))}{Z}, \quad (86)$$

where a weight over a neighbourhood is conditional on that neighbourhood and enumeration is refused past its cap. The structure may be a labelling, a hidden path, or a topology with $\mathcal{H}$ the

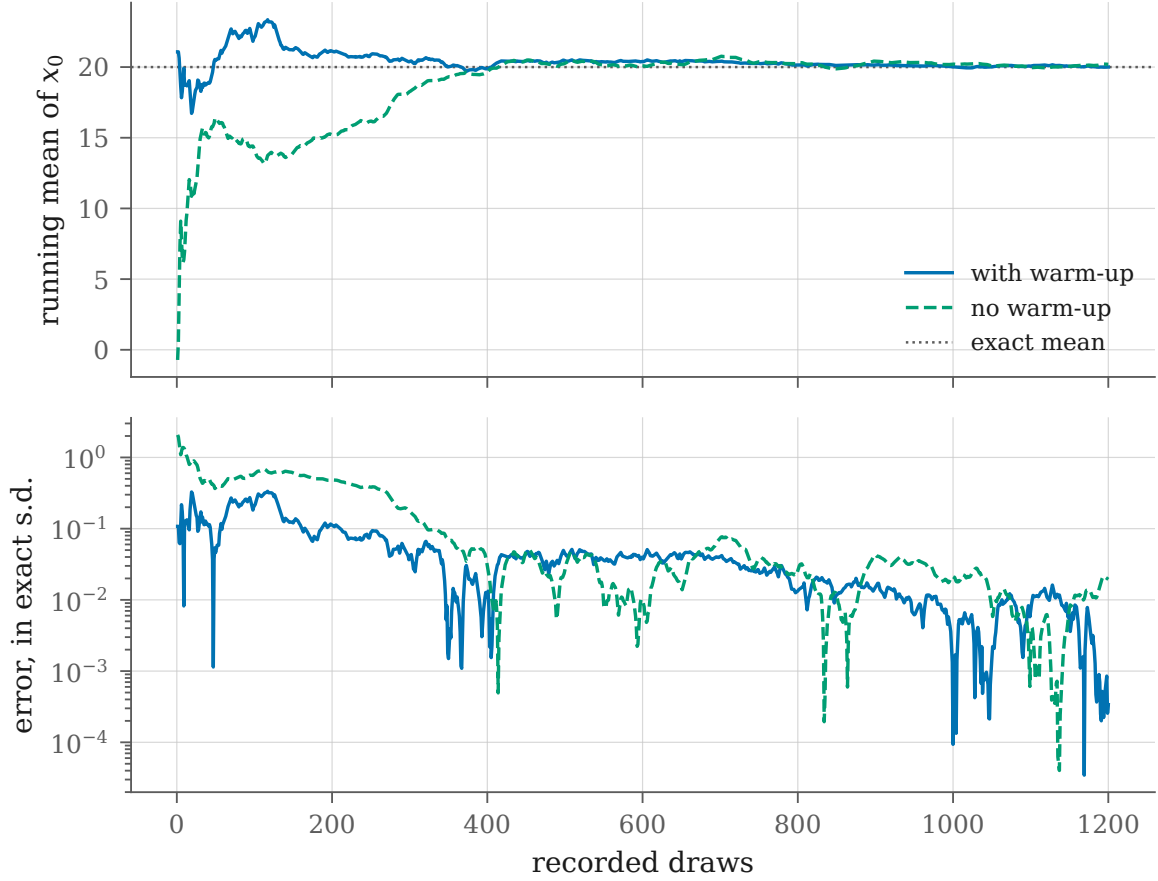


Figure 23: What a warm-up buys, on the target whose answer is closed form. Two chains of 1200 draws at 12 leapfrog steps run from the same start on a 2-dimensional Gaussian of mean $(20.0, -2.0)$ and covariance $((100.0, 2.5), (2.5, 0.25))$, whose eigenvalues differ by a factor of 534. One adapts its step size and mass diagonal over 400 discarded proposals and then runs at fixed values; the other keeps the starting step of 0.6 and unit mass throughout. Upper: the running mean of the wider coordinate, the one a step small enough to be stable on the narrower direction crosses by a random walk, against its exact value of 20; the start sits two of its standard deviations away. Lower: the same error in units of that coordinate's exact standard deviation of 10. The running mean reaches the final 0.1 standard deviations and stays there at draw 211 with the warm-up and draw 323 without; at the last draw the error is 0.000 and 0.019. The warm-up settled on a step of 1.273 and a mass diagonal of $(0.014, 3.912)$, accepting 0.63 over the warm-up against a target of 0.65. Both chains are correct: the acceptance is 0.65 adapted and 0.90 fixed, and each is a kernel with the right stationary distribution, so what separates them is cost per effective draw — $2.08\text{e-}02$ effective draws per gradient with the warm-up, its own gradients charged to it, against $5.85\text{e-}03$ without. Seed 20260914.

negative fitted log-likelihood, where the limit is the flat-prior weight over maximized likelihoods and not a marginal over branch lengths. It is a Monte Carlo estimate, named a posterior weight only beside the exchange acceptance of every adjacent pair and the autocorrelation time of the indicator, which say whether the ensemble mixed.

B.2 Setting the ladder from what it measures

The same principle sets a tempering ladder for Section B. What makes a ladder right is the exchange acceptance of Equation 85 between each neighbouring pair—near zero the replicas are independent chains, near one a rung is redundant—so a warm-up measures it on a candidate ladder and revises: a pair below a stated band is bisected at the geometric mean, a rung both of whose pairs are above the band is removed, and a pair above the band beside one inside it has their shared rung moved geometrically halfway toward the far end of the pair inside. Geometric, because the exchange ratio depends on the temperatures through $\beta_i - \beta_j$, so for an energy whose spread the temperature sets, the acceptance is a function of the ratio of neighbours. The endpoints are the caller’s and never move; the warm-up’s cost in sweeps is charged to any comparison against a ladder set by hand.

B.3 Tempering against a sampler, and against restarts

Three ways to spend a budget on a multimodal continuous objective, and they are not variants of one method. *Random restart* (Algorithm 8) fits from many starts and reports every fit with its spread; it assumes nothing, parallelizes without communication, and finds a basin only if a start lands in it, so on a surface whose basins are narrow its success is a rate in the number of restarts. *Hamiltonian Monte Carlo* (Appendix A) follows the gradient of one density and crosses a barrier only by paying its height in the acceptance ratio; it is the method when the question is a *distribution* — an interval, a posterior mean — and it is the wrong instrument for a global minimum, since a chain that has not crossed a barrier reports the basin it started in with no sign that it did not. *Tempering* is the one that attacks the barrier: hot replicas cross what the cold one cannot and an exchange carries what they find down the ladder, at the price of R chains for one answer and a ladder that must itself be set (Section B.2).

What follows is an accounting rule rather than a ranking. The budget is the problem’s own unit — objective evaluations for a fit, sweeps for a chain — and every replica’s sweeps and every restart’s evaluations are charged to it, the warm-up’s and the ladder’s included. Under that rule none of the three dominates, and which wins is a property of the instance: it is measured on the frustrated lattices of Section 11, where the answer is known from outside at every size, and on the multimodal test functions of Section 4, where the minimizers are published. What is asserted in general is only the refusal: a comparison at unequal budgets, or one quoting the best of many restarts against the single answer of one chain, has measured the accounting and not the method.

B.4 What pins it

Every replica’s marginal must still be $\exp(-\mathcal{H}/T_r)$, enumerated from the *unscaled* model, with exchanges on: a wrong exchange ratio contaminates the cold replica with hot configurations and only that test says so. Its power is pinned by the paired negative—an exchange omitting the energy term still runs and still mixes, and is rejected on every replica. Whether tempering or annealing beats restarts of descent at an equal budget of sweeps is a property of the instance, measured on the frustrated, past-enumeration instances the roadmap needs and never asserted in general.

C Reinforcement Learning

A self-contained account of the algorithms a learned proposal (Section 21) is drawn from. It is organised on the two axes that separate them: how an algorithm handles the *state space* — a table over discrete states, or a parameterized function over a continuous one — and how it *updates an estimate* — from a full observed return, or by bootstrapping from its own estimate at the next state. The progression runs from an exact framework that requires a model of the environment to scalable approximations that require only samples from it. Sutton and Barto’s chapters are cited at each step [73].

Two conventions hold throughout, and both are the searches’ rather than the literature’s. The discount is $\gamma = 1$, because the return of Equation 89 telescopes to an improvement in the objective and a discount would prefer improvement found early over the same improvement found late. And every cost is counted in objective evaluations, so an algorithm that reaches a better optimum by evaluating more candidates has not won.

C.1 The decision process, and what is exact in it

A finite Markov decision process is a set of states $\mathcal{S}$, a set of actions $\mathcal{A}(s)$ available at each, a transition law $p(s', r \mid s, a)$ and a policy $\pi(a \mid s)$. The state value is the expected return from s under π , and it satisfies the Bellman equation

$$V^\pi(s) = \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + V^\pi(s')], \quad (87)$$

with the action value $Q^\pi(s, a)$ the same expectation with the first action fixed [73, Ch. 3]. Equation 87 is a linear system in $|\mathcal{S}|$ unknowns, so on a state space small enough to enumerate every quantity below is available exactly — and that is what makes the machinery refereeable here: an environment small enough to enumerate is the oracle for a learning algorithm, as a tree small enough to enumerate is for a search.

The optimal values satisfy the same equation with a maximum in place of the policy average,

$$V^*(s) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + V^*(s')], \quad (88)$$

and a policy greedy in V^* is optimal. Everything that follows is a way of solving Equation 88 with less than a model, or with less than a table.

In the searches of Section 21 the reward is the improvement in the objective, so the undiscounted return telescopes,

$$G = \sum_{t=0}^{T-1} R_t = \sum_{t=0}^{T-1} [\mathcal{L}(s_t) - \mathcal{L}(s_{t+1})] = \mathcal{L}(s_0) - \mathcal{L}(s_T), \quad (89)$$

the improvement between the first and last state whatever path joined them. That licenses an undiscounted return—a discount would prefer improvement found early over the same improvement found late—and it is why a truncation landing between a sacrifice and its payoff teaches the opposite of the truth, so a truncated episode is recorded as one.

C.2 Dynamic programming: policy iteration and value iteration

Dynamic programming assumes the transition law is known and the states are enumerable, and solves Equation 87 by iteration [73, Ch. 4]. *Policy evaluation* sweeps

$$V_{k+1}(s) \leftarrow \sum_a \pi(a \mid s) \sum_{s', r} p(s', r \mid s, a) [r + V_k(s')], \quad (90)$$

a contraction whose fixed point is V^π ; *policy improvement* sets $\pi'(s) = \arg \max_a Q^\pi(s, a)$, which the policy improvement theorem guarantees is no worse at every state. *Policy iteration* alternates the two to an optimal policy in finitely many steps on a finite process. *Value iteration* truncates the evaluation to one sweep, applying Equation 88 as an assignment, and converges to V^* from any start.

Both are *bootstrapped* — an estimate is updated from other estimates — and both are *exact*, in that the expectation is computed rather than sampled. The cost is one pass over every state and action per sweep, which is the reason they do not reach the searches of this document: the state of a topology search is a topology with its fitted parameters, and the transition law is a move set whose reward is a fit. Where an instance *is* small enough — the enumerable lattice environment — value iteration supplies the V^* every approximate method below is scored against, and that is what it is used for here.

C.3 Monte Carlo methods

A Monte Carlo method removes the model. It runs an episode under the policy, observes the return G_t from each visited state, and averages

$$V(s_t) \leftarrow V(s_t) + \alpha [G_t - V(s_t)], \quad (91)$$

which for a running mean with $\alpha = 1/n$ is the sample average of the returns observed from s [73, Ch. 5]. It bootstraps from nothing, so its estimate is unbiased for V^π and its variance is the return's; it needs episodes that terminate, which the searches here have by construction, since an episode ends at a step budget or at a state no move improves.

Two properties matter for what follows. A Monte Carlo estimate at one state is independent of the estimates at the others, so it does not propagate an error — and does not propagate information either, which is why it is slow where states are visited rarely. And control by Monte Carlo needs every action tried from every state, supplied either by exploring starts or by an ϵ -soft policy, the same requirement the exploring wrapper of Section 21 meets and for the same reason.

C.4 Temporal-difference learning

Temporal-difference learning takes the bootstrapping of dynamic programming and the sampling of Monte Carlo. One step of experience (s_t, a_t, r_t, s_{t+1}) gives the error and the update

$$\delta_t = r_t + V(s_{t+1}) - V(s_t), \quad V(s_t) \leftarrow V(s_t) + \alpha \delta_t, \quad (92)$$

which is TD(0) [73, Ch. 6]. The target $r_t + V(s_{t+1})$ is biased by the current estimate and has the variance of one transition rather than of a whole episode: the trade that runs through every method here.

Control splits on *whose* action the target reads. *Sarsa* is on-policy, reading the action the behaviour policy actually took next,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + Q(s_{t+1}, a_{t+1}) - Q(s_t, a_t)], \quad (93)$$

so it evaluates the policy it is following, exploration included. *Q-learning* is off-policy, reading the best action available instead,

$$Q(s_t, a_t) \leftarrow Q(s_t, a_t) + \alpha [r_t + \max_a Q(s_{t+1}, a) - Q(s_t, a_t)], \quad (94)$$

so it learns the greedy policy's values while behaving otherwise. The difference is not cosmetic for a search that must take a worsening move to leave a local optimum: Sarsa's values account for the cost of exploring and Q-learning's do not, and which is wanted depends on whether the exploration is part of the method or a means of collecting data.

The n -step returns interpolate between Equation 92 and Equation 91, and the eligibility trace of Section C.5 is the same interpolation run online [73, Ch. 7].

C.5 Value function approximation

A table is impossible where the states are a continuum, and pointless where they are astronomically many and each is visited once — the regime of every search in this document, whose states are structures. The value is then parameterized, $\hat{V}(s; w)$, and the update becomes a gradient step [73, Ch. 9]. *Gradient Monte Carlo* descends the squared error to the observed return,

$$w \leftarrow w + \alpha [G_t - \hat{V}(s_t; w)] \nabla_w \hat{V}(s_t; w), \quad (95)$$

which is stochastic gradient descent on an unbiased target and converges to a local optimum of the mean squared value error. *Semi-gradient TD* replaces the return by the bootstrapped target,

$$w \leftarrow w + \alpha [r_t + \hat{V}(s_{t+1}; w) - \hat{V}(s_t; w)] \nabla_w \hat{V}(s_t; w), \quad (96)$$

and is *semi*-gradient because the target’s own dependence on w is not differentiated: it is not the gradient of any objective, which is why its convergence holds for linear approximation on-policy and can diverge off-policy with a function approximator and bootstrapping together — the deadly triad [73, Ch. 11]. *Eligibility traces* carry a decaying record of the features recently visited,

$$z_t = \lambda z_{t-1} + \nabla_w \hat{V}(s_t; w), \quad w \leftarrow w + \alpha \delta_t z_t, \quad (97)$$

so one error updates every state that led to it; $\lambda = 0$ is Equation 96 and $\lambda = 1$ is Equation 95, and the parameter moves between the two continuously [73, Ch. 12]. The generalized advantage of Equation 99 is the same interpolation written for an advantage rather than a value.

Two consequences bind what a state’s features may be. Generalization is the point — experience at one structure must inform a structure like it — so the features are what makes two states alike, and a feature that is constant over a neighbourhood contributes nothing to a softmax over that neighbourhood and is excluded (Section 21). And a parameterized value is scored against V^π where enumeration supplies it, never against its own training loss, which it can minimize while estimating the wrong function.

C.6 Policy gradient

The methods above derive a policy from values. A policy gradient parameterizes the policy and optimizes it directly, which is what a search over a neighbourhood of varying size needs: a softmax over the scores of the available actions admits a neighbourhood of any size, where an $\arg \max$ over a fixed action set does not. With $J(\pi)$ the expected return, the policy gradient theorem gives an expectation over trajectories the policy itself generates [73, Ch. 13], and REINFORCE is its Monte Carlo estimator,

$$\nabla J = \mathbb{E} \left[\sum_t \nabla \log \pi(a_t | s_t) (G_t - b(s_t)) \right], \quad G_t = \sum_{u \geq t} R_u, \quad (98)$$

where a baseline b independent of the sample it is subtracted from leaves the estimator unbiased and reduces its variance, not its bias: G_t is a sum of objective differences whose scale is the problem’s, so an uncentred gradient varies by orders of magnitude between instances. The comparison against the greedy baseline is at a matched budget of objective evaluations per decision, never in wall-clock time. so the update is Equation 95’s trade in a second place: unbiased, and of the return’s variance. A baseline $b(s)$ depending on the state alone leaves the estimator unbiased — the proof is Appendix C.10 — and the natural choice is the state value, which turns REINFORCE into the actor–critic below.

C.7 Actor–Critic and Proximal Policy Optimization

The baseline in (98) may depend on the state. The natural choice is the state value $V^\pi(s) = \mathbb{E}[G_t \mid s_t = s]$, the improvement the search still expects from s , estimated by a *critic* fitted to sampled returns; the estimator is then weighted by the *advantage* $A_t = G_t - V(s_t)$, unbiased for any critic and of least variance in this family when the critic is exact (Appendix C). Where the return is exact by enumeration, so is V^π , and a critic is scored against it rather than against its own loss. The Monte Carlo return and the one-step bootstrap $R_t + V(s_{t+1})$ are the two ends of the *generalized advantage* [68], at $\gamma = 1$,

$$A_t^{(\lambda)} = \sum_{l \geq 0} \lambda^l \delta_{t+l}, \quad \delta_t = R_t + V(s_{t+1}) - V(s_t), \quad (99)$$

with $V(s_T) = 0$ at a terminal state and the critic’s value at a truncated one; $\lambda = 1$ recovers $G_t - V(s_t)$ and $\lambda = 0$ the temporal difference, trading the critic’s bias against the return’s variance.

The score-function estimator is valid only for the policy that collected the episodes. *Proximal policy optimization* [69] reuses a batch for several gradient steps by importance-weighting each decision with $\rho_t = \pi(a_t \mid s_t) / \pi_{\text{old}}(a_t \mid s_t)$ and clipping the ratio’s effect on the objective,

$$\mathcal{L}_{\text{PPO}} = \mathbb{E} \left[\sum_t \min(\rho_t A_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) A_t) \right], \quad (100)$$

so an update cannot move the policy far from the one the advantages were estimated under. At the collecting policy every ratio is one and, without clipping, the gradient of (100) is the actor–critic’s; the regression suite pins that identity and measures on the Potts chain what the reuse buys: the fraction of the 81 starts from which the trained policy reaches the enumerated optimum, at a matched number of episodes. The same ratio makes the update valid when a *behaviour* policy collected the episodes: with π_{old} replaced by β , the ϵ -greedy wrapper of the policy being trained, the search can leave a local optimum the softmax would settle into, and the estimator is corrected by π/β .

What the critic reads, and what it may not. V is a function of the state, while the decision process supplies features of *actions*. The state’s features are therefore aggregates over its neighbourhood — the mean and the maximum of the action features, and the logarithm of the neighbourhood’s size — covering every instance without widening the interface, a terminal state having no actions and so the zero vector. Appendix C carries the rule that makes the baseline legitimate: the critic may read the state and never the action sampled at that step. Where the return is exact by enumeration V^π is exact too, and the critic is scored against *that*; a critic scored against its own training loss can fit its targets, miss V^π , and leave an estimator unbiased and useless.

Off-policy, and what the ratio costs. Equation 100 is one case of a general correction. For any behaviour policy β with $\beta(a \mid s) > 0$ wherever $\pi(a \mid s) > 0$, weighting each decision by $\rho_t = \pi(a_t \mid s_t) / \beta(a_t \mid s_t)$ makes the estimator of Equation 98 unbiased for π on episodes that β collected [73]. The support condition is not a technicality: an action β never samples contributes nothing, so a *deterministic* behaviour policy corrects nothing. That is the second reason for the exploring wrapper’s uniform draw, and why its mixture must be available as a distribution and not only as a sampler — $(1 - \epsilon)$ on the best action and $\epsilon/|\mathcal{N}(s)|$ everywhere is what the ratio divides by. That ϵ is the exploration rate; the one in Equation 100 is a clip width, and the two are unrelated.

The correction costs variance, and the cost has a shape. A ratio is unbounded above, so one decision the behaviour policy made rarely can dominate a batch; a weight formed as the *product*

of ratios along a trajectory is worse, its variance growing with the episode length. Two answers are carried, and they are not the same. The per-decision form of Equation 100 weights each step by its own ratio rather than the trajectory’s product, which removes the growth with episode length. The clip bounds the ratio’s effect on the objective, and so the size of an update, while leaving the estimator’s variance where it was: a trust region and not a variance reduction, and reading it as the second is the standing error about it.

C.8 Planning with a Learned Prior and Value

A policy answers “which move” from what it has seen; a planner looks ahead. From the current state, each of n simulations descends a tree of successors choosing, at every expanded node, the child that maximizes

$$Q(s, a) + c \pi(a | s) \frac{\sqrt{N(s)}}{1 + N(s, a)}, \quad (101)$$

the mean return backed up through that child plus an exploration bonus that is the prior the policy gives the move, decaying with the child’s visits [70]; the first unexpanded child is scored by the critic and the return backed up to the root. The root’s visit counts, normalized, are a policy that improves on the prior, and *expert iteration* [4] trains the policy toward that distribution by cross-entropy and the critic toward the achieved return, so the planner’s answers become the policy’s. Every expansion evaluates a successor, so the search is counted in the unit the rest of this document counts, and the comparison with greedy states the rate and the evaluations together (Appendix C).

Planning with a learned model. The planner above expands the *true* successor and reads the true reward, affordable only because a move’s reward is a closed form at known parameters. It stops being affordable the moment the reward is a fit per candidate, the regime every problem here is in. The answer is then to plan on a learned model: a transition and a reward in a latent space, trained so that the values, policies and rewards the plan reads agree with the observed ones, and not so that the latent state reconstructs the real one [66]. Requiring only the quantities the search reads to be right makes it a method rather than a reconstruction, and makes it checkable here: on an instance small enough to enumerate, a planner on a learned model must return an optimal action wherever the planner on the true one does. It is not built.

C.9 What applies to the problems of this document, and what does not

The axes decide it. Every search here has a state space no table reaches — a topology, a labelling, a configuration — so the tabular methods of Sections C.2 to C.4 appear only as oracles on an enumerable instance, where they supply the V^* and the exact policy gradient that referee an approximate method. The methods that *run* are the approximating ones, and three properties of these problems choose among them.

The reward is a difference in the objective, so the return telescopes and is available exactly at the end of an episode: a Monte Carlo return costs nothing that the search has not already paid, which is why REINFORCE with a baseline is the reference estimator here and bootstrapping is an optimization rather than a necessity. The action set is a neighbourhood whose size varies with the instance, which excludes every method whose output is indexed by a fixed action set — Q-learning over a table of (s, a) pairs among them — and admits the softmax policy and the critic over aggregate state features. And an evaluation is expensive, which makes sample efficiency the whole of the comparison: a method is measured against greedy search with restarts at a matched budget of objective evaluations, never at matched wall-clock time and never against a single greedy run, which stops early and leaves the budget unspent.

What that leaves is a ladder of its own. The linear scorer is the reference, because a recovery test can state a truth about its weights; the multilayer-perceptron scorer is measured against

it and never assumed better; the critic is added where the return’s variance is what limits the estimator, and scored against the enumerated V^π ; and the planner of Equation 101 is admissible only where a successor can be evaluated cheaply enough to expand many of them, which on these problems means a learned model (Section C.8) and is not built.

C.10 Derivations

Any state-dependent baseline leaves the estimator unbiased. For a function b of the state alone, $\mathbb{E}_{a \sim \pi(\cdot|s)}[\nabla \log \pi(a|s) b(s)] = b(s) \sum_a \nabla \pi(a|s) = b(s) \nabla \sum_a \pi(a|s) = 0$, so subtracting $b(s_t)$ from G_t in (98) changes no expectation. The variance of the per-step term $\nabla \log \pi \cdot (G_t - b)$ is minimized over constant b by a weighted mean of G_t , and over state-dependent b the value $V^\pi(s_t)$ is the choice that removes the part of G_t predictable from the state; a fitted critic approximates it, and its error costs variance, never bias, as long as the fit does not read the action sampled at that step [73].

The generalized advantage telescopes to its two limits. With $\delta_t = R_t + V(s_{t+1}) - V(s_t)$ and $V(s_T) = 0$, $\sum_{l=0}^{T-t-1} \delta_{t+l} = \sum_{u \geq t} R_u - V(s_t) = G_t - V(s_t)$, which is (99) at $\lambda = 1$; at $\lambda = 0$ only δ_t survives. The exponential weights interpolate the two, and the bias of $A_t^{(\lambda)}$ as an estimate of the true advantage is $O(\lambda^k)$ in the critic’s error at k steps ahead [68].

PUCT as policy improvement. With exact leaf values and one decision of depth, every backed-up value in (101) is $Q^*(s, a)$ exactly, and as $N(s) \rightarrow \infty$ the bonus of every action vanishes relative to the gap between the best mean and the rest, so the most visited action is an argmax of Q^* . At depth the means are of returns under the search’s own descents, which concentrate on the children it rates highest; the normalized visit counts are then a distribution whose one-step expected value under Q^π is at least the prior’s, which is the policy-improvement step of policy iteration with the prior as π and the visit distribution as the improved policy [73, 70]. The regression suite checks both on the Potts chain: the argmax exactly at depth one, and the improvement on 13 of 14 sampled states at depth three, the exception being a state whose prior already puts most of its mass on an optimal move.

The clipped objective at the collecting policy. With $\pi = \pi_{\text{old}}$, $\rho_t = 1$ for every decision, so both arguments of the minimum in (100) equal A_t and $\nabla \rho_t = \nabla \log \pi(a_t | s_t)$; the gradient of (100) is $\sum_t \nabla \log \pi(a_t | s_t) A_t$, the actor–critic’s estimator. Away from the collecting policy the minimum removes the gradient wherever the ratio has left $[1 - \epsilon, 1 + \epsilon]$ in the direction the advantage would push it, which is the pessimistic bound on the surrogate improvement that licenses the reuse of a batch [69].

D Interval Estimates

An interval is the part of a fit a reader acts on, and two kinds are reported here. Which applies is decided by the estimand and not by preference: a continuous parameter gets an interval from the curvature of the likelihood at the optimum or from a sampled posterior, and a rate — a coverage, a recovery fraction, a block error rate — gets an interval for a binomial proportion. Reporting the first where the second applies is the standing error, because a proportion at zero or one has no curvature to read.

D.1 A continuous parameter: the observed information

At a maximum $\hat{\theta}$ of the log-likelihood, the observed information $\mathcal{I} = -\nabla^2 \log p(\mathbf{D} | \hat{\theta})$ gives the asymptotic covariance $\Sigma = \mathcal{I}^{-1}$ of $\hat{\theta}$. A constrained parameter $\phi(\theta)$ is a smooth function

of θ , so to first order $\text{Var}(\phi(\hat{\theta})) \approx J \Sigma J^\top$ with $J = \partial \phi / \partial \theta$ at $\hat{\theta}$. The statement is asymptotic in the sample size and requires $\mathcal{I}$ to be invertible; a singular or ill-conditioned information is a parameter the data does not identify, and the interval is refused rather than reported (Section 34).

Where the asymptotics are what is in doubt, the posterior is sampled instead (Appendix A) and the interval read as a quantile of the draws; the two are compared on the analytic Gaussian, where both are known, and their disagreement is the measurement that says which regime a fit is in.

D.2 A rate: an interval for a binomial proportion

A coverage over R replicates, the fraction of starts reaching the global optimum, and a block error rate over R transmitted words are all one estimand: a proportion $\hat{p} = r/R$ from independent trials. The normal interval $\hat{p} \pm z \sqrt{\hat{p}(1-\hat{p})/R}$ is the delta method applied to a binomial and fails exactly where these quantities live — at $\hat{p} = 0$ or 1 it has zero width, which is a claim of certainty from a finite sample. Two intervals are used instead. The Wilson interval inverts the score test,

$$\frac{\hat{p} + \frac{z^2}{2R}}{1 + \frac{z^2}{R}} \pm \frac{z}{1 + \frac{z^2}{R}} \sqrt{\frac{\hat{p}(1-\hat{p})}{R} + \frac{z^2}{4R^2}}, \quad (102)$$

which is non-degenerate at both ends and is what a reported rate carries. The Clopper–Pearson interval inverts the exact binomial test instead, is conservative by construction, and is what a *refusal* is stated against: where no failure is observed in R words, the one-sided bound $1 - \alpha^{1/R}$ is the largest rate the observation is consistent with, and it is that number rather than zero that a claim may use.

A rate and its interval are reported with the seed set and the budget behind them [35, 1]; a rate without its trial count is not a measurement.

E Proofs

Each of these is standard; each is here because a claim in the main text depends on it, and the main text cites rather than derives, for the reader the preface names.

E.1 Pruning is the marginalization over internal states

Root $\mathcal{T}$ at ρ and write z_u for the state at node u , so the leaves' states at one site are the data $\mathbf{D}_s$ and the internal states are summed. Along each branch the state evolves by the Markov process of Section 8, and the branches are conditionally independent given the state at their common node, so the joint over every node is $p(z) = \pi_{z_\rho} \prod_{v \neq \rho} \mathbf{P}_{z_{\text{pa}(v)} z_v}(t_v)$, and the site likelihood is its sum over the internal states with the leaves clamped—the summand of Equation 13, with k^{n-2} terms. Define $L_u(i) = p(\mathbf{D}_s^{(u)} | z_u = i)$, the probability of the data at the leaves below u given u 's state. The subtrees hanging from u 's children share no variable and, given z_u , no factor, so the sum over their states factorizes into one sum per child:

$$L_u(i) = \prod_{v \in \text{children}(u)} p(\mathbf{D}_s^{(v)} | z_u = i) = \prod_{v \in \text{children}(u)} \sum_{j \in \mathcal{A}} \mathbf{P}_{ij}(t_v) p(\mathbf{D}_s^{(v)} | z_v = j),$$

the second equality conditioning on z_v and using that the data below v depends on z_u only through z_v . The inner probability is $L_v(j)$, which is Equation 54; at a leaf $L_u(i) = \mathbf{1}[z_u = i]$ because the state is observed; and summing the root state against its prior gives Equation 55. Nothing in the argument fixes the order of the children or the depth of the tree, so the recursion is a post-order traversal in which each node's sum is done once per parent state: $O(nk^2)$ per

site against k^{n-2} terms. Each L_u is linear in each L_v , so rescaling L_u by a positive c_u scales the site likelihood by $\prod_u c_u$ and nothing else, which is why the logarithm of the scaling may be accumulated apart from the recursion and the rescaled recursion is a transformation of this one rather than a second algorithm [24, 25].

E.2 Forward–backward is the same marginalization on a chain

With the factors of Section 7 the joint over the hidden path and the observations is $p(\mathbf{Z}, \mathbf{D}) = \pi_{z_1} \mathcal{E}_{z_1}(y_1) \prod_{t \geq 1} \mathbf{A}_{z_{t-1} z_t} \mathcal{E}_{z_t}(y_t)$, and the evidence sums it over k^T paths. Define $\alpha_t(i) = p(y_1, \dots, y_t, z_t = i)$. Conditioning on z_{t-1} , and using that $y_1, \dots, y_{t-1}$ depend on z_t only through z_{t-1} , splits the sum over the first $t-1$ states from the rest: $\alpha_t(i) = [\sum_j \alpha_{t-1}(j) \mathbf{A}_{ji}] \mathcal{E}_i(y_t)$, with $\alpha_1(i) = \pi_i \mathcal{E}_i(y_1)$, and the evidence is $\sum_i \alpha_T(i)$: Equation 51. Define $\beta_t(i) = p(y_{t+1}, \dots, y_T \mid z_t = i)$, so $\beta_T = 1$; conditioning on z_{t+1} in the same way gives the backward recursion of Equation 52. The two meet at any position: $p(z_t = i, \mathbf{D}) = \alpha_t(i) \beta_t(i)$, because the observations before t and after t are independent given z_t , and dividing by the evidence gives the posterior of Equation 52; the same split one step wider gives the pairwise posterior $p(z_{t-1} = i, z_t = j \mid \mathbf{D}) = \alpha_{t-1}(i) \mathbf{A}_{ij} \mathcal{E}_j(y_t) \beta_t(j) / p(\mathbf{D})$, the sufficient statistic of the expectation step.

The chain is the caterpillar tree of Section 7 rooted at z_1 : each z_t has one child z_{t+1} carrying the branch factor $\mathbf{A}$ and one observed leaf y_t carrying the emission. Appendix E.1 then gives $L_{z_t}(i) = \mathcal{E}_i(y_t) \sum_j \mathbf{A}_{ij} L_{z_{t+1}}(j)$, so $L_{z_t}(i) = \mathcal{E}_i(y_t) \beta_t(i)$ and Equation 55 is $\sum_i \pi_i \mathcal{E}_i(y_1) \beta_1(i) = \sum_i \alpha_1(i) \beta_1(i) = p(\mathbf{D})$: pruning on the chain is the backward pass, and the forward pass is the same recursion run toward the other end with the prior entering as a leaf factor at z_1 . Both passes in the logarithm are the rescaled recursion of Appendix E.1 with the scale taken at every step. Replacing each sum by a maximum leaves every step of the argument intact, since a maximum distributes over a product of non-negative factors as a sum does, and gives Equation 53 [21, 41].

E.3 Sum-product on a tree, and its loopy stationary point

Both derivations above are one fact about a tree-structured factor graph. Take an edge between factor a and variable i and remove it; because the graph is a tree, the component containing a is a subtree $\mathcal{T}_{a \rightarrow i}$ that meets the rest only at x_i . Define the message from a to i as the partition function of that subtree with x_i clamped, $m_{a \rightarrow i}(x_i) = \sum_{x_{\mathcal{T}_{a \rightarrow i} \setminus x_i}} \prod_{b \in \mathcal{T}_{a \rightarrow i}} \psi_b(x_{\partial b})$, and $m_{i \rightarrow a}$ likewise for the subtree on i 's side. The subtrees hanging from a 's other variables $j \in \partial a \setminus i$ are disjoint, so the sum over their variables factorizes into a product of their partition functions, each a function of its own x_j ; and the subtrees hanging from i 's other factors share only x_i , so their product is $m_{i \rightarrow a}(x_i)$. That is Equation 2, with the messages at a leaf variable or leaf factor being 1 and the factor itself. At any variable the whole graph is the union of the subtrees on its factors, so $Z = \sum_{x_i} \prod_{a \in \partial i} m_{a \rightarrow i}(x_i)$ and the marginal is that product normalized; at a factor, $p(x_{\partial a}) \propto \psi_a(x_{\partial a}) \prod_{i \in \partial a} m_{i \rightarrow a}(x_i)$. A message depends only on messages from further from any chosen root, so one leaf-to-root pass computes every message toward it and one root-to-leaf pass every message away, and every marginal follows: two passes, each factor summed once per direction [60, 45, 41]. Pruning is this with a node's state as the variable, a branch matrix as the factor between parent and child, the prior as a factor on the root and a leaf's observation as an indicator factor: $L_u(i)$ is the product of the factor-to-variable messages from below, which is the variable-to-factor message $m_{u \rightarrow \psi_u}$ that u sends up its own branch. Forward–backward is this on the chain. The argument used only that sum distributes over product, so max-product follows by replacing every sum with a maximum, exact on a tree when the maximum is unique.

On a graph with a cycle no edge separates the graph into two subtrees, so the message has no such definition; iterating Equation 2 regardless defines a map on messages, and a fixed point of that map is a stationary point of the Bethe free energy, which Appendix E.6 derives for the

pairwise case of Equation 68 and which reads, for factors of any degree,

$$F_{\text{Bethe}} = \sum_a \sum_{x_{\partial a}} b_a (\log b_a - \log \psi_a) - \sum_i (d_i - 1) \sum_{x_i} b_i \log b_i, \quad (103)$$

with d_i the number of factors on i . On a tree $p(x) = \prod_a b_a / \prod_i b_i^{d_i-1}$ exactly, the beliefs being the true marginals, so Equation 103 is $-\log Z$ and the stationary point is the answer of the two passes; on a loopy graph the factorization fails and the fixed point is the approximation [83, 50].

E.4 The region-based free energy

Equation 103 approximates the entropy by one term per factor and a correction per variable. A *region graph* replaces the factor by any set of regions R — a region being a set of variables together with the factors whose arguments it contains — ordered by inclusion and closed under intersection. Each carries a counting number fixed by requiring that every variable and every factor be counted exactly once,

$$c_R = 1 - \sum_{A \supsetneq R} c_A, \quad \sum_{R \ni v} c_R = 1 \text{ for every variable } v, \quad (104)$$

the first being Möbius inversion over the partial order and the second the condition it enforces. The region-based free energy is then

$$F_K = \sum_R c_R \sum_{x_R} b_R (\log b_R - \log \psi_R), \quad (105)$$

with ψ_R the product of the factors the region owns [83, 50]. Two cases place it. With one region per factor and one per variable, Equation 104 gives $c_i = 1 - d_i$ at a variable of degree d_i and Equation 105 is Equation 103 — the same expression, not an approximation of it, which is what makes the reduction a test rather than a remark. With the unit cells of a square lattice as the largest regions, their pairwise intersections are the edges and the sites sit below those, and the counting numbers come out +1 on a cell, −1 on a shared edge and +1 at the centre site of a 3×3 — read off Equation 104 rather than written down.

A region graph that counts a variable twice returns a number that looks like a free energy and is not one, so the condition is checked on construction and a violation is refused rather than run.

E.5 The evidence gradient is a posterior expectation

Equation 3 is what makes one message pass serve both the marginalization and the fit, so it is derived rather than asserted. Write the unnormalized measure of Equation 1 as $\tilde{p}(x; \theta) = \prod_a \psi_a(x_{\partial a}; \theta)$ and $Z(\theta) = \sum_x \tilde{p}(x; \theta)$. Differentiating under the sum, which is finite here and so needs no further condition,

$$\frac{\partial \ln Z}{\partial \theta} = \frac{1}{Z} \sum_x \frac{\partial \tilde{p}}{\partial \theta} = \sum_x \frac{\tilde{p}(x; \theta)}{Z} \frac{\partial \ln \tilde{p}(x; \theta)}{\partial \theta} = \mathbb{E}_{p(x)} \left[\frac{\partial \ln \tilde{p}(x; \theta)}{\partial \theta} \right], \quad (106)$$

the middle step being $\partial \tilde{p} = \tilde{p} \partial \ln \tilde{p}$. The logarithm turns the product over factors into a sum, and a factor's log-potential depends on x only through $x_{\partial a}$, so each term's expectation is taken under the marginal of that factor's own variables — the belief b_a , which sum-product returns:

$$\mathbb{E}_{p(x)} \left[\sum_a \frac{\partial \ln \psi_a(x_{\partial a}; \theta)}{\partial \theta} \right] = \sum_a \mathbb{E}_{p(x_{\partial a})} \left[\frac{\partial \ln \psi_a(x_{\partial a}; \theta)}{\partial \theta} \right], \quad (107)$$

which is Equation 3. Two consequences are used in the body. The expectation step of an expectation–maximization is the beliefs and nothing further, since the quantity its maximization step differentiates is this same sum; and on a loopy graph the b_a are the Bethe beliefs rather than the marginals, so the gradient is of an approximate evidence and the optimum it reaches is reported as the stationary point of one.

E.6 The Bethe free energy is stationary at a sum-product fixed point

Write the beliefs of a pairwise graph as b_i and b_{ij} and constrain them to be normalized and locally consistent, $\sum_{\sigma_j} b_{ij} = b_i$. Introducing a multiplier $\lambda_{ij}(\sigma_i)$ for each consistency constraint and setting the derivative of Equation 68 to zero gives $b_{ij} \propto \psi_{ij} e^{\lambda_{ij}(\sigma_i) + \lambda_{ji}(\sigma_j)}$ and $b_i \propto \psi_i e^{\sum_{j \in \partial i} \lambda_{ji}(\sigma_i)/(d_i-1)}$; identifying $e^{\lambda_{ji}(\sigma_i)}$ with $\prod_{l \in \partial i \setminus j} m_{l \rightarrow i}(\sigma_i)$ turns the consistency constraint into Equation 67. So a fixed point of belief propagation is a stationary point of the Bethe free energy and conversely; on a tree the Bethe free energy is the exact variational free energy, which is why the fixed point there gives $-\log Z$ [83]. The argument uses only that each b_a is a function of the variables its factor touches, so it holds for factors of any degree, with Equation 103 in place of Equation 68 and $d_i - 1$ counting the factors on i beyond the first (Appendix E.3).

E.7 The region-based free energy is stationary at a parent-to-child fixed point

Constrain the region beliefs of Equation 105 to be normalized and consistent wherever regions overlap: a parent's belief marginalized onto a child must be the child's. Introducing a multiplier for each parent-child constraint and setting the derivative of Equation 105 to zero gives $b_R \propto \psi_R \prod$ (the multipliers of the constraints R takes part in), and identifying each multiplier with a message from the parent to the child turns the consistency constraint into the update the messages are iterated under — the same argument as Appendix E.6, with a region in place of a factor. So a fixed point of parent-to-child message passing is a stationary point of the region-based free energy and conversely [83].

Three consequences are used in the body. At the Bethe region graph the fixed point is belief propagation's, which is how the implementation is refereed — two routes to one fixed point, agreeing to 1.7×10^{-10} on a 3×3 . Stationarity is not optimality: the functional is not convex, so $-F_K$ may sit either side of $\log Z$ and no claim here is a bound. And nothing guarantees a fixed point is *reached*: the map has none of belief propagation's contraction properties, damping slows the approach rather than creating one, and a run that does not settle is refused — a free energy read off messages that never settled estimates nothing.

E.8 Detailed balance for a cluster move in a field

In the Edwards–Sokal representation the joint over spins and bonds is $p(\sigma, \omega) \propto \prod_{(i,j): \omega_{ij}=1} (1 - e^{-\beta J_{ij}}) \delta(\sigma_i, \sigma_j) \prod_{\omega_{ij}=0} e^{-\beta J_{ij}} \prod_i e^{\beta h_i(\sigma_i)}$, whose spin marginal is the Boltzmann distribution of Equation 15. Drawing bonds given spins and then recolouring a bond cluster $\mathcal{C}$ uniformly are both Gibbs steps at zero field, so the move preserves p there. With a field the recolouring is no longer a Gibbs step: the ratio of the target before and after recolouring $\mathcal{C}$ from μ to ν is $\exp(\beta \sum_{i \in \mathcal{C}} [h_i(\nu) - h_i(\mu)])$, the bond configuration being unchanged, and a uniform proposal accepted with the Metropolis probability of Equation 74 satisfies detailed balance with respect to that ratio. Omitting the accept step gives a chain that runs and mixes and converges to the zero-field distribution, which is why the distributional test is run with a field as well as without.

E.9 The exchange ratio in parallel tempering

The replicas' joint target is the product $\prod_r \exp(-\beta_r E(\sigma_r))$. Swapping the configurations of replicas i and j changes it by $\exp(-\beta_i E_j - \beta_j E_i) / \exp(-\beta_i E_i - \beta_j E_j) = \exp[(\beta_i - \beta_j)(E_i - E_j)]$, and the proposal is symmetric, so the Metropolis acceptance is Equation 85. Each replica's marginal under the joint is its own tempered Boltzmann distribution whatever the exchanges do, which is the property the per-replica goodness-of-fit test asserts.

E.10 Density evolution on the erasure channel

On a cycle-free (d_v, d_c) graph the messages arriving at a node are independent, so the erasure probability of a message is a number rather than a distribution and Equation 78 reduces to a scalar recursion. A check-to-bit message is known only if every other incoming bit message is known; a bit-to-check message is erased only if the channel and every other incoming check message are. With x_ℓ the probability that a bit-to-check message is erased after ℓ iterations,

$$x_0 = \epsilon, \quad x_\ell = \epsilon(1 - (1 - x_{\ell-1})^{d_c-1})^{d_v-1}, \quad (108)$$

the recursion Richardson and Urbanke [61, Sec. 3.12] derives and names density evolution. The right-hand side is increasing in $x_{\ell-1}$ and in ϵ , so x_ℓ is non-increasing and its limit is the largest fixed point in $[0, \epsilon]$; the threshold ϵ^* is the largest ϵ for which that fixed point is zero, found by bisection, and for $(3, 6)$ it is 0.4294, with 0.3834 for $(4, 8)$ and 0.5176 for $(3, 5)$. A finite code is not cycle-free, but the neighbourhood of a bit to depth ℓ is a tree with probability tending to one as n grows at fixed ℓ , which is why the recursion is the limit of the flooding decoder and why the code at size is held to bracketing ϵ^* rather than to a per-draw agreement—the rule this repository states for every asymptotic result (Section 34).

F Derivations of the Bounds

The plug-in bound, (41). $\ell^*(\mathcal{T})$ is a maximum over the feasible set $\tau \geq 0$, and $\hat{\tau}$ lies in it by construction: projected gradient descent on the convex least-squares objective clamps every iterate to the non-negative orthant, so the value at $\hat{\tau}$ is one of the values maximized over. Equality holds exactly when $\hat{\tau}$ is a maximizer.

The parsimony bound, (42). Write $a_e = e^{-kt_e/(k-1)} \in [0, 1]$. Each entry of $\mathbf{P}(t_e)$ is affine in a_e and each branch appears exactly once in the product of (54), so the site likelihood is a polynomial of degree at most one in each a_e : multilinear. A multilinear function on a box attains its maximum at a vertex, where each $\mathbf{P}_e$ is either $\mathbf{I}$ (no change permitted) or $\mathbf{J}/k$ (the child's state independent of the parent's, at weight $1/k$). Root the tree at leaf 1, which the pulley principle permits for a reversible model. At a vertex with cut set C (the branches carrying $\mathbf{J}/k$), the site likelihood is a sum over ancestral assignments constant on each component of $\mathcal{T} \setminus C$: $\pi(x_{1,s}) k^{-|C|} k^{\ell'}$, with ℓ' the number of components containing no leaf (each contributes a free sum over k states), provided every leafy component is monochromatic, and zero otherwise. Merging each leafless component into a neighbour across one of its boundary branches removes one cut and one leafless component at a time and leaves a valid partition with $|C| - \ell'$ cuts, all of whose components are leafy and monochromatic; assigning each its leaf state gives an ancestral labelling with at most $|C| - \ell'$ changes, so $F_s \leq |C| - \ell'$ by minimality of the Fitch score. Hence every vertex value is at most $\pi(x_{1,s}) k^{-F_s}$, and so is the maximum over the box. Summing over sites gives (42); the regression suite enumerates the $2^{|E|}$ vertices at five taxa and finds the bound attained on every site. The entrywise limit $\sup_t \mathbf{P}_{ij}(t)$, one on the diagonal and $1/k$ off it, gives a bound this one dominates, because that matrix is not row-stochastic and the polynomial is increasing in every entry.

The block-frequency bracket, (43). Two claims, and only the second needs an argument. First, the partition is a partition: every site lies in exactly one block, the frequent blocks contribute $\ell_{\mathcal{F}}$ by Equation 56, and each of the remaining B sites contributes a term lying in $[\underline{\lambda}, \bar{\lambda}]$ by definition, so summing gives (43). Second, $\underline{\lambda}$ and $\bar{\lambda}$ are computable in one pass. Run (54) in the log domain and carry an interval $[\underline{L}_u(i), \bar{L}_u(i)]$ in place of each partial. A leaf v observed in state x sends its parent the column $\mathbf{P}_x(t_v)$, so over every x that message lies entrywise in $[\min_j \mathbf{P}_{ij}(t_v), \max_j \mathbf{P}_{ij}(t_v)]$, which is the leaf case and the only place the data

enters. Above a leaf, $\sum_j \mathbf{P}_{ij}(t_v)L_v(j)$ and the product over children in (54) are non-decreasing in each $L_v(j)$, since transition probabilities and partials are non-negative, and so is the root sum (55); a non-decreasing function of arguments confined to intervals is confined to the interval its endpoints give, so the bounds propagate and $\underline{\lambda}, \bar{\lambda}$ bracket $\log p(x \mid \mathcal{T}, \boldsymbol{\tau}, \mathbf{Q})$ for every $x \in \mathcal{A}^n$ at once. The relaxation is that each child's extreme is taken independently of the others: the bracket is therefore sound, and no single column need attain either end. $\underline{\lambda} = -\infty$ where a branch of length zero makes some column impossible, which is that column's value and not a failure. The regression suite enumerates all $k^n = 256$ columns of the four-taxon fixture and finds none outside the bracket.

The mean-field bound, (44). For any distribution q on configurations, $\log Z = \mathbb{E}_q[-\mathcal{H}] + H(q) + \text{KL}(q \parallel p)$, and the divergence is non-negative (Gibbs' inequality). Restricting q to products and evaluating the expectation gives the right-hand side; the fixed-point iteration $q_i \propto \exp(h_i + \sum_{j \in \partial i} J_{ij}q_j)$ ascends it coordinate-wise, and the bound holds at every iterate, so a truncated iteration is a looser bound and never a wrong one [47, 50].

The spanning-tree bound, (45). $\log Z(\theta)$ is the log-normalizer of an exponential family and therefore convex in θ . For trees T with weights $\rho(T)$ and parameters $\theta(T)$ with $\sum_T \rho(T)\theta(T) = \theta$, Jensen gives $\log Z(\theta) \leq \sum_T \rho(T) \log Z(\theta(T))$. The implementation takes one tree per edge, built by breadth-first search from that edge so it contains it, weighted uniformly; an edge appearing in a fraction ρ_e of the trees carries J_e/ρ_e on each tree containing it and 0 elsewhere, so the weighted mean is J_e , and the fields are shared. Each $\log Z(\theta(T))$ is exact by the leaf-to-root recursion, in the log domain. The bound is tight when the graph is a tree [78].

The ground-state bracket, (46). $Z(\beta) = \sum_{\sigma} e^{-\beta \mathcal{H}(\sigma)}$ has k^N terms, each at most $e^{-\beta E_{\min}}$ and at least one equal to it, so $-\beta E_{\min} \leq \log Z(\beta) \leq N \log k - \beta E_{\min}$. Combining with $L \leq \log Z(\beta) \leq U$ and dividing by $\beta > 0$ gives the bracket; as β grows the $N \log k$ term is divided away, until the bounds on $\log Z$ themselves loosen.

The decoupled bounds, (47) and (48). $\mathcal{H}(\sigma) = -\sum_i h_i(\sigma_i) - \sum_{\langle i,j \rangle} J_{ij}[\sigma_i = \sigma_j]$ is a sum of terms, each bounded below by its own minimum over the configuration, and those minima are attained termwise by $\max_{\sigma} h_i(\sigma)$ and by $\max(J_{ij}, 0)$; no configuration need attain all of them at once, which is the slack. Exponentiating, $Z \leq k^N e^{-E_{\text{dec}}}$ as in (46), which is the right inequality. For the left, $J \geq 0$ makes every coupling term of each exponent non-negative, so dropping all of them lowers every summand of Z , and what remains factorizes over sites. Both are one pass over the sites and one over the edges, against one exact tree pass per edge for (45), which is why they are the bracket at the sizes that bound does not reach.

G Algorithms Cited From the Text

The algorithms each problem statement cites, grouped by what each is for: initializers, which choose where a search starts; samplers, which draw from a distribution at a temperature; optimizers, which move toward an optimum; and surrogates, which stand in for an expensive evaluation. They are collected here rather than at the point of use because most of them apply to more than one problem, and a statement of an algorithm belongs where it can be cited from each. Four algorithms stay in the body, because there the text derives them as it states them: forward simulation, Algorithm 1; neighbor joining, Algorithm 3; the Wolff move in an external field, Algorithm 4; and the data-sampled burn-in, Algorithm 5.

Algorithm 6 k -means++ seeding of K centres, Equation 49

- 1: draw c_1 uniformly from the observations $y_1, \dots, y_N$
 - 2: **for** $j = 1$ to $K - 1$ **do**
 - 3: $D(y) \leftarrow \min_{j' \leq j} |y - c_{j'}|$ for every observation y
 - 4: draw $c_{j+1} = y$ with probability $D(y)^2 / \sum_{y'} D(y')^2$
 - 5: **end for**
 - 6: **return** the centres $c_1, \dots, c_K$
-

Algorithm 7 The closest tree of a Hadamard spectrum, Equation 64

- 1: recode the alignment to two states and index the 2^{n-1} patterns by the set of taxa differing from a reference taxon
 - 2: $s \leftarrow$ the observed pattern frequencies; $q \leftarrow H^{-1} \log(Hs)$
 - 3: $\mathcal{S} \leftarrow \emptyset$
 - 4: **for** each split in decreasing order of its weight in q **do**
 - 5: **if** the split is compatible with every split in $\mathcal{S}$ **then**
 - 6: $\mathcal{S} \leftarrow \mathcal{S} \cup \{\text{split}\}$
 - 7: **end if**
 - 8: **stop** once $|\mathcal{S}| = n - 3$
 - 9: **end for**
 - 10: **return** the topology whose internal edges are $\mathcal{S}$, with the corresponding weights as branch lengths
-

Algorithm 8 Multi-start from a generator, at a declared budget

- 1: $\theta_0 \leftarrow$ the objective's own start, or that start tilted by a fixed alternating amount where it is stationary
 - 2: **for** $r = 1$ to R **do**
 - 3: $\theta_r \leftarrow \theta_0 + \varepsilon_r$, ε_r drawn from the passed-in generator
 - 4: **end for**
 - 5: fit from every θ_r , each fit charged to the shared budget of objective evaluations
 - 6: **return** every fit with its spread, and the best of them — never the best alone
-

G.1 Initializers

G.2 Samplers

Algorithm 9 One heat-bath sweep at inverse temperature β , Equation 73

- 1: **for** each node i of $\mathcal{G}$, in a fixed order **do**
 - 2: $p(\sigma) \propto \exp(\beta[\sum_{j \in \partial i} J_{ij}\delta(\sigma, \sigma_j) + h_i(\sigma)])$ for $\sigma \in \{1, \dots, q\}$
 - 3: draw $\sigma_i \sim p$ $\triangleright \beta \rightarrow \infty$ is the argmax, which is iterated conditional modes
 - 4: **end for**
 - 5: **return** the configuration after every node has been updated once
-

Algorithm 10 A cluster move at zero field, Edwards–Sokal bonds

- 1: refuse the move if any $J_{ij} < 0$, where $1 - e^{-\beta J_{ij}}$ is not a probability
 - 2: **for** each edge $\langle i, j \rangle$ with $\sigma_i = \sigma_j$ **do**
 - 3: place a bond with probability $1 - e^{-\beta J_{ij}}$
 - 4: **end for**
 - 5: $\mathcal{C} \leftarrow$ the bond-connected components; in Wolff, the one component containing a uniformly drawn root
 - 6: **for** each component in $\mathcal{C}$ **do**
 - 7: draw a new label ν uniformly from $\{1, \dots, q\}$
 - 8: recolour the component to ν , accepted with Equation 74 where a field is present
 - 9: **end for**
-

Algorithm 11 Parallel tempering over R replicas, Equation 85

- 1: $T_1 < \dots < T_R$; replica r draws from its own generator, spawned from one parent that draws only the exchange uniforms
 - 2: **for** each sweep of the budget **do**
 - 3: **for** $r = 1$ to R **do**
 - 4: advance replica r by one sweep at $\beta_r = 1/T_r$ $\triangleright$ Algorithm 9 or 10
 - 5: **end for**
 - 6: **for** each adjacent pair $(r, r+1)$ **do**
 - 7: exchange the configurations with probability $\min\{1, \exp[(\beta_r - \beta_{r+1})(E_r - E_{r+1})]\}$
 - 8: **end for**
 - 9: record the cold replica's state, and the acceptance of every pair
 - 10: **end for**
 - 11: **return** the lowest energy seen, and the cold replica's recorded weights, Equation 86
-

G.3 Optimizers

G.4 Surrogates

Algorithm 12 One block sweep over a chain, given the rest of the model

- 1: run the forward recursion Equation 51 over the chain, keeping every α_t
 - 2: draw $z_T \sim \alpha_T$, normalized
 - 3: **for** $t = T - 1$ down to 1 **do**
 - 4: draw z_t with probability $\propto \alpha_t(z_t) \mathbf{A}_{z_t z_{t+1}}$
 - 5: **end for**
 - 6: **return** the path, one exact draw from its conditional and not a sequence of single-site updates
-

Algorithm 13 Alpha expansion, Equation 72

- 1: $\sigma \leftarrow$ any labelling
 - 2: **repeat**
 - 3: **for** each label $\alpha \in \{1, \dots, q\}$ **do**
 - 4: build the two-state graph in which every site keeps σ_i or switches to α
 - 5: $\sigma' \leftarrow$ the labelling of its minimum s - t cut
 - 6: **if** $\mathcal{H}(\sigma') < \mathcal{H}(\sigma)$ **then** $\sigma \leftarrow \sigma'$
 - 7: **end if**
 - 8: **end for**
 - 9: **until** no expansion lowers the energy
 - 10: **return** $\sigma \triangleright$ within the constant of Equation 72 of the optimum, provided each expansion was solved exactly
-

Algorithm 14 Hill climbing over a structural neighbourhood

- 1: $x \leftarrow$ a start; $\text{score}(x) \leftarrow$ its full evaluation
 - 2: **repeat**
 - 3: $\mathcal{N} \leftarrow \mathcal{N}(x)$, each candidate scored at most once and charged to the budget
 - 4: $x' \leftarrow \arg \max_{y \in \mathcal{N}} \text{score}(y)$
 - 5: **if** $\text{score}(x') > \text{score}(x)$ **then** $x \leftarrow x'$
 - 6: **end if**
 - 7: **until** no neighbour improves, or the budget is spent
 - 8: **return** x , its score always a full evaluation and never a surrogate's
-

Algorithm 15 Expectation-maximization on a chain, or without one

- 1: **repeat**
 - 2: **E step:** the posteriors of Equation 52 by forward-backward; on a model with no chain, the responsibilities of Equation 8 by a division
 - 3: **M step:** re-estimate π and $\mathbf{A}$ in closed form from the posterior counts, and the emission parameters by the family's own rule — itself an optimization where no closed form exists
 - 4: assert the likelihood did not decrease
 - 5: **until** the increase falls below a tolerance, or the iteration cap is reached
-

Algorithm 16 Sum-product decoding in the log domain, Equation 78

- 1: $m_{b \rightarrow i} \leftarrow 0$ for every edge; L_i from Equation 21
 - 2: **for** iteration 1 to the cap **do**
 - 3: $m_{i \rightarrow a} \leftarrow \text{clip}(L_i + \sum_{b \in \partial i \setminus a} m_{b \rightarrow i})$ for every edge, at once
 - 4: $m_{a \rightarrow i} \leftarrow 2 \operatorname{artanh} \prod_{j \in \partial a \setminus i} \tanh(m_{j \rightarrow a}/2)$, or Equation 79 for min-sum
 - 5: $\tilde{L}_i \leftarrow L_i + \sum_{b \in \partial i} m_{b \rightarrow i}$; $\hat{c}_i \leftarrow \mathbf{1}[\tilde{L}_i < 0]$
 - 6: **stop** if every bit is decided and $\mathbf{H}\hat{\mathbf{c}} = 0$
 - 7: **end for**
 - 8: **return** $\hat{\mathbf{c}}$, and whether the syndrome check was reached — a failure here is the event a block error rate counts
-

Algorithm 17 Turbo decoding: the serial extrinsic exchange, Equation 82

- 1: split the received ratios into L^s, L^{p1}, L^{p2} and the second encoder's tail, Equation 34
 - 2: $L^a \leftarrow 0$ over the first chain's $K + m$ steps
 - 3: **for** iteration 1 to the cap **do**
 - 4: $\tilde{L}^1 \leftarrow \text{BCJR}(L^s, L^{p1}, L^a)$ by Equation 81; $L^{e1} \leftarrow \tilde{L}^1 - L^s - L^a$
 - 5: $\tilde{L}^2 \leftarrow \text{BCJR}(\Pi L^s, L^{p2}, \Pi L^{e1})$, the message positions interleaved and the tails left alone
 - 6: $L^a \leftarrow \Pi^{-1}(\tilde{L}^2 - \Pi L^s - \Pi L^{e1})$
 - 7: **stop** if the two chains' hard decisions on u agree
 - 8: **end for**
 - 9: **return** $\hat{u}_i = \mathbf{1}[(\Pi^{-1}\tilde{L}^2)_i < 0]$, and whether the two agreed — which is a stopping rule and not a certificate, since the graph has cycles
-

Algorithm 18 Successive cancellation on the butterfly, Equation 39

- 1: **function** $\text{DECODE}(L, \mathcal{F})$ $\triangleright L$: the ratios this subtree sees; $\mathcal{F}$: its frozen mask
 - 2: **if** $|L| = 1$ **then**
 - 3: **return** 0 if $\mathcal{F}_0$, else $\mathbf{1}[L_0 < 0]$
 - 4: **end if**
 - 5: $a, b \leftarrow$ the first and second halves of L
 - 6: $\hat{u}^\ell \leftarrow \text{DECODE}(f(a, b), \text{first half of } \mathcal{F})$
 - 7: $\hat{u}^r \leftarrow \text{DECODE}(g(a, b, \hat{c}^\ell), \text{second half of } \mathcal{F})$
 - 8: **return** $(\hat{u}^\ell, \hat{u}^r)$ and the codeword bits $(\hat{c}^\ell \oplus \hat{c}^r, \hat{c}^r)$
 - 9: **end function**
 - 10: **return** the message bits of $\hat{u}$ on $\mathcal{I}$
-

Algorithm 19 Successive-cancellation list decoding, L prefixes [77]

- 1: one path, metric 0
 - 2: **for** each input i in order, through the recursion of Algorithm 18 **do**
 - 3: **if** i is frozen **then**
 - 4: every path takes $\hat{u}_i = 0$ and pays $\log(1 + e^{-(1-2\hat{u}_i)L_i})$
 - 5: **else**
 - 6: every path forks into $\hat{u}_i \in \{0, 1\}$, each paying the same penalty
 - 7: keep the L paths of smallest metric, and reorder the arrays of every surviving path to match
 - 8: **end if**
 - 9: **end for**
 - 10: **return** the message of the surviving path of smallest metric
-

Algorithm 20 Ranking a neighbourhood by a surrogate, Section 20

- 1: $\mathcal{N} \leftarrow \mathcal{N}(x)$
 - 2: $\hat{s}(y) \leftarrow$ the surrogate's value at every $y \in \mathcal{N}$, one cheap pass each
 - 3: **if** the surrogate is an analytic bound **then**
 - 4: discard every y whose bound cannot beat the incumbent $\triangleright$ certified: no violation is permitted on any structure an oracle can score
 - 5: **else**
 - 6: shift $\hat{s}$ by a quantile of the held-out residuals, to the coverage the predictor claims
 - 7: **end if**
 - 8: evaluate in full the best κ candidates by $\hat{s}$, and the accepted move in full whatever its rank
 - 9: **return** the accepted move, with a reported score that is always a full evaluation
-

Algorithm 21 The block-frequency bracket, Equation 43

- 1: cut the L columns into $\lfloor L/N \rfloor$ blocks of N consecutive sites, and a shorter remainder if $N \nmid L$ $\triangleright$ a remainder is never equal to a full block
 - 2: count the occurrences of each distinct block; $\mathcal{F} \leftarrow$ those occurring at least m times
 - 3: $\ell_{\mathcal{F}} \leftarrow$ Equation 56 over the columns of $\mathcal{F}$, each weighted by its block's count $\triangleright$ a column shared by two frequent blocks is one pattern with the counts added
 - 4: $B \leftarrow L -$ the sites $\mathcal{F}$ covers
 - 5: $\underline{\lambda}, \bar{\lambda} \leftarrow$ Equation 54 with each leaf's message replaced by the extremes of its transition row, one pass, no data read
 - 6: **return** $[\ell_{\mathcal{F}} + B\underline{\lambda}, \ell_{\mathcal{F}} + B\bar{\lambda}]$, and $\ell_{\mathcal{F}} \cdot L/(L - B)$ as the point prediction that ranks
-